

SUPSI

Rilevamento dati personali in documenti non strutturati aziendali

Studente/i

Algisi Gabriele

Relatore

Gay Mario

Correlatore

Consoli Angelo

Committente

Gay Mario

Corso di laurea

Ingegneria Informatica

Modulo

C11303

Anno

2025-2026

Data

16 agosto 2026

Indice

1	Introduzione	1
1.1	Riassunto / Abstract	1
1.2	Progetto assegnato	1
1.3	Contesto	2
1.4	Motivazione	3
1.5	Obiettivi del progetto	3
2	Analisi dei requisiti	5
2.1	Requisiti funzionali	5
2.1.1	Acquisizione dell'input	5
2.1.2	Estrazione del testo	5
2.1.3	Rilevamento delle PII	6
2.1.4	Ingestione del/i ROPA	6
2.1.5	Confronto dei risultati di analisi con il ROPA	6
2.1.6	Rilevamento delle anomalie di conformità	7
2.1.7	Generazione report per DPO	7
2.1.8	Persistenza e aggiornamento del registro delle PII rilevate	7
2.1.9	Feedback loop	8
2.1.10	Analisi contestuale della struttura del filesystem	8
2.1.11	Impieghi selettivi dell'AI generativa	8
2.2	Requisiti normativi	9
2.2.1	Conformità ai principi di limitazione delle finalità e della conservazione	9
2.2.2	Riferimento al registro delle attività di trattamento come fonte normativa	10
2.2.3	Copertura delle categorie di dati degni di particolare attenzione	10
2.2.4	Posizionamento rispetto all'art. 32(1)(d) GDPR	11
2.2.5	Esclusione delle basi giuridiche dal perimetro di analisi automatica	11
2.3	Requisiti operativi	12
2.3.1	Elaborazione locale dei dati	12
2.3.2	Vincolo sui modelli AI impiegabili	12
2.3.3	Facilità di installazione e aggiornamento	13

2.3.4	Estensibilità delle categorie e dei detector	13
2.3.5	Minimizzazione dei dati trattati dal sistema stesso	14
2.3.6	Efficienza delle rianalisi periodiche	14
2.3.7	Operatività senza addestramento iniziale (zero-training)	15
2.4	Requisiti di qualità	15
2.4.1	Prestazioni su larga scala	15
2.4.2	Precisione del rilevamento	16
2.4.3	Tracciabilità delle anomalie	16
2.5	Requisiti di interfaccia	16
2.5.1	Interfaccia web come modalità primaria	16
2.5.2	Interfaccia a riga di comando (CLI)	17
2.5.3	Fruizione del report	17
3	Stato dell'arte	19
3.1	Il problema: dalla <i>data anonymization</i> alla <i>data governance</i>	19
3.1.1	Il problema tecnico: ambiguità contestuale e frammentazione dei domini	19
3.1.2	La verifica automatizzata della conformità	20
3.2	Background tecnologico	20
3.2.1	Supervised vs unsupervised	20
3.2.2	Pre-tuning vs fine-tuning	21
3.2.3	Zero-shot learning	22
3.3	Approcci tecnici di PII detection	23
3.3.1	Recognition rule-based	23
3.3.2	Recognition con NLP - NER	23
3.3.3	Recognition con AI generativa	23
3.3.4	Limitazioni nella soluzione	24
3.3.5	Soluzioni ibride	24
3.3.6	Confronto empirico degli approcci	24
3.3.6.1	Prestazioni dei sistemi rule-based	25
3.3.6.2	Prestazioni dei modelli NER supervisionati	25
3.3.6.3	Prestazioni dei modelli NER zero-shot	25
3.3.6.4	Prestazioni dell'AI generativa	26
3.3.6.5	Convergenza verso la pipeline ibrida	26
3.3.6.6	Matrice di confronto sintetica	27
3.4	Panoramica dei sistemi esistenti	27
3.4.1	Sistemi open-source	27
3.4.2	Sistemi commerciali	28
3.5	Sintesi del gap e posizionamento del progetto	29
3.5.0.0.1	Posizionamento del progetto	29

4	Analisi dei rischi	31
4.1	Rischi di progetto	31
4.1.1	Disponibilità e qualità dei dataset di valutazione	31
4.1.2	Complessità dell'integrazione multi-approccio	32
4.1.3	Costo computazionale e tempi di elaborazione	32
4.1.4	Dipendenza da modelli e librerie di terze parti	33
4.1.5	Dilatazione del perimetro (<i>scope creep</i>)	33
4.1.6	Competenze richieste e curva di apprendimento	34
4.2	Rischi di prodotto	34
4.2.1	Accuratezza insufficiente nella detection delle PII	34
4.2.2	Falsi positivi e impatto sull'usabilità	35
4.2.3	Falsi negativi e mancata rilevazione di PII critiche	35
4.2.4	Generalizzazione cross-domain (<i>domain-silo problem</i>)	36
4.2.5	Supporto multilingue e multilocalizzazione	36
4.2.6	Scalabilità su filesystem di grandi dimensioni	37
4.2.7	Qualità e completezza dei ROPA in input	38
4.2.8	Conformità normativa e aggiornamento dei framework legali	38
4.2.9	Sicurezza e protezione dei dati durante l'analisi	39
4.3	Valutazione dei rischi	39
4.3.1	Criteri di classificazione	39
4.3.2	Matrice probabilità–impatto	40
4.4	Strategie di mitigazione	41
4.4.0.0.1	Disaccoppiamento tramite contratti e configurazione.	41
4.4.0.0.2	Asimmetria dichiarata fra i due tipi di errore.	41
4.4.0.0.3	Minimizzazione come proprietà strutturale.	41
4.4.0.0.4	Misura riproducibile su corpus costruiti a questo scopo.	42
4.4.0.0.5	Contenimento del costo di esecuzione.	42
4.4.0.0.6	Esposizione delle lacune anziché loro occultamento.	42
4.4.0.0.7	Governo esplicito del perimetro.	42
4.4.0.0.8	Rischi accettati.	42
5	Studi effettuati	43
5.1	Uso dell'AI in rapporto al suo impatto ambientale ed economico	43
5.1.1	Il divario energetico fra modelli discriminativi e generativi	44
5.1.2	Il quadro comparativo	44
5.1.3	Conseguenza per l'architettura del sistema	44
5.2	Costo di addestramento e costo di inferenza	45
5.3	Riutilizzo di un motore di detection esistente: valutazione di Microsoft Presidio	46
5.3.0.0.1	Profilo hardware.	47

5.4	Scelta dello Small Language Model per gli impieghi selettivi dell'AI	47
5.4.1	Criteri di selezione	48
5.4.2	Fattibilità dell'inferenza su CPU	48
5.4.3	Modelli candidati	49
5.4.3.0.1	Qwen2.5 (Alibaba)	49
5.4.3.0.2	Llama 3.2 1B/3B (Meta)	49
5.4.3.0.3	Phi-3-mini / Phi-4-mini (Microsoft)	50
5.4.3.0.4	Gemma 3 (Google)	50
5.4.3.0.5	SmolLM2 (Hugging Face)	50
5.4.3.0.6	IBM Granite 3.x	50
5.4.4	Analisi per task	50
5.4.4.0.1	Task 1 — ROPA verso categorie	50
5.4.4.0.2	Task 2 — Analisi di documenti campione	50
5.4.4.0.3	Task 3 — Struttura del file system	50
5.4.5	Confronto e conclusione	51
5.4.6	Energia, efficienza e limiti	51
5.4.6.0.1	Limiti delle evidenze raccolte	52
6	Architettura del sistema	53
6.1	Ingestione e modello dati del ROPA	56
6.1.1	Strutturazione a monte o delega all'AI	56
6.1.2	Il registro-tipo CNIL come schema di riferimento	56
6.1.3	Normalizzazione in un modello relazionale	57
6.1.4	Ruolo circoscritto dell'AI: normalizzazione, non strutturazione	58
6.1.5	Punti aperti di progettazione	58
6.1.5.0.1	Allineamento nLPD–GDPR	58
6.1.5.0.2	ROPA non strutturato	59
6.2	Un blocco previsto e non realizzato: l'analisi della struttura (B2)	59
6.3	Livello di estrazione del testo	59
6.3.0.0.1	Uno scope a stadi	59
6.3.0.0.2	Estrazione e rilevamento come <i>concern</i> separati	60
6.4	Livello di rilevamento ibrido delle PII	60
6.4.1	Due sorgenti complementari dietro un contratto unico	60
6.4.1.0.1	Estendere il rilevamento senza scrivere codice	61
6.4.2	Il merge: unificazione e grado di certezza	61
6.4.2.0.1	Principio recall-first	61
6.4.2.0.2	Un solo modello dato	62
6.4.3	Il ramo generativo: un innesto predisposto	62
6.5	Persistenza del registro delle PII	62

6.5.1	Il requisito di tracciamento temporale	63
6.5.2	Approcci valutati	63
6.5.2.1	Opzione A — Snapshot completi	63
6.5.2.2	Opzione B — Event sourcing puro	63
6.5.2.3	Opzione C — Changelog ibrido delta-based	64
6.5.3	Motivazione della scelta	65
6.5.4	Modello dati concettuale	65
6.6	Associazione e verifica di conformità	67
6.6.1	Associazione documento–trattamento (B6)	67
6.6.2	Il confronto dichiarato–rilevato e il suo vocabolario (B7)	68
6.6.3	Verifica della conservazione	69
6.7	Interfaccia e restituzione al DPO	69
6.7.0.0.1	Una sola applicazione, due interfacce.	70
6.7.0.0.2	Consultare non deve modificare.	70
6.7.0.0.3	Riferimenti, mai valori.	70
6.7.0.0.4	Due letture, per documento e per corpus.	70
7	Implementazione	71
7.1	Ingestione del ROPA (blocco B1)	71
7.1.1	Il formato sorgente: il template CNIL reale	71
7.1.2	Lettura: un'unica forma per ODS ed Excel	71
7.1.3	Interpretazione: guidata dalle etichette, non dalle posizioni	72
7.1.4	Persistenza e mappatura assistita	72
7.2	Tecnologia di persistenza	73
7.3	Dati persistiti nei database	74
7.3.1	Registro ROPA (blocco B1)	74
7.3.1.0.1	processing_activity — l'attività di trattamento (livello 1).	74
7.3.1.0.2	macro_category — il blocco di categorie con la sua conservazione (livello 2).	74
7.3.1.0.3	declared_category — la categoria dichiarata, ponte verso il rilevamento (livello 3).	75
7.3.2	Registro delle PII rilevate (blocco B5)	75
7.3.2.0.1	registry_document — il documento analizzato.	75
7.3.2.0.2	registry_scan — una scansione, cioè un'osservazione nel tempo.	76
7.3.2.0.3	registry_pii_instance — la singola PII, stato corrente e mutabile.	76

7.3.2.0.4	registry_pii_change — il log append-only delle variazioni.	77
7.3.2.0.5	registry_folder_rule — la regola cartella→trattamento (blocco B6).	77
7.4	Realizzazione del livello di rilevamento	78
7.4.1	Estensibilità config-driven: aggiungere categorie e pattern	78
7.5	Deployment containerizzato e portabilità	78
7.5.1	Portabilità e isolamento delle dipendenze	79
7.5.2	Architettura di deployment	79
7.5.3	Contratti e direzione	80
8	Test e validazione	81
8.1	Validazione del livello di rilevamento	81
8.1.0.0.1	Metodo.	81
8.1.0.0.2	Risultati.	81
8.1.0.0.3	Limiti.	81
9	Risultati e discussione	83
10	Sviluppi futuri	85
10.1	Analisi contestuale della struttura del file system	85
10.2	Canale di segnalazione dai dipendenti	86
10.3	Estensione dei formati documentali	88
10.4	Quadro riassuntivo	88
10.5	Limiti dichiarati e direzioni residue	89
11	Diario degli sviluppi (<i>work in progress</i>)	91
11.1	Data di riferimento del documento e provenienza della stima	91
11.2	Due orologi distinti nel registro dei documenti	92
11.3	Il silenzio non è conformità: conservazione non verificabile	93
11.4	Dalla scheda del documento alla vista d'insieme	94
11.5	Rianalizzare solo ciò che è cambiato	95
12	Punti aperti (<i>work in progress</i>)	97
12.1	Decisioni progettuali aperte	97
12.1.1	Requisiti da confermare	97
12.1.1.0.1	PA-1 — Aggregazione delle PII frammentate.	97
12.1.1.0.2	PA-2 — Costruzione di una knowledge base delle entità.	97
12.1.1.0.3	PA-12 — Affidabilità della data del documento.	97
12.1.2	Modello dati del ROPA	98

12.1.2.0.1	PA-3 — Granularità della corrispondenza categorie – PII.	98
12.1.2.0.2	PA-4 — Allineamento nLPD–GDPR.	98
12.1.2.0.3	PA-5 — ROPA non strutturato.	98
12.1.3	Persistenza del registro PII	98
12.1.3.0.1	Snapshot o event-logger	98
12.1.3.0.2	PA-6 — Registrazione della «scomparsa» di una PII: esplicita o implicita?	98
12.1.3.0.3	PA-7 — Significato dell'evento «modifica».	98
12.1.3.0.4	PA-8 — Vocabolario chiuso dei tipi di variazione.	98
12.1.4	Scelte implementative	99
12.1.4.0.1	PA-9 — Tecnologia di persistenza.	99
12.1.4.0.2	PA-10 — Riutilizzo di Microsoft Presidio come livello di detection.	99
12.1.5	Metodologia di detection	99
12.1.5.0.1	PA-11 — Strategia per il modello NER.	99
12.2	Contenuti ancora da redigere	99

Capitolo 1

Introduzione

1.1 Riassunto / Abstract

Qui ci sarà l'abstract...

1.2 Progetto assegnato

Le aziende e le organizzazioni soggette alla nLPD (nuova Legge federale sulla protezione dei dati) sono tenute a sapere dove si trovano i dati personali che trattano, a quale trattamento riportato del registro dei trattamenti sono riconducibili e se sono ancora entro i termini di conservazione dichiarati nel registro. Nella pratica però, gran parte di questi dati è dispersa in documenti non strutturati — ad esempio PDF, Word, Excel, scansioni — senza che l'organizzazione ne abbia visibilità sistematica.

Gli strumenti disponibili sul mercato, sia quelli più abordabili (es. PII Crawler, PII Tools) sia quelli enterprise (es. BigID, Varonis), sono progettati per garantire la conformità con normative europee (GDPR) o statunitensi (es.: CCPA) e non necessariamente gestiscono le categorie di dati specifiche della nLPD svizzera senza configurazioni e validazioni specifiche. Oltre a ciò, solo alcuni prodotti enterprise più avanzati integrano il registro dei trattamenti come input normativo configurabile. Di conseguenza la verifica del superamento del periodo di retention è spesso possibile solo tramite una categorizzazione molto precisa all'origine.

Il progetto propone lo sviluppo di un sistema in grado di evidenziare il divario tra le attività di trattamento dichiarate nel registro e l'effettiva gestione dei documenti aziendali, anche utilizzando tecniche NLP e AI. Per ciascun documento analizzato, il sistema deve identificare eventuali dati personali presenti (Personally Identifiable Information, PII), associare il documento alla corrispondente attività di trattamento combinando diversi indicatori (es. posizione nel file system, contenuto, PII identificati) e verificare la conformità al periodo di conservazione dichiarato nel registro. I documenti che hanno superato il loro periodo

di conservazione e le PII [1] che non possono essere collegate ad alcuna operazione di trattamento registrata devono essere segnalati come anomalie.

1.3 Contesto

La protezione dei dati personali costituisce oggi uno dei nodi centrali della governance organizzativa, tanto sul piano giuridico quanto su quello operativo. In Europa, il regolamento (UE) 2016/679 (GDPR) [2] ha introdotto un corpus di obblighi sostanziali per i titolari del trattamento, tra cui il principio di limitazione della conservazione sancito dall'art. 5(1)(e), secondo cui i dati personali devono essere conservati "in una forma che consenta l'identificazione degli interessati per un arco di tempo non superiore al conseguimento delle finalità per le quali sono trattati". In Svizzera, la nuova Legge federale sulla protezione dei dati (nLPD, RS 235.1, in vigore dal 1° settembre 2023) ha recepito principi analoghi: l'art 6(4) dispone che "i dati personali sono distrutti o resi anonimi appena non sono più necessari per lo scopo del trattamento", stabilendo un legame diretto tra la finalità dichiarata e il ciclo di vita del dato. Entrambe le normative impongono inoltre la tenuta di un registro delle attività di trattamento (ROPA, Record of Processing Activities). Il GDPR, all'art. 30(1)(f), richiede che il registro contenga, "ove possibile, i termini ultimi previsti per la cancellazione delle diverse categorie di dati"; la nLPD [3], all'art. 12(2)(e), prevede analogamente l'indicazione della durata di conservazione dei dati personali o dei criteri per determinarne la durata. Il ROPA costituisce dunque il documento normativo di riferimento per qualsiasi verifica di conformità: è in esso che l'organizzazione dichiara cosa tratta, perché, con quale base giuridica e per quanto tempo.

I due ordinamenti divergono tuttavia su un punto strutturalmente rilevante per qualsiasi sistema di verifica automatica. Il GDPR enumera positivamente le basi giuridiche lecite all'art. 6(1), subordinando la liceità del trattamento alla sussistenza di almeno una di esse. La nLPD adotta invece un approccio invertito: il trattamento è lecito per default in quanto esercizio della libertà d'azione dei privati, ma diventa illecito se lede ingiustificatamente la personalità dell'interessato ai sensi dell'art. 30. La giustificazione può provenire dal consenso, da un interesse preponderante o dalla legge, secondo il catalogo dell'art. 31. Questa differenza strutturale impone che un sistema di analisi della conformità tenga conto del framework giuridico applicabile, poiché il criterio con cui si valuta l'esistenza di una base legittima non è identico nei due contesti.

Nella pratica organizzativa, la conformità dichiarata nel registro tende a divergere dalla situazione reale dei dati. Gran parte dell'informazione trattata da un'organizzazione è incorporata in documenti non strutturati (file PDF, Word, Excel, messaggi di posta elettronica...) distribuiti su file system locali, server condivisi e servizi cloud. Questi documenti non recano metadati normalizzati che consentono di associarli automaticamente a un'attività di trattamento specifica, né di verificarne la conformità al periodo di retention dichiarato. Il risultato è

un divario strutturale tra ciò che il registro descrive e ciò che esiste effettivamente nei sistemi informativi dell'organizzazione.

Questo divario assume rilevanza giuridica diretta. Il GDPR all'art. 32(1)(d) qualifica come misura di sicurezza obbligatoria una procedura per testare, verificare e valutare regolarmente l'efficacia delle misure tecniche e organizzative al fine di garantire la sicurezza del trattamento. La nLPD, pur con formulazione più sintetica, impone all'art. 8(1) che il titolare e il responsabile del trattamento garantiscano, mediante appropriati provvedimenti tecnici e organizzativi, che la sicurezza dei dati personali sia adeguata al rischio. Un sistema in grado di rilevare automaticamente i dati personali presenti nei documenti aziendali e di verificarne la coerenza con il registro può essere inteso come implementazione tecnica di quest'obbligo.

1.4 Motivazione

Il divario descritto nella sezione precedente tra le attività di trattamento dichiarate nel registro e la gestione effettiva dei documenti non è soltanto un problema teorico: ha conseguenze dirette sulla capacità di un'organizzazione di dimostrare conformità. Un Data Protection Officer che debba rispondere a una richiesta dell'autorità di controllo, o semplicemente verificare se un'attività di trattamento è ancora coperta da una fase giuridica valida, si trova nella pratica privo di uno strumento che colleghi sistematicamente il contenuto dei documenti aziendali al registro. La verifica, quando viene fatta è perlopiù manuale, parziale e non ripetibile nel tempo.

Come verrà approfondito nell'analisi degli strumenti esistenti ([LINK A SEZIONE]), il panorama commerciale attuale non risolve in modo sufficiente questo problema per il contesto svizzero, lasciando aperto uno spazio applicativo concreto. Allo stesso tempo, come discusso nella motivazione architettuale ([LINK A SEZIONE]), la maturità raggiunta dalle tecniche di elaborazione del linguaggio naturale rende oggi percorribile, a un costo computazionale sostenibile, un approccio che pochi anni fa avrebbe richiesto un investimento sproporzionato.

Il progetto nasce dunque dall'intersezione di tre fattori: un obbligo normativo concreto, non pienamente automatizzabile con gli strumenti esistenti; uno spazio applicativo non coperto in modo adeguato dal mercato; e una maturità tecnologica che rende oggi realistico costruire un sistema di verifica della conformità basato sul confronto tra registro dichiarato e contenuto documentale effettivo.

1.5 Obiettivi del progetto

Per ora solamente obiettivi del progetto, valutare se riformulare

Il progetto intende produrre un sistema che renda visibile lo scarto tra le attività di trattamento dichiarate nel registro e la gestione effettiva dei dati nei documenti aziendali. In pratica, il prototipo dovrà:

- rilevare le categorie di dati personali presenti in documenti non strutturati, sulla base delle categorie canoniche della nLPD e di quelle dichiarate nel registro dei trattamenti;
- associare i documenti al trattamento corrispondente combinando più segnali;
- verificare la conformità ai periodi di retention dichiarati nel registro;
- segnalare categorie di dati personali non dichiarate nel registro e trattamenti non censiti;
- produrre un report leggibile dal Data Protection Officer dell'organizzazione.

Capitolo 2

Analisi dei requisiti

La struttura di questo capitolo si ispira allo standard ISO/IEC/IEEE 29148:2018 - Systems and Software Engineering - Life cycle processes - Requirements engineering [4]. Ogni requisito è enunciato in forma verificabile e, dove necessario, ne è motivata l'origine normativa o progettuale. Gli elenchi puntati sono impiegati esclusivamente per enumerare obblighi distinti; le soglie quantitative e i criteri di accettazione sperimentali sono rimandati al Capitolo 8.

2.1 Requisiti funzionali

2.1.1 Acquisizione dell'input

Il sistema deve poter analizzare come input un insieme di file selezionati, una cartella oppure un intero filesystem. L'acquisizione deve avvenire senza presupporre una struttura predefinita dell'albero di directory e deve supportare l'elaborazione ricorsiva delle sottocartelle.

L'utente deve poter specificare percorsi multipli e pattern di esclusione, in modo da limitare la scansione ai soli percorsi rilevanti.

2.1.2 Estrazione del testo

Il sistema deve estrarre il contenuto testuale dai documenti non strutturati [5] indipendentemente dal formato di partenza, supportando almeno i documenti impaginati in formato PDF, i documenti di videoscrittura, i fogli di calcolo e il testo semplice.

L'estrazione deve preservare, dove il formato di partenza lo consente, i metadati strutturali necessari a contestualizzare le PII rilevate e a tracciarne la posizione nel report finale (cfr. 2.4.3): almeno la suddivisione in pagine, l'organizzazione tabellare e le intestazioni di sezione.

2.1.3 Rilevamento delle PII

Il sistema deve identificare le occorrenze di dati personali nel testo estratto tramite una combinazione di meccanismi complementari — pattern deterministici, modelli statistici e tecniche di elaborazione del linguaggio naturale [6, 7, 8] — la cui composizione esatta è oggetto di definizione architetturale.

Il rilevamento deve coprire almeno i dati identificativi, i dati di contatto, gli identificatori numerici nazionali e i dati degni di particolare protezione ai sensi dell'art. 5 lett. c nLPD [3], corrispondenti alle categorie particolari di dati personali dell'art. 9 GDPR [2].

2.1.4 Ingestione del/i ROPA

Il sistema deve accettare in input uno o più registri delle attività di trattamento (ROPA) in formato strutturato, da utilizzare come riferimento normativo autoritativo per il confronto con le PII rilevate nei documenti. Il ROPA costituisce, ai sensi dell'art. 30 GDPR e dell'art. 12 nLPD, il documento in cui l'organizzazione dichiara le proprie attività di trattamento: senza la sua ingestione il sistema non potrebbe effettuare alcuna verifica di conformità.

Il sistema deve supportare almeno i formati di foglio di calcolo ODS e XLSX, nei quali i registri sono comunemente redatti e distribuiti dalle autorità di controllo. Per ogni attività di trattamento, il formato di ingestione deve includere almeno i seguenti campi:

- categorie di dati personali previste;
- finalità del trattamento;
- periodo di conservazione dichiarato, o i criteri per la sua determinazione;
- categorie di interessati;
- eventuali categorie di destinatari.

Il sistema deve segnalare all'utente, in fase di ingestione, i campi obbligatori mancanti o i valori non interpretabili, senza interrompere l'elaborazione per i record validi. Il requisito si considera soddisfatto quando il sistema importa correttamente un registro di riferimento in ciascuno dei formati supportati, estraendone tutti i campi previsti e segnalando i record incompleti.

2.1.5 Confronto dei risultati di analisi con il ROPA

Il sistema deve confrontare le entità e le categorie di dati personali effettivamente rilevate nei documenti con quanto dichiarato nel/i ROPA, verificando la coerenza tra dati trovati e dati dichiarati.

Il confronto deve produrre, per ciascun documento analizzato, un'associazione — anche parziale o nulla — con una o più attività di trattamento del ROPA, secondo criteri ed euristiche definiti in fase di progettazione architettuale.

2.1.6 Rilevamento delle anomalie di conformità

Il sistema deve identificare e classificare, sulla base del confronto di cui alla sezione 2.1.5, le seguenti tipologie di anomalie:

- categorie di PII rilevate ma non dichiarate nel ROPA per l'attività di trattamento associata;
- dati personali privi di una base giuridica o di un'attività di trattamento associabile ("PII orfane");
- violazioni dei periodi di conservazione dichiarati nel ROPA, ossia documenti contenenti PII oltre il termine di retention dichiarato.

Ogni anomalia rilevata deve essere classificata secondo una tipologia predefinita e associata a un livello di gravità.

2.1.7 Generazione report per DPO

Il sistema deve produrre, come esito dell'analisi, un report di conformità destinato al Data Protection Officer (DPO), che raccolga le anomalie rilevate e sia consultabile senza competenze tecniche.

Il report deve includere, per ciascuna anomalia, la tipologia, i documenti coinvolti, l'attività di trattamento di riferimento ove presente e la motivazione del rilevamento. Deve inoltre essere organizzato **per documento**, in modo che il DPO possa passare dal quadro d'insieme del corpus documentale al dettaglio del singolo file, delle sue PII e del suo verdetto.

Il report deve descrivere le PII rilevate come *riferimenti* — categoria, posizione, provenienza del rilevamento — e non deve in nessun caso riportare il valore del dato personale rilevato (cfr. 2.3.5).

2.1.8 Persistenza e aggiornamento del registro delle PII rilevate

Il sistema deve mantenere un registro interno persistente delle PII rilevate, che associ a ciascuna occorrenza il file sorgente, la posizione all'interno del documento, la categoria di PII e un identificatore temporale dell'analisi.

Il registro deve consentire il confronto tra esecuzioni successive sullo stesso corpus documentale, al fine di rilevare:

- PII precedentemente segnalate che risultano rimosse o modificate nel documento sorgente;
- nuove PII introdotte da documenti aggiunti o modificati dopo l'ultima analisi;
- documenti eliminati dal filesystem che contenevano PII precedentemente censite.

Il formato del registro deve essere strutturato e interrogabile meccanicamente, così da supportare sia la consultazione diretta sia l'integrazione con il report di conformità.

2.1.9 Feedback loop

Il sistema deve mettere a disposizione un canale di segnalazione che consenta al personale dell'organizzazione di notificare al DPO le PII presenti nei documenti aziendali e sfuggite al rilevamento automatico.

Le segnalazioni raccolte devono confluire nel registro interno (cfr. 2.1.8) mantenendo distinta la provenienza manuale da quella automatica, in modo che il DPO possa valutarle e, ove pertinenti, utilizzarle per estendere le regole di rilevamento (cfr. 2.3.4).

2.1.10 Analisi contestuale della struttura del filesystem

Il sistema deve considerare la posizione di ciascun documento all'interno della gerarchia del filesystem come segnale contestuale per l'associazione all'attività di trattamento. La struttura delle cartelle — nomi, profondità, organizzazione logica — può fornire indicazioni sulla finalità del trattamento e sulla legittimità della presenza di determinati dati personali in una posizione specifica.

Il sistema deve segnalare nel report come anomalia posizionale il documento le cui categorie di PII non risultano coerenti con la finalità della collocazione che lo ospita, in quanto indicativo di una possibile violazione del principio di limitazione delle finalità.

2.1.11 Impieghi selettivi dell'AI generativa

Il sistema può impiegare modelli di AI generativa per attività mirate che beneficiano della comprensione semantica profonda, senza richiederne l'applicazione sistematica a ogni documento del corpus. Tali impieghi sono circoscritti a:

- **arricchimento dei ROPA:** qualora il registro fornito in input contenga descrizioni sintetiche o generiche delle categorie di dati, il sistema può utilizzare un modello linguistico per espanderle e normalizzarle [9], migliorando la granularità del confronto con le PII rilevate;

- **analisi campionata:** il sistema può sottoporre a un modello linguistico un sottoinsieme rappresentativo di documenti per ottenerne un'analisi contestuale approfondita, identificando PII deducibili dal contesto che sfuggirebbero ai metodi deterministici e statistici;
- **analisi della struttura del filesystem:** il sistema può utilizzare un modello linguistico per interpretare la semantica dei percorsi e dei nomi di cartella, al fine di rilevare incoerenze tra la posizione del documento e il suo contenuto (cfr. 2.1.10).

Questi impieghi devono essere documentati e giustificati da studi comparativi (cfr. 5) che ne attestino il valore aggiunto rispetto al costo computazionale sostenuto.

2.2 Requisiti normativi

2.2.1 Conformità ai principi di limitazione delle finalità e della conservazione

Il sistema deve supportare la verifica di conformità ai seguenti principi normativi:

- **limitazione della finalità** (art. 5(1)(b) GDPR [2]; art. 6 cpv. 3 nLPD [3]): i dati personali devono essere raccolti per finalità determinate, esplicite e legittime, e non ulteriormente trattati in modo incompatibile con tali finalità;
- **limitazione della conservazione** (art. 5(1)(e) GDPR [2]; art. 6 cpv. 4 nLPD [3]): i dati personali devono essere conservati per un arco di tempo non superiore al conseguimento delle finalità per le quali sono trattati.

Questi due principi costituiscono il fondamento giuridico dell'intera logica di verifica del sistema: il confronto tra le PII rilevate nei documenti e quanto dichiarato nel ROPA è, nella sua essenza, una verifica che i dati trattati siano coerenti con le finalità dichiarate e che non siano conservati oltre i termini previsti.

La verifica deve operare su due assi:

1. **asse delle finalità:** per ciascun documento analizzato, il sistema deve verificare che le categorie di PII rilevate rientrino tra quelle dichiarate per l'attività di trattamento associata;
2. **asse della conservazione:** per ciascun documento contenente PII, il sistema deve confrontare la data del documento, o quella del suo ultimo aggiornamento, con il periodo di retention dichiarato nel ROPA per l'attività di trattamento associata.

Sull'asse della conservazione va precisato che cosa il sistema può realmente garantire. La data di un documento non strutturato non è un dato accertabile ma una **stima**, ricavata

dalle sole informazioni che il file rende disponibili: i segnali utilizzabili — i metadati interni al formato e la data di ultima modifica registrata dal file system — hanno affidabilità diversa fra loro e sono entrambi alterabili da eventi estranei al contenuto, come una copia massiva o un ripristino da backup. Il sistema deve pertanto **registrare e mostrare la provenienza** della data impiegata, affinché il DPO possa distinguere una segnalazione fondata da un indizio da verificare. Deve inoltre **segnalare esplicitamente** i casi in cui il confronto non è possibile — tipicamente quando il registro esprime la conservazione come criterio anziché come durata — anziché considerarli conformi: l'assenza di verifica non deve essere indistinguibile da un esito favorevole.

Il requisito si considera soddisfatto quando, dato un corpus documentale di test con anomalie note, il sistema le identifica correttamente su entrambi gli assi, senza segnalare come non conformi i documenti che non lo sono.

2.2.2 Riferimento al registro delle attività di trattamento come fonte normativa

Il sistema deve trattare il/i ROPA forniti in input come riferimento normativo autoritativo per ogni valutazione di conformità, e non deve generare, inferire o modificare autonomamente le attività di trattamento: il ROPA rappresenta la *ground truth* dichiarativa dell'organizzazione [10].

L'art. 30 GDPR [2] e l'art. 12 nLPD [3] impongono al titolare del trattamento la tenuta di un registro che documenti, tra l'altro, le categorie di interessati, le categorie di dati personali trattati, le finalità, la durata di conservazione e le categorie di destinatari. Il sistema si fonda su questo documento come base di confronto, senza sindacarne la correttezza né sostituirsi alla responsabilità del titolare nella sua compilazione.

Qualora il ROPA fornito risulti incompleto o incoerente, il sistema deve:

- segnalare le carenze nel report, classificandole come anomalie del ROPA stesso;
- proseguire l'analisi limitatamente ai campi disponibili, senza interrompere l'elaborazione.

Il requisito si considera soddisfatto quando il sistema, ricevendo un ROPA con campi mancanti, produce le segnalazioni appropriate senza errori di esecuzione e senza alterare i dati del registro fornito.

2.2.3 Copertura delle categorie di dati degni di particolare attenzione

Il sistema deve riconoscere le categorie di dati personali degni di particolare protezione secondo l'art. 5 lett. c nLPD [3] e le categorie particolari di dati personali secondo l'art. 9(1) GDPR [2], segnalandone la presenza con priorità elevata nel report delle anomalie, in

considerazione del regime di consenso espresso richiesto (art. 6 cpv. 7 lett. a nLPD [3]; art. 9(2) GDPR [2]).

2.2.4 Posizionamento rispetto all'art. 32(1)(d) GDPR

Il sistema, nella sua esecuzione periodica, deve costituire un'implementazione tecnica della procedura di *test, verifica e valutazione regolare dell'efficacia delle misure tecniche e organizzative* prevista dall'art. 32(1)(d) GDPR [2] e, analogamente, dall'art. 8 nLPD [3].

L'art. 32(1)(d) GDPR [2] qualifica come misura di sicurezza obbligatoria una procedura per testare e valutare regolarmente l'efficacia delle misure adottate. Un sistema che rileva periodicamente i dati personali nei documenti aziendali e ne verifica la coerenza con il registro dei trattamenti implementa, di fatto, questa procedura con riferimento alla conformità documentale.

Il perimetro del sistema è tuttavia limitato alla verifica di conformità documentale. Restano esplicitamente esclusi:

- le funzioni di anonimizzazione o pseudonimizzazione dei dati;
- la gestione degli incidenti di sicurezza (*data breach*);
- la valutazione d'impatto sulla protezione dei dati (DPIA).

Il requisito si considera soddisfatto quando il sistema può essere eseguito periodicamente in modo non presidiato, producendo report comparabili tra esecuzioni successive.

2.2.5 Esclusione delle basi giuridiche dal perimetro di analisi automatica

Il sistema non deve determinare autonomamente la base giuridica applicabile a un trattamento (art. 6 GDPR [2]; art. 30-31 nLPD [3]): tale valutazione resta di competenza esclusiva del DPO o del consulente giuridico dell'organizzazione.

La determinazione della base giuridica richiede una valutazione contestuale di natura giuridica che non è riducibile a un'analisi automatica del contenuto documentale. Come evidenziato nella sezione 2.2.1, il GDPR e la nLPD adottano inoltre approcci strutturalmente diversi alla liceità del trattamento, rendendo impossibile un'automazione generalizzata senza introdurre rischi di errore significativi.

Il sistema si limita pertanto a:

- segnalare la presenza di PII non riconducibili ad alcuna attività di trattamento dichiarata nel ROPA ("PII orfane");
- fornire al DPO gli elementi contestuali — documento, posizione, categoria di PII — necessari per una valutazione umana.

Il requisito si considera soddisfatto quando il sistema non associa in nessun caso autonomamente una base giuridica a un trattamento e le PII non associabili ad alcuna attività di trattamento risultano segnalate come orfane.

2.3 Requisiti operativi

2.3.1 Elaborazione locale dei dati

Il sistema deve garantire che ogni dato analizzato rimanga nell'infrastruttura in cui si trova inizialmente. Nessun contenuto documentale, testo estratto, PII rilevata o dato intermedio deve essere trasmesso a servizi esterni, API cloud o endpoint di inferenza remoti, in nessuna fase dell'elaborazione.

I documenti analizzati possono contenere dati degni di particolare protezione ai sensi dell'art. 5 lett. c nLPD [3] e categorie particolari ai sensi dell'art. 9 GDPR [2]. La trasmissione di tali dati a servizi terzi introdurrebbe un trattamento ulteriore che richiederebbe una base giuridica propria, un'analisi dei rischi specifica e garanzie contrattuali (art. 28 GDPR [2]), vanificando la finalità stessa del sistema.

Il vincolo si applica a tutte le componenti del sistema, incluse:

- i modelli di AI e NLP (cfr. 2.3.2);
- i moduli di estrazione del testo (cfr. 2.1.2);
- il registro interno delle PII (cfr. 2.1.8);
- la generazione e la presentazione del report di conformità (cfr. 2.1.7).

Il requisito si considera soddisfatto quando, durante l'esecuzione del sistema, non viene effettuata alcuna connessione di rete in uscita per la trasmissione di dati documentali o da essi derivati; la verifica può essere condotta tramite analisi del traffico di rete o ispezione del codice sorgente.

2.3.2 Vincolo sui modelli AI impiegabili

Il sistema deve impiegare esclusivamente modelli di intelligenza artificiale eseguibili interamente in locale, senza richiedere la trasmissione dei dati analizzati a servizi di inferenza esterni. Tale vincolo discende direttamente dal requisito di elaborazione locale (cfr. 2.3.1) e dalla natura potenzialmente sensibile dei dati trattati.

La scelta dei modelli deve bilanciare accuratezza e sostenibilità computazionale:

- per il rilevamento sistematico delle PII sull'intero corpus documentale, il sistema deve privilegiare approcci computazionalmente leggeri, eseguibili su hardware aziendale standard [11, 12];

- per le attività selettive che richiedono comprensione semantica profonda (cfr. 2.1.11), il sistema può impiegare modelli generativi di dimensioni compatibili con l'esecuzione locale [9], limitandone l'applicazione a sottoinsiemi mirati del corpus.

La giustificazione della scelta architetturale dei modelli deve essere documentata attraverso studi comparativi che confrontino i risultati ottenuti con il consumo di risorse sostenuto (cfr. Capitolo 5).

2.3.3 Facilità di installazione e aggiornamento

Il sistema deve essere installabile e operativo in qualsiasi infrastruttura aziendale con un numero minimo di passaggi di configurazione, senza richiedere personalizzazioni specifiche per l'ambiente di destinazione. Il sistema è destinato a organizzazioni con livelli di maturità IT eterogenei: un processo di installazione complesso o dipendente da configurazioni ad-hoc ne ridurrebbe l'adozione e renderebbe difficoltosa la manutenzione nel tempo.

In particolare, il sistema deve soddisfare i seguenti vincoli:

- l'installazione deve avvenire tramite un meccanismo standardizzato e riproducibile, che integri le dipendenze necessarie all'esecuzione;
- la configurazione iniziale deve limitarsi alla specifica dei percorsi di input — documenti e ROPA — e delle preferenze di output;
- l'aggiornamento del sistema deve poter avvenire senza perdita dei dati del registro interno (cfr. 2.1.8) e senza richiedere una reinstallazione completa;
- in caso di malfunzionamento, il sistema deve poter essere riportato a una configurazione funzionante nota senza interventi specialistici.

Il requisito si considera soddisfatto quando l'installazione su un ambiente pulito può essere completata seguendo la documentazione fornita, senza interventi manuali non documentati.

2.3.4 Estensibilità delle categorie e dei detector

Il sistema deve adottare un'architettura modulare che consenta l'aggiunta, la rimozione e la modifica delle categorie di PII rilevabili e dei relativi detector senza richiedere modifiche al codice sorgente. Le categorie di dati personali rilevanti variano infatti in funzione del framework normativo applicabile, del settore di attività dell'organizzazione e delle specifiche attività di trattamento dichiarate nel ROPA: un sistema non estensibile costringerebbe l'organizzazione ad attendere un aggiornamento del prodotto per coprire le categorie specifiche del proprio contesto.

L'estensibilità deve essere garantita su due livelli:

- **categorie di PII:** l'operatore deve poter definire nuove categorie di dati personali specificandone il nome, la descrizione e le regole di rilevamento associate;
- **detector:** il sistema deve supportare l'aggiunta di nuovi moduli di rilevamento tramite un'interfaccia di configurazione dichiarativa, senza richiedere competenze di sviluppo software.

Il requisito si considera soddisfatto quando un operatore non sviluppatore è in grado di aggiungere una nuova categoria di PII con il relativo detector seguendo la documentazione fornita, e il sistema la rileva correttamente al successivo ciclo di analisi.

2.3.5 Minimizzazione dei dati trattati dal sistema stesso

Il sistema, nel corso della propria esecuzione, deve rispettare il principio di minimizzazione dei dati (art. 5(1)(c) GDPR [2]; art. 6 cpv. 2 nLPD [3]), limitando la quantità di informazioni personali che tratta e conserva internamente a quanto strettamente necessario per produrre il report di conformità.

In particolare:

- il sistema non deve duplicare integralmente i documenti analizzati; il contenuto estratto deve essere mantenuto in memoria per la sola durata dell'elaborazione;
- le PII rilevate devono essere riportate nel registro interno (cfr. 2.1.8) e nel report finale attraverso riferimenti posizionali, e non tramite la riproduzione del dato personale;
- i dati intermedi devono essere eliminati al termine dell'esecuzione.

2.3.6 Efficienza delle rianalisi periodiche

Il sistema è destinato a un impiego **ricorrente** sullo stesso corpus documentale: la conformità non è una fotografia ma uno stato da mantenere nel tempo, e il registro interno (cfr. 2.1.8) esiste proprio per confrontare esecuzioni successive. Un'analisi che rielabori integralmente l'intero corpus a ogni esecuzione rende però l'esecuzione periodica impraticabile su condizioni aziendali reali, e il costo sostenuto è interamente superfluo per i documenti che nel frattempo non sono stati toccati.

Il sistema deve pertanto:

- rianalizzare i soli documenti **modificati** dall'ultima esecuzione, riconoscendoli da informazioni ottenibili senza leggerne il contenuto;
- **dichiarare** all'operatore quali documenti sono stati analizzati e quali omessi in quanto invariati, sia prima dell'esecuzione sia nel resoconto finale;

- consentire di **forzare** una rianalisi completa, e provvedervi autonomamente quando muta la configurazione del rilevamento, poiché in tal caso i risultati conservati non descrivono più ciò che il sistema troverebbe;
- garantire che l'omissione di un documento invariato non alteri in alcun modo il registro, e in particolare non lo faccia apparire rimosso.

Il requisito si considera soddisfatto quando una seconda esecuzione su un corpus immutato non rianalizza alcun documento, lascia il registro identico a quello prodotto dalla prima e lo dichiara nel proprio resoconto.

2.3.7 Operatività senza addestramento iniziale (zero-training)

Il sistema deve essere operativo immediatamente dopo l'installazione, senza richiedere una fase di addestramento su dati specifici dell'organizzazione. Il rilevamento delle PII deve basarsi su conoscenze preconfigurate — modelli pre-addestrati, regole predefinite, dizionari standard — sufficienti a garantire un livello di detection accettabile per le categorie canoniche della nLPD e del GDPR.

Il sistema deve tuttavia supportare meccanismi di personalizzazione facoltativa, attivabili dopo il primo impiego:

- dizionari specifici per il framework normativo di riferimento;
- specializzazioni basate su documenti rappresentativi dell'organizzazione, per migliorare la precisione del rilevamento sulla terminologia e sui formati propri del dominio aziendale.

2.4 Requisiti di qualità

I requisiti di qualità definiscono le proprietà non funzionali che il sistema deve soddisfare per essere considerato adeguato al contesto operativo previsto. Essi riguardano in particolare le prestazioni su volumi documentali realistici, l'affidabilità del rilevamento e la capacità di giustificare ogni anomalia segnalata.

2.4.1 Prestazioni su larga scala

Il sistema deve essere in grado di analizzare volumi documentali rappresentativi di un'organizzazione di medie dimensioni, nell'ordine delle migliaia di documenti, in tempi compatibili con un'esecuzione periodica non presidiata.

L'architettura deve prevedere meccanismi di parallelizzazione dell'elaborazione e, ove necessario, di analisi incrementale, in modo da evitare la rielaborazione integrale del corpus

documentale a ogni esecuzione. Il tempo di elaborazione deve scalare in modo al più lineare rispetto al numero di documenti analizzati.

2.4.2 Precisione del rilevamento

Il sistema deve privilegiare un elevato valore di *recall*, al fine di minimizzare il rischio che dati personali effettivamente presenti nei documenti non vengano segnalati: un falso negativo equivale, in questo contesto, a una non-conformità non rilevata, con potenziali conseguenze sanzionatorie per l'organizzazione.

Al contempo, il tasso di falsi positivi deve essere mantenuto a un livello tale da non compromettere l'usabilità del report per il DPO, poiché un numero eccessivo di segnalazioni errate riduce la fiducia nel sistema e ne vanifica l'utilità operativa.

Le soglie di *precision* e *recall* accettabili per ciascuna categoria di PII sono definite in fase di validazione sperimentale (cfr. Capitolo 8).

2.4.3 Tracciabilità delle anomalie

Il sistema deve rendere ogni anomalia segnalata nel report tracciabile fino alla sua origine, associandole:

- il documento sorgente, identificato da percorso, nome del file ed eventuale impronta crittografica;
- la posizione specifica all'interno del documento, ove applicabile;
- la regola o il modello che ha generato il rilevamento, con la categoria di PII attribuita;
- l'attività di trattamento del ROPA a cui l'anomalia è stata associata, o l'indicazione esplicita di assenza di associazione.

La tracciabilità è condizione necessaria per consentire al DPO di verificare ciascuna segnalazione e, ove necessario, di utilizzare il report come evidenza documentale nell'ambito delle procedure di *accountability* richieste dal GDPR (art. 5(2)) e dalla nLPD.

2.5 Requisiti di interfaccia

2.5.1 Interfaccia web come modalità primaria

Il sistema deve esporre un'interfaccia web come modalità primaria di interazione, servita dal sistema stesso su una porta di rete e raggiungibile da browser senza installazione lato client. Il destinatario è il DPO, figura non necessariamente tecnica, e il sistema è installato come unità autocontenuta e avviabile con un numero minimo di passaggi (cfr. 2.3.3): un'interfaccia

servita dall'unità stessa è operabile senza accesso alla shell della macchina che la ospita e senza conoscenza dei comandi, a differenza di una CLI.

L'interfaccia deve consentire almeno le seguenti operazioni:

- avviare l'analisi di una cartella, indicandone il percorso o selezionandola dal browser, con anteprima dei file che verranno analizzati e visualizzazione dello stato di avanzamento;
- fornire il/i ROPA in input e revisionare le categorie di dati riconosciute (cfr. 2.1.4);
- consultare il registro delle PII rilevate e il verdetto di conformità per ciascun documento (cfr. 2.1.8);
- gestire l'associazione tra documenti e attività di trattamento (cfr. 2.1.5).

Il requisito si considera soddisfatto quando, a partire dal solo avvio del sistema, un utente è in grado di importare un ROPA, avviare una scansione e consultare il verdetto di conformità senza ricorrere alla riga di comando.

2.5.2 Interfaccia a riga di comando (CLI)

Il sistema deve esporre, come modalità secondaria, interfacce a riga di comando destinate all'automazione e alla diagnosi. La CLI è il punto di aggancio per l'esecuzione periodica non presidiata (cfr. 2.2.4) e per l'ispezione dei singoli stadi dell'elaborazione, e deve consentire almeno:

- l'ingestione di un ROPA e la mappatura delle sue categorie;
- la scansione di un singolo documento o di un albero di cartelle, con aggiornamento del registro interno;
- la verifica di conformità e la stampa del relativo verdetto.

Non è richiesto che tali operazioni siano esposte da un unico comando orchestratore: è ammessa una CLI per stadio dell'elaborazione, purché ciascuna sia componibile in uno script di schedulazione. L'interfaccia deve produrre messaggi di log progressivi che consentano di monitorare l'avanzamento e di diagnosticare gli errori.

2.5.3 Fruizione del report

Il sistema deve rendere il report di conformità (cfr. 2.1.7) fruibile in due modi complementari:

- **consultazione interattiva:** l'interfaccia web (cfr. 2.5.1) deve presentare il report al DPO in forma navigabile — elenco dei documenti analizzati, dettaglio delle PII rilevate e verdetto per documento — senza richiedere un passo di esportazione né strumenti esterni;

- **lettura meccanica:** il contenuto del report deve poggiare sul registro interno persistente (cfr. 2.1.8), in un formato strutturato e interrogabile, così da restare accessibile agli strumenti di governance, risk e compliance già in uso nell'organizzazione.

Il report non è un artefatto separato ma una *vista* sui dati che il sistema già mantiene: il registro è la fonte di verità, l'interfaccia ne è la presentazione. Un file di report generato a fine analisi sarebbe invece una fotografia che invecchia — disallineata alla prima ri-scansione — e duplicherebbe in un secondo formato informazioni già persistite, con l'onere di mantenerli coerenti. La consultazione dal browser garantisce inoltre che il DPO veda sempre lo stato corrente del corpus documentale, e l'assenza di file di report esportati è coerente con il vincolo di minimizzazione (cfr. 2.3.5): non si moltiplicano le copie del quadro delle PII dell'organizzazione.

L'esportazione del report in file autonomi — strutturati per l'integrazione con sistemi di governance, o impaginati per la presentazione in sede di audit — non rientra nel perimetro del prototipo: è una resa aggiuntiva sopra i medesimi dati, che non aggiunge capacità di analisi né di verifica.

Il requisito si considera soddisfatto quando, al termine di un'analisi di test, il DPO può consultare dal browser l'elenco dei documenti analizzati e il verdetto di ciascuno, e il medesimo contenuto informativo è ricavabile dal registro interno mediante interrogazione diretta, senza passare dall'interfaccia.

Capitolo 3

Stato dell'arte

3.1 Il problema: dalla *data anonymization* alla *data governance*

Gran parte degli strumenti che trattano dati personali in testo libero nasce per rispondere a una domanda operativa: *come rimuovo questo dato?* L'anonimizzazione, la pseudonimizzazione e il mascheramento condividono infatti il medesimo presupposto — il dato personale è un ostacolo da neutralizzare prima che il documento venga condiviso, pubblicato o impiegato per addestrare un modello. È un presupposto legittimo, ma risponde a un problema diverso da quello affrontato in questa tesi.

La domanda della *data governance* non è come rimuovere un dato, bensì se quel dato **possa legittimamente trovarsi** dove si trova: se rientri fra le categorie che l'organizzazione ha dichiarato di trattare per quella finalità, e se sia conservato entro il termine dichiarato. Il dato non va quindi eliminato ma **messo in relazione** con ciò che l'organizzazione afferma di sé nel proprio registro dei trattamenti. Ne discende una conseguenza che orienta l'intera architettura: il rilevamento delle PII smette di essere il fine del sistema e ne diventa un **presupposto**. Un rilevatore, per quanto accurato, non risponde ancora ad alcuna domanda di conformità finché ciò che trova non viene confrontato con ciò che è stato dichiarato.

3.1.1 Il problema tecnico: ambiguità contestuale e frammentazione dei domini

Sul piano tecnico, la difficoltà del testo non strutturato non consiste nel *leggere* il documento, ma nel disambiguare le informazioni in base al contesto in cui compaiono. Una medesima sequenza di caratteri può essere un identificativo personale o un codice gestionale a seconda di ciò che la circonda; un nome proprio può designare una persona fisica, una via o una ragione sociale. La formattazione, che rende affidabile il riconoscimento di un IBAN o di un codice fiscale, non offre alcun appiglio per queste categorie.

A questa difficoltà si somma quella che la letteratura descrive come problema del *domain-silo* [13]: i sistemi di rilevamento vengono addestrati e valutati su dati provenienti da un dominio

specifico e degradano sensibilmente su testi eterogenei. È precisamente la condizione in cui si trova una condivisione documentale aziendale reale, dove contratti, verbali, corrispondenza, fogli di calcolo e manualistica convivono nella stessa cartella: un corpus che nessun singolo dominio di addestramento rappresenta.

3.1.2 La verifica automatizzata della conformità

Il confronto fra quanto un'organizzazione dichiara e quanto effettivamente tratta è, di per sé, un problema di ricerca autonomo. Amaral et al. [10] affrontano il caso duale a quello qui considerato — la verifica automatizzata, mediante tecniche di NLP, della completezza di un'informativa privacy rispetto agli obblighi del GDPR — mostrando come un requisito normativo espresso in prosa possa essere reso in criteri verificabili meccanicamente.

Il presente lavoro sposta il termine di confronto: non l'informativa rispetto alla norma, ma il **contenuto documentale effettivo** rispetto al registro dei trattamenti dell'organizzazione. Il registro fornisce l'atteso — quali categorie di dati personali, per quali finalità, per quanto tempo — e il rilevamento fornisce l'osservato; la verifica consiste nel differenziale fra i due insiemi. Questa impostazione richiede che il registro, tipicamente redatto in prosa libera, sia reso confrontabile a livello di macchina: un'esigenza che ricade sull'ingestione del registro e sulla riconduzione delle categorie dichiarate a un vocabolario condiviso con il rilevatore.

3.2 Background tecnologico

Questa sezione è specificamente indicata per inserire definizioni e spiegazioni su argomenti discussi successivamente.

3.2.1 Supervised vs unsupervised

Come spiegato in uno dei testi cardine del machine learning, redatto da Goodfellow et al. [14], queste due metodologie di apprendimento si distinguono in base al tipo di *esperienza* che l'algoritmo acquisisce a partire dai dati durante l'addestramento. Sono definite come segue:

- per **Supervised learning** si intende quel paradigma di apprendimento automatico in cui l'algoritmo viene addestrato su un insieme di dati in cui ogni esempio, oltre alle sue caratteristiche, è associato a un'etichetta o a un valore target noto a priori. Il modello apprende quindi una funzione che mappa le caratteristiche di input all'etichetta corrispondente, sulla base di esempi già classificati o annotati. Un caso tipico è quello di un classificatore addestrato a riconoscere la specie di appartenenza di un fiore a partire da misurazioni morfologiche etichettate.

- per **Unsupervised learning** si intende quel paradigma in cui l'algoritmo opera su un insieme di dati privo di etichette, con l'obiettivo di individuare autonomamente strutture, regolarità o proprietà latenti presenti nei dati stessi. Nel contesto del deep learning, questo si traduce spesso nel tentativo di apprendere — in modo esplicito (ad esempio tramite stima di densità) o implicito (ad esempio per compiti di generazione o denoising) — la distribuzione di probabilità sottostante ai dati. Altri algoritmi non supervisionati svolgono invece compiti diversi, come il clustering, che consiste nel raggruppare gli esempi del dataset in base alla loro similarità.

Storicamente i modelli impiegati per l'estrazione di informazioni si sono divisi lungo questa linea, e il confronto fra i due paradigmi è documentato in termini di scalabilità: i modelli supervisionati garantiscono prestazioni superiori sui pattern noti, ma richiedono onerose fasi di etichettatura manuale per adattarsi a nuovi scenari [15].

Nel contesto specifico della NER, l'approccio supervisionato conserva un vantaggio di accuratezza sulle alternative prive di annotazione, ma a un costo che non è di calcolo bensì di **etichettatura**: la letteratura di rassegna individua nella disponibilità di un corpus annotato di dimensioni adeguate, e non nella potenza di inferenza, l'ostacolo principale all'adozione [16, 17]. Il guadagno è inoltre distribuito in modo diseguale fra le categorie: è massimo proprio su quelle che gli approcci privi di addestramento coprono peggio — le entità sprovviste di un formato riconoscibile — e trascurabile su quelle già ben servite da regole deterministiche. I valori misurati che sostengono questa lettura sono riportati nel confronto empirico (cfr. 3.3.6).

3.2.2 Pre-training vs fine-tuning

Il paradigma pre-training/fine-tuning è stato reso centrale nell'NLP moderno dal lavoro di Devlin et al. [18], che lo applicano al modello BERT (Bidirectional Encoder Representations from Transformers). Le due fasi si articolano come segue:

- per **pre-training** si intende la fase iniziale di addestramento in cui il modello apprende rappresentazioni linguistiche generali a partire da grandi quantità di testo non etichettato, tramite obiettivi di tipo *self-supervised* (ad esempio indovinare delle parole mascherate casualmente all'interno di una frase). In questa fase il modello non è ancora specializzato su alcun compito specifico: l'obiettivo è costruire una rappresentazione del linguaggio sufficientemente generale per poter essere riutilizzata per molteplici task.
- per **fine-tuning** si intende la fase successiva, in cui i parametri del modello pre-addestrato vengono ulteriormente ottimizzati su un dataset etichettato e di dimensioni ridotte, specifico per il compito che si intende risolvere. A differenza del pre-training, che richiede risorse computazionali ingenti, il fine-tuning può avvenire con un costo

computazionale contenuto, poiché parte da una rappresentazione linguistica già appresa e la adatta al task target, spesso aggiungendo un solo livello di output specifico per il compito.

Applicato al rilevamento di PII, questo paradigma presenta due criticità che ne condizionano l'adozione. La prima è il **costo di annotazione**: l'adattamento richiede dataset etichettati di dimensioni rilevanti, il che nel caso delle PII comporta l'intervento manuale di esperti e solleva problemi di riservatezza intrinseci alla manipolazione di dati reali. La seconda è la **perdita di generalizzazione** (*catastrophic forgetting*): una specializzazione spinta su un singolo formato documentale può compromettere la capacità del modello di operare su testi imprevisti.

Entrambi i vincoli si acuiscono negli ambienti operativi ristretti. Lo studio di Zhang et al. [19] sul rilevamento di PII in architetture di *smart edge computing* evidenzia come l'esigenza di eseguire inferenza locale e decentralizzata richieda modelli sufficientemente robusti da non imporre continui cicli di riaddestramento a ogni nuova categoria di dati da coprire.

3.2.3 Zero-shot learning

Il concetto di zero-shot learning è stato formalizzato in modo sistematico da Xian et al. [20], che lo definiscono come quel paradigma di apprendimento in cui un modello viene valutato su classi (o, nel caso della Named Entity Recognition, su tipologie di entità) che non erano presenti nel suo insieme di addestramento.

A differenza dei paradigmi supervised e unsupervised descritti in 3.2.1, lo zero-shot learning non riguarda la presenza o assenza di etichette nei dati di training, bensì la capacità del modello di generalizzare a categorie mai osservate, sfruttando informazioni semantiche ausiliarie (ad esempio descrizioni testuali delle classi target) anziché esempi annotati specifici per quelle categorie. Tale capacità si distingue dal *one-shot* e dal *few-shot learning*, in cui il modello dispone rispettivamente di uno o di un numero ridotto di esempi annotati per le classi target [21].

Il meccanismo con cui questo paradigma viene realizzato nella NER è lo *span-matching*: architetture come GLiNER [22] ricevono in ingresso sia il documento da analizzare sia le descrizioni testuali delle entità da individuare (ad esempio «numero di previdenza sociale»), e calcolano la similarità semantica fra le porzioni di testo e la descrizione dell'etichetta cercata. Estendere le categorie riconosciute non richiede quindi un riaddestramento, ma l'aggiornamento della lista di istruzioni testuali fornite in input — proprietà che risponde direttamente ai vincoli di flessibilità e di riservatezza locale richiamati sopra [19]. Le prestazioni effettivamente misurate su questa famiglia di modelli sono discusse nel confronto empirico (cfr. 3.3.6).

3.3 Approcci tecnici di PII detection

Per rilevare le PII, esistono più metodi e strategie, tutte ampiamente discusse in studi accademici. Ognuna di queste ha dei punti di forza e di debolezza, infatti esiste una area di problemi dove ogni singola opzione diventa la migliore, riducendo o addirittura annullando gli effetti negativi.

Le soluzioni in questione sono:

3.3.1 Recognition rule-based

Approccio più semplice e rule-based. La detection delle PII è solamente affidata a regole di pattern-matching e meccanismi simili. Come si può già intuire, questo approccio, per alcune categorie di dati sensibili, è particolarmente efficace: la massima performance si ottiene quando si trattano dati strettamente formattati in determinati modi (IBAN, AVS, codice fiscale etc...). Non appena i testi diventano liberi e non strutturati, allora l'accuratezza crolla. [13, Sezione 5.2] Questo approccio tuttavia, ha dei problemi seri che non possono essere trascurati: categorie nettamente più difficili da trovare (i nomi propri, per esempio, si baserebbero su regole facilmente mal-interpretabili e con un alto numero di falsi positivi, come la sola lettera maiuscola).

Vantaggio: più veloce in assoluto tra tutte le soluzioni.

3.3.2 Recognition con NLP - NER

I modelli di Natural Language Processing (NLP), tramite la Named Entity Recognition (NER), classificano le entità testuali analizzando la struttura logica della frase. Architetture basate su Transformers (es. BERT) superano i limiti delle regole, comprendono il contesto e raggiungono accuratezze fino al 99.55% [23].

Vantaggio: eccellente bilanciamento tra comprensione del testo ed efficienza computazionale.

Svantaggio: Rischio di calo prestazionale su dati aziendali particolarmente specifici senza fare fine-tuning su dataset simili.

3.3.3 Recognition con AI generativa

Ultima soluzione "pura" è l'utilizzo di LLM con AI generativa per l'individuazione di PII. Questo approccio, fortemente documentato e discusso, è senza dubbio il migliore per quanto riguarda solamente le performance. Infatti, seppur riesce ad avere un F1-score e una recall senza precedenti, ha un costo computazionale e di risorse senza precedenti, spesso ingestibile per la maggior parte delle macchine. Gli unici modi per rendere accettabili tempi di esecuzione è di effettuare down-scaling dei modelli utilizzati, riducendo contesto e "intelligenza" dello

stesso. Seppur di aiuto, più si scende con la dimensione e il peso dei modelli, più il vantaggio di usare gli LLM si assottiglia, fino ad azzerarsi rispetto all'uso dell'NLP.

Tuttavia, gli LLM hanno un vantaggio unico per quanto riguarda un aspetto fondamentale nel nostro problema: la presenza nativa del 'contesto'. Infatti gli LLM possiedono una comprensione semantica avanzata del linguaggio naturale. Tramite il paradigma Zero-Shot, identificano PII deducibili puramente dal contesto verbale (es. dedurre "minorenne" dalla frase "mio figlio, nato 3 anni fa") senza alcun addestramento. Inoltre, offrono interpretazioni del loro operato, agevolando gli audit di conformità [9].

3.3.4 Limitazioni nella soluzione

Come visto nella sezione precedente, la soluzione parzialmente migliore sembrerebbe essere la detection con NLP. Questo perché riesce ad avere una percentuale di matching particolarmente alta, senza aver bisogno della quantità ingiustificabilmente alta di risorse dell'AI generativa.

Tuttavia, questa soluzione diventa praticamente insignificante se teniamo in considerazione l'aspetto dello "**Zero training**". Infatti, da requisiti (cfr. 2.3.7), il nostro sistema di detection deve essere ready-to-use non appena installato, garantendo una buona percentuale di detection senza aver bisogno dei dati su cui sta lavorando. Le metodologie NLP - NER hanno alla base proprio la knowledge dei dati che si stanno analizzando.

3.3.5 Soluzioni ibride

Per soddisfare il vincolo estremamente stringente dello "Zero-Training", bilanciando l'elevata richiesta di risorse dell'AI generativa e la rigidità dei sistemi rule-based, lo **standard industriale** prevede un'architettura ibrida (es. Microsoft Presidio) [24]

3.3.6 Confronto empirico degli approcci

Le sezioni precedenti presentano ciascun approccio in termini qualitativi. Per motivare la scelta architetturale in modo rigoroso, è necessario un confronto quantitativo basato su evidenze empiriche dalla letteratura recente. Le metriche considerate in questa sede sono quelle della *capacità di rilevamento*: precisione (P), recall (R), F1-score e necessità di dati annotati. Il *costo* di ciascun approccio — energetico ed economico — pesa in modo altrettanto determinante sulla scelta, ma costituisce un tema a sé ed è raccolto per intero nello studio dedicato del Capitolo 5: le due tabelle vanno lette in corrispondenza, riga per riga.

3.3.6.1 Prestazioni dei sistemi rule-based

Le espressioni regolari raggiungono alta precisione su pattern strutturati come email, numeri di telefono e indirizzi IP, specialmente se accompagnate da post-processing per eliminare falsi positivi sistematici (ad esempio ISBN confusi con numeri di telefono) [25]. Tuttavia, il limite strutturale è la *brittleness*: i detectors rule-based non considerano il contesto del testo circostante e falliscono sistematicamente su PII contestuali [26]. La letteratura di survey su NER conferma il pattern: per i metodi rule-based la precisione è tipicamente alta, mentre il recall tende a essere basso a causa di regole domain-specific; il processo di introduzione di nuove regole è costoso [16].

3.3.6.2 Prestazioni dei modelli NER supervisionati

Con corpus annotati adeguati, i modelli encoder fine-tuned raggiungono le prestazioni più elevate in assoluto tra tutti gli approcci considerati. In un benchmark comparativo su documenti clinici, BERT, ModernBERT e ClinicalBERT raggiungono tutti $P = 0.99$ e $R = 0.99$ ($CIRE \approx 0.99$), con throughput di 1528–2446 token/secondo [27]. Un modello DeBERTa fine-tuned sul dataset *ai4privacy* raggiunge $F1 = 0.9757$ su un benchmark PII multi-dominio [28].

Su documenti finanziari non strutturati (audit report, fatture), una pipeline ibrida NLP rule-based + ML + NER custom raggiunge $P = 94.7\%$, $R = 89.4\%$, $F1 = 91.1\%$ su dataset sintetici, con il 93% di accuratezza su documenti reali [29].

Il principale ostacolo non è il costo di inferenza (basso dopo il training) ma il costo di annotazione del corpus: le soluzioni supervisionate richiedono tipicamente un corpus annotato di grandi dimensioni [17]. Si tratta però di un costo da sostenersi *una tantum*, e non per ogni documento analizzato: la distinzione fra oneri *una tantum* e oneri ricorrenti è dirimente per la scelta architetturale ed è ripresa nel Capitolo 5 (cfr. 5.2).

3.3.6.3 Prestazioni dei modelli NER zero-shot

GLiNER (cfr. 3.2.3) usa un encoder bidirezionale (DeBERTa) con un meccanismo di span-extraction (individuare inizio e fine di una label, invece di processare parola per parola) prompt-style (inserire le etichette da estrarre come testo, senza averne di predefinite nell'architettura del modello): le label vengono fornite come input testuale e il modello identifica span corrispondenti nel documento senza retraining [22]. Test su dataset PII multi-domain mostrano GLiNER-base a circa l'81% di F1, superiore ad approcci spaCy o regex standard, con recall particolarmente elevato su ID numerici e date [30]. Il modello *gliner-pii-base-v1.0* raggiunge $F1 = 80.99\%$ ed è ottimizzato per applicazioni privacy-focused in cui i servizi NER cloud pongono rischi di conformità [31]. Va però precisato che nessuna delle due misure proviene da una pubblicazione sottoposta a revisione paritaria — si tratta rispettivamente

di un articolo divulgativo e della scheda di un modello — e che entrambe sono qui assunte come ordine di grandezza, non come risultato consolidato.

Tuttavia, la specificità di dominio rimane determinante anche per approcci zero-shot: su testo clinico, GLiNER scende a circa $F1 = 0.41$, perdendo oltre il 50% delle entità PHI rispetto ai modelli domain-specific [32]. Questo impone una valutazione empirica obbligatoria sul corpus aziendale target prima di escludere il fine-tuning.

3.3.6.4 Prestazioni dell'AI generativa

GPT-4o, LLaMA 3.3 70B e LLaMA 3.1 8B raggiungono tutti recall medio superiore a 0.90, ma la precisione rimane un problema irrisolto: anche il top performer GPT-4o raggiunge soltanto $P = 0.579$ su dataset educativi [33]. Su benchmark strutturati multi-task (PII-Bench), GPT-4o con metodi avanzati (Self-CoT, Self-Consistency) raggiunge $F1 \approx 0.76\text{--}0.77$, con Llama 3.1 e Qwen 2.5 in linea [13].

Il limite decisivo per il contesto di questa tesi non è però l'accuratezza, bensì il **costo ricorrente** dell'inferenza generativa: un onere che, a differenza dell'addestramento di un encoder, va sostenuto per ogni documento e a ogni nuova analisi del corpus. Poiché è questo il fattore che determina il ruolo assegnabile all'AI generativa nel sistema, il confronto quantitativo — energetico ed economico — è raccolto nello studio dedicato del Capitolo 5.

3.3.6.5 Convergenza verso la pipeline ibrida

La convergenza della letteratura recente indica la pipeline ibrida come soluzione ottimale per il rilevamento PII in contesti di compliance. Il sistema RECAP (NeurIPS 2025) combina regex deterministiche con LLM context-aware in un framework modulare a tre fasi: (1) rilevamento regex, (2) classificazione NER, (3) risoluzione ambiguità via LLM. Sul proprio banco di prova, costituito da 13 lingue a bassa disponibilità di risorse, RECAP riporta un *weighted* F1-score superiore dell'82% ai modelli NER fine-tuned e del 17% agli LLM impiegati in zero-shot. Entrambi i valori sono miglioramenti **relativi** misurati in quel regime, nel quale le *baseline* sono assai più deboli di quelle dei benchmark ad alta disponibilità di risorse riportati sopra: non sono quindi confrontabili con i valori assoluti della Tabella 3.1 [7].

Un sistema ibrido indipendente (CHPDA) documenta lo stesso principio: le regex eccellono su pattern chiaramente definiti ma mancano di flessibilità per gestire complessità contestuali; i modelli NER eccellono nell'identificare entità ambigue come nomi e indirizzi, difficili da catturare con regex, ma producono falsi positivi senza pattern guida. La combinazione risolve i limiti di entrambi [34].

La letteratura non esclude l'AI generativa in assoluto, ma ne circoscrive il ruolo: l'impiego su casi ambigui come supervisor di una pipeline ibrida è documentato come cost-effective [7]. L'esclusione riguarda l'inferenza sistematica su tutto il corpus, non l'impiego selettivo su un sottoinsieme di casi irrisolti.

3.3.6.6 Matrice di confronto sintetica

La Tabella 3.1 riassume le evidenze empiriche discusse nelle sezioni precedenti. Il confronto sui costi energetici ed economici dei medesimi approcci è riportato separatamente nella Tabella 5.1.

Tabella 3.1: Prestazioni di rilevamento degli approcci di PII detection, secondo le evidenze di letteratura.

Approccio	P	R	F1 tipico	Corpus annotato
Regex / rule-based	Alta	Bassa-media	0.60–0.75	No
NER supervised	0.95–0.99	0.89–0.99	0.94–0.99	Sì, esteso ^a
NER zero-shot (GLiNER)	Media	Media	~0.81	No
LLM generativo (GPT-4o)	0.58	>0.90	0.70–0.77	No
Pipeline ibrida	Alta	Alta	~0.91 ^c	Opzionale ^b

^a L'annotazione di un corpus di dimensioni adeguate è il principale ostacolo all'adozione di questo approccio [17].

^b L'annotazione è necessaria solo se la componente NER viene specializzata: la pipeline resta impiegabile anche interamente zero-shot [7]. ^c Pipeline ibrida regex + ML + NER su documenti finanziari non strutturati: P = 94.7%, R = 89.4%, F1 = 91.1% su dataset sintetici [29].

3.4 Panoramica dei sistemi esistenti

L'analisi dello stato dell'arte evidenzia una profonda spaccatura tra gli strumenti open-source, spesso focalizzati su un singolo task tecnico (anonimizzare o estrazione brutta), e le suite commerciali, ricche di feature ma close-source e inaccessibili.

3.4.1 Sistemi open-source

- **Microsoft Presidio** è un framework open-source sviluppato da Microsoft, diventato uno standard industriale per l'identificazione e l'anonimizzazione. Offre un'architettura modulare basata su classi estensibili come *EntityRecognizer* (per i modelli di Machine Learning) e *PatternRecognizer* (per il pattern matching classico) [35]. Il motore di rilevamento (*Analyzer*) è disaccoppiato dal componente di anonimizzazione (*Anonymizer*), include riconoscitori pre-costruiti anche per il contesto italiano (ad esempio il codice fiscale, con validazione a checksum) e consente di collegare motori NER esterni. Il suo **limite rispetto allo scopo del progetto** non è tecnologico ma di *finalità*: nasce per la rilevazione e l'anonimizzazione del dato, non per la governance e la verifica di conformità. Non implementa alcuna logica di registro dei trattamenti (ROPA), di gap analysis o di reportistica per il DPO, e non copre nativamente identificatori del contesto svizzero (ad esempio il numero AVS). Il suo motore di detection resta però riusabile come componente di un sistema più ampio, che ne governi gli esiti anziché adottarne la finalità: è la forma di riuso che questo lavoro considera.

- **GLiNER**: architettura Zero-Shot basata su Transformers, in grado di identificare PII passandole dinamicamente come query, senza addestramento. Le sue iterazioni recenti (GLiNER2-PII) mostrano prestazioni eccellenti in contesti multilingua [22].
Limiti: è esclusivamente un "motore algoritmico". Prende stringhe in input ed estrae etichette. Non implementa il parsing di documenti (PDF/Word), non genera reportistiche strutturate e non possiede alcuna logica di compliance (ROPA). [36]
- **Scrubadub / Datafog**: strutture open-source modulari. Scrubadub utilizza un'architettura a *Detector* multipli (Regex, detector predefiniti, NLP esterni) [37].
Limiti: sono fortemente legati al test-masking e all'oscuramento. Spesso si affidano pesantemente al pattern matching deterministico (Regex) (cfr:3.3.1), una tecnologia ormai insufficiente da sola per gestire documenti destrutturati e ambigui.

3.4.2 Sistemi commerciali

L'esame delle soluzioni commerciali non ha finalità comparative rispetto al sistema qui progettato, con il quale non condividono né scala né perimetro: serve a documentare quali funzionalità il mercato consideri costitutive di uno strumento di conformità, e a quali condizioni di adozione siano associate. Le piattaforme considerate integrano in un'unica architettura rilevamento, classificazione e governo del dato, ma a costi e vincoli strutturali che ne rendono l'adozione difficilmente sostenibile per le piccole e medie imprese, o incompatibile con l'esecuzione interamente locale. Le prestazioni riportate nel seguito sono dichiarazioni dei rispettivi produttori e non misurazioni indipendenti: sono richiamate per documentare il posizionamento del prodotto, non come termine di paragone.

- **Varonis**: piattaforma di *Data Security Posture Management* (DSPM). Impiega un motore ibrido per il quale il produttore dichiara un'accuratezza del 98% su dati strutturati e non strutturati (SharePoint, OneDrive, NAS), senza addestramento sui dati del cliente [38]. Due elementi sono pertinenti in questa sede: la *User and Entity Behavior Analytics* (UEBA), che rileva accessi anomali ai file classificati come sensibili [39], e il ricorso a scansioni *incrementali* per contenere il costo delle rianalisi periodiche — la medesima esigenza che il presente lavoro registra fra i propri requisiti (cfr. 2.3.6).
Limiti: il baricentro del prodotto è il *controllo degli accessi* e la sicurezza delle reti aziendali, non la verifica del contenuto documentale rispetto a un registro dei trattamenti. Il costo di licenza, riportato attorno agli 80.000 CHF/anno [40], e l'architettura chiusa lo collocano su una scala di adozione diversa da quella qui considerata.
- **BigID**: piattaforma di *data privacy* e *data governance*; è, fra i sistemi esaminati, quello più vicino al problema affrontato in questa tesi. Il rilevamento è *identity-aware*: i dati individuati non sono soltanto classificati per categoria, ma correlati a specifiche persone fisiche (*data subjects*) incrociando più sorgenti [41]; la priorità di scansione è

determinata da un componente proprietario (*Hyperscan*). Rilevante in questa sede è soprattutto la gestione automatizzata del registro dei trattamenti: i dati rilevati vengono posti in corrispondenza con gli obblighi normativi e con le politiche di conservazione dichiarate, fino alla cancellazione di quelli scaduti o ridondanti [42]. Il confronto fra dato rilevato e dato dichiarato risulta quindi una funzionalità già attesa dal mercato, e non un'ipotesi ancora da validare.

Limiti: l'offerta è di fascia *enterprise*, con costi determinati su preventivo. Diverse operazioni di classificazione poggiano su architetture cloud o ibride, che sollevano problemi di sovranità del dato incompatibili con il vincolo di mantenere l'intera inferenza all'interno dell'infrastruttura dell'organizzazione.

- **PII Tools:** soluzione verticale sul solo rilevamento delle PII. Adotta un approccio basato su apprendimento automatico che, secondo il produttore, riduce i falsi positivi delle espressioni regolari grazie a una maggiore considerazione del contesto [43]. Due caratteristiche la distinguono dalle precedenti: è distribuibile interamente *on-premise* in un contenitore, senza dipendenze da servizi remoti, e riporta la posizione del dato all'interno del documento anziché il solo esito della classificazione [44].

Limiti: resta un prodotto proprietario e non ispezionabile. Il modello impiegato non è sostituibile, e l'assenza di accesso al funzionamento interno impedisce sia di verificare come un esito sia stato prodotto, sia di estendere il rilevamento a categorie o ordinamenti non previsti dal fornitore.

3.5 Sintesi del gap e posizionamento del progetto

Alla luce di questa panoramica, il **gap nel mercato** risulta evidente. Le organizzazioni, in particolare le PMI svizzere, si confrontano con un panorama fortemente polarizzato:

1. **Soluzioni open-source:** Rilevano i dati in locale ma lo fanno con l'ottica prioritaria dell'*anonimizzazione* (Presidio), oppure offrono solo modelli potenti linguistici disconnessi da qualsiasi logica di governance e reportistica aziendale (GLINER).
2. **Soluzioni Commerciali:** Dimostrano quali siano le reali necessità delle aziende, introducendo l'*Identity-aware discovery*, l'automazione dei ROPA, lo scanning incrementale e le dashboard per DPO. Tuttavia, lo fanno imponendo ecosistemi Black-box, costi esorbitanti e massicce dipendenze da Cloud/API di terze parti, con conseguenti problematiche di *Data Sovereignty*.

3.5.0.0.1 Posizionamento del progetto Il software proposto si inserisce al centro di questo divario. L'obiettivo è fornire uno strumento open-source ed eseguibile **esclusivamente in locale** (on-premise), rispondendo all'esigenza di confidenzialità totale. Abbandonando il paradigma dell'anonimizzazione, il sistema non si propone di rimuovere il dato personale,

ma di stabilire se esso possa legittimamente trovarsi dove si trova: il rilevamento cessa di essere il fine e diventa il presupposto della verifica. A questo scopo il riconoscimento poggia su motori NLP Zero-Shot (cfr. 3.2.3), che eliminano i costi di addestramento discussi in precedenza. Il sistema fungerà così da ponte tra il dato reale e le policy aziendali (ROPA), fornendo un quadro clinico dell'aderenza normativa aziendale.

da inserire tabella riassuntiva (o paragrafo comparativo) che fa capire cosa "fanno gli altri" e "cosa manca".

Capitolo 4

Analisi dei rischi

4.1 Rischi di progetto

4.1.1 Disponibilità e qualità dei dataset di valutazione

La letteratura sul trattamento delle PII è orientata in prevalenza all'anonimizzazione, e i corpus pubblici disponibili sono costruiti per quel fine: scarseggiano insiemi di valutazione pensati per la sola *detection*, annotati in modo esaustivo e liberamente utilizzabili. Il problema è noto e non specifico di questo lavoro: il principale ostacolo alle soluzioni supervisionate è riconosciuto proprio nel costo di costruzione di un corpus annotato di dimensioni adeguate [17]. A ciò si aggiunge un vincolo proprio di questo progetto: un corpus di dati reali sarebbe esso stesso un archivio di dati personali, in contraddizione diretta con il principio di minimizzazione che il sistema è chiamato a rispettare (cfr. 2.3.5).

Se il rischio si materializza, il sistema resta privo di una base di misura: le prestazioni del rilevamento non sarebbero verificabili in modo riproducibile e ogni scelta architettuale — quale rilevatore adottare, se l'AI aggiunga valore — si ridurrebbe a una preferenza soggettiva, indebolendo l'intero impianto argomentativo della tesi.

La contromisura prevista è la generazione di corpus **sintetici** e riproducibili a partire da un seme — pratica consolidata quando i dati reali non sono utilizzabili, e oggetto essa stessa di studio nella costruzione di insiemi annotati per il riconoscimento di entità [45]: i valori devono essere realistici e, dove il formato lo prevede, aritmeticamente validi, mentre il *gold* va derivato dalla stessa sorgente che produce il testo, così che annotazione e documento non possano divergere. La valutazione va inoltre articolata su più livelli di granularità, dal documento singolo all'albero documentale, con le soglie e i protocolli definiti in sede sperimentale (cfr. Capitolo 8). Resta un rischio residuo di cui tenere conto nella lettura dei risultati: un corpus sintetico è per costruzione più ordinato del reale, e i valori misurati su di esso vanno intesi come limite superiore delle prestazioni attese sul campo.

4.1.2 Complessità dell'integrazione multi-approccio

Il rilevamento non può essere affidato a una tecnica sola: il requisito impone la composizione di meccanismi complementari — espressioni regolari, riconoscimento di entità nominate e, in modo selettivo, modelli generativi (cfr. 2.1.3, 2.1.11) — ed è la direzione che la letteratura indica come standard (cfr. 3.3.5). Ciascuna sorgente produce però risultati in modo autonomo, con confini di testo e categorie che possono sovrapporsi, coincidere o contraddirsi.

Far convivere esiti eterogenei senza una disciplina esplicita produce duplicazione delle segnalazioni, esiti incoerenti sullo stesso frammento di testo e, soprattutto, l'impossibilità di attribuire un errore a uno stadio preciso: se il sistema sbaglia, non si saprebbe se la responsabilità sia del rilevamento, della fusione dei risultati o dell'estrazione a monte.

Il rischio va contenuto imponendo un **contratto unico** a tutte le sorgenti e concentrando la riconciliazione in un solo punto, che unifichi i candidati secondo un vocabolario chiuso di esiti e che, in caso di disaccordo sul tipo, conservi entrambe le ipotesi anziché arbitrare autonomamente. Ogni rilevamento deve inoltre conservare la traccia delle sorgenti che l'hanno prodotto, come già impone il requisito di tracciabilità (cfr. 2.4.3): l'attribuzione di un errore diventa così un'operazione di lettura e non di ricostruzione.

4.1.3 Costo computazionale e tempi di elaborazione

L'analisi deve percorrere l'intero patrimonio documentale di un'organizzazione, e il costo unitario dell'inferenza si moltiplica quindi per il numero di documenti. Il rischio è particolarmente acuto per i modelli generativi, il cui costo per interrogazione — in energia e in denaro — è superiore di ordini di grandezza a quello dei modelli discriminativi e va sostenuto *a ogni* nuova analisi, non una sola volta: il confronto quantitativo fra le due famiglie, con i dati di letteratura che lo sostengono, è riportato nel Capitolo 5.

Un sistema che richiedesse giorni o settimane per completare una passata non sarebbe semplicemente lento: sarebbe inutilizzabile nel modo in cui è pensato, ossia come verifica periodica e non presidiata (cfr. 2.4.1). Il vincolo, inoltre, non è solo di tempo ma di dotazione hardware: richiedere acceleratori dedicati escluderebbe dall'adozione proprio le organizzazioni di piccole e medie dimensioni a cui il progetto si rivolge.

La risposta si articola su due fronti, entrambi già fissati in sede di requisiti. Sul piano della scelta tecnologica, l'analisi sistematica dell'intero corpus va affidata ad approcci computazionalmente leggeri ed eseguibili su hardware ordinario, mentre l'impiego di modelli generativi resta circoscritto a sottoinsiemi mirati (cfr. 2.3.2, 2.1.11). Sul piano dell'esecuzione, le analisi successive alla prima devono rielaborare i soli documenti effettivamente modificati (cfr. 2.3.6), così che il costo a regime risulti proporzionale al cambiamento e non al volume complessivo.

4.1.4 Dipendenza da modelli e librerie di terze parti

Il sistema poggia su una catena di componenti esterni: il motore di rilevamento e i suoi riconoscitori, i modelli di riconoscimento di entità e l'infrastruttura che li distribuisce, i lettori dei diversi formati documentali e il runtime che esegue localmente il modello linguistico. Ciascuno di essi può introdurre modifiche incompatibili, interrompere il proprio sviluppo o adottare condizioni di licenza inadatte all'impiego previsto.

La forma più insidiosa di questo rischio non è però l'obsolescenza, ma l'**incompatibilità di piattaforma**: le librerie di apprendimento automatico vincolano in modo stringente versione dell'interprete, architettura del processore e sistema operativo, cosicché una dipendenza può risultare indisponibile non perché abbandonata, ma perché l'ambiente su cui si opera non ne soddisfa i vincoli. Un sistema destinato a infrastrutture aziendali eterogenee (cfr. 2.3.3) è esposto a questa eventualità per definizione.

La mitigazione è duplice. Le dipendenze più onerose vanno dichiarate come componenti opzionali e caricate solo nel momento in cui servono davvero, così che l'indisponibilità di una di esse non impedisca il funzionamento del resto del sistema; e l'esecuzione di riferimento va collocata in un ambiente isolato e riproducibile, che renda irrilevante la piattaforma dell'host (cfr. 2.3.3). Sul piano della progettazione, il contratto unico imposto alle sorgenti di rilevamento (cfr. 4.1.2) e la definizione dichiarativa dei detector richiesta dal requisito di estensibilità (cfr. 2.3.4) fanno sì che la sostituzione di un componente resti circoscritta alla sua sede, senza propagarsi al resto dell'elaborazione.

4.1.5 Dilatazione del perimetro (*scope creep*)

Il dominio affrontato confina con almeno tre ambiti attigui e tutti pertinenti: l'anonimizzazione dei documenti, la gestione degli incidenti di sicurezza e la valutazione d'impatto sulla protezione dei dati. Ciascuno di essi appare, preso singolarmente, come un'estensione naturale — il sistema individua già i dati personali, sembrerebbe ragionevole che li mascherasse anche — e la loro somma eccede di gran lunga ciò che un lavoro di tesi può portare a compimento.

Il rischio non è di realizzare la funzionalità sbagliata, ma di realizzarne troppe in modo superficiale: un prototipo che tocca molti ambiti senza approfondirne alcuno non consente né una valutazione significativa né una conclusione difendibile, e la dispersione si paga sull'unica parte che dovrebbe essere solida, ossia la verifica di conformità.

La contromisura è il governo esplicito del perimetro. Le esclusioni sono dichiarate nei requisiti in forma altrettanto vincolante delle inclusioni (cfr. 2.2.4, 2.2.5), e ogni funzionalità che ecceda la verifica documentale va registrata come sviluppo futuro anziché essere assorbita in corso d'opera. Il criterio di ammissione adottato è la riconducibilità al confronto fra dato rilevato e dato dichiarato: ciò che non vi contribuisce resta fuori, per quanto affine.

4.1.6 Competenze richieste e curva di apprendimento

Il progetto richiede la padronanza simultanea di due domini distanti fra loro: da un lato l'elaborazione del linguaggio naturale, con le famiglie di modelli, i paradigmi di addestramento e le metriche che le governano; dall'altro il quadro normativo sulla protezione dei dati, in due ordinamenti che non coincidono. Nessuno dei due può essere trattato superficialmente senza compromettere il risultato: un errore di interpretazione normativa produrrebbe un sistema che verifica la cosa sbagliata, un errore di valutazione dei modelli produrrebbe una scelta tecnica indifendibile.

La manifestazione tipica del rischio è la decisione presa per inerzia: adottare la prima tecnologia incontrata perché documentata meglio, o assumere un'interpretazione normativa perché più semplice da implementare, rendendo di fatto arbitraria la parte più qualificante del lavoro.

La contromisura è duplice. Sul versante tecnico, ogni scelta rilevante va motivata su evidenze — misure proprie o dati di letteratura — anziché su preferenze, e il confronto fra le alternative va documentato insieme al suo esito (cfr. Capitolo 3, Capitolo 5). Sul versante normativo, il sistema è deliberatamente costruito per *non* pronunciarsi sulle questioni che richiedono una valutazione giuridica (cfr. 2.2.5): le assunzioni normative sono ridotte al minimo indispensabile e il giudizio resta all'organizzazione, il che circoscrive il danno di un'eventuale interpretazione imprecisa.

4.2 Rischi di prodotto

4.2.1 Accuratezza insufficiente nella detection delle PII

Il sistema non modifica i documenti: il suo esito è una segnalazione destinata a un giudizio umano. L'intero valore del prodotto dipende perciò dall'attendibilità di ciò che segnala, e un rilevamento impreciso non degrada semplicemente la qualità del risultato — ne annulla la funzione, perché un report di cui non ci si può fidare non viene usato.

Il rischio è aggravato dal fatto che l'accuratezza non è una grandezza uniforme: le sorgenti di rilevamento hanno profili complementari e molto diversi fra loro. Gli identificatori a formato rigido sono riconoscibili con precisione prossima al massimo, mentre le categorie dipendenti dal contesto — nomi di persona, indirizzi — dipendono interamente dalla qualità del modello di riconoscimento impiegato, le cui prestazioni variano sensibilmente fra le famiglie disponibili (cfr. 3.3.6). Una misura aggregata, quindi, nasconderebbe più di quanto riveli.

La contromisura è misurare in modo **disaggregato per categoria** e su un protocollo riproducibile (cfr. Capitolo 8), assumendo la copertura delle singole categorie — e non la media — come criterio di accettabilità: una categoria scoperta è un difetto anche quando l'indicatore

complessivo è buono, perché corrisponde a una classe di dati personali che il sistema non vede affatto.

4.2.2 Falsi positivi e impatto sull'usabilità

Una segnalazione errata ha un costo che non ricade sul sistema ma sull'operatore: ogni falso positivo si traduce in una verifica manuale infruttuosa. Poiché il tempo del DPO è la risorsa più scarsa dell'intero processo, un tasso elevato di segnalazioni errate non rende il report semplicemente meno preciso, ma progressivamente inutilizzato — l'esito peggiore per uno strumento di conformità, perché indistinguibile dall'assenza dello strumento stesso.

Il rischio si concentra sui documenti privi di dati personali ma ricchi di sequenze che vi somigliano — numeri di protocollo, importi, codici di articolo — e sulle categorie prive di un formato che le vincoli, che sono anche le più esposte a un riconoscimento fondato sul solo contesto. Non è la fragilità di un singolo approccio: i rilevatori basati su espressioni regolari producono falsi positivi sistematici quando una sequenza lecita ne imita un'altra — codici bibliografici scambiati per numeri di telefono ne sono un esempio documentato [25] — e i modelli generativi, pur raggiungendo un *recall* elevato, mostrano precisioni sensibilmente inferiori [33]. È inoltre un rischio che nessun corpus composto di soli documenti «popolati» potrebbe rilevare: senza documenti negativi, i falsi positivi sono per costruzione invisibili alla misura.

Per questa ragione il corpus di valutazione deve comprendere una quota deliberata di documenti **privi di qualsiasi PII**, corredati di elementi distrattori, così che la precisione sia una grandezza effettivamente misurata e non presunta (cfr. Capitolo 8). Sul piano della restituzione, ogni rilevamento deve esporre il proprio grado di conferma e le sorgenti che lo hanno prodotto (cfr. 2.4.3), consentendo al DPO di affrontare per prime le segnalazioni più solide anziché scorrere un elenco indifferenziato.

4.2.3 Falsi negativi e mancata rilevazione di PII critiche

Un dato personale presente e non rilevato non produce alcun segnale: è un rischio che resta *invisibile* tanto al sistema quanto all'organizzazione, e che si manifesta solo quando è troppo tardi per rimediare. L'asimmetria rispetto al rischio precedente è netta: un falso positivo costa una verifica, un falso negativo costa una non-conformità non rilevata, con conseguenze tanto più gravi quando riguarda le categorie particolari di dati previste dall'art. 9 GDPR [2] e dall'art. 5 lett. c nLPD [3] (cfr. 2.2.3).

La conseguenza è che i due errori non possono essere trattati come equivalenti in un bilanciamento simmetrico: il sistema deve dichiarare quale dei due preferisce commettere.

L'impostazione assunta è il principio **recall-first**, già fissato fra i requisiti di qualità (cfr. 2.4.2): nessuna sorgente deve scartare autonomamente un candidato debole, i casi di disaccordo

vanno conservati anziché risolti eliminando un'ipotesi, e la decisione finale è rimessa all'operatore. È una scelta che aumenta deliberatamente i falsi positivi per ridurre i falsi negativi, e va letta congiuntamente al rischio precedente (cfr. 4.2.2): la tensione fra i due non è risolta a livello di algoritmo ma arbitrata dal requisito di precisione (cfr. 2.4.2), che assegna la priorità al *recall* entro un limite di segnalazioni errate compatibile con l'usabilità.

4.2.4 Generalizzazione cross-domain (*domain-silo problem*)

I sistemi di rilevamento addestrati o messi a punto su un dominio specifico degradano sensibilmente quando applicati a testi di natura diversa, fenomeno documentato in letteratura come problema del *domain-silo* [13] (cfr. 3.1.1). Il contesto d'impiego previsto è però l'opposto di un dominio omogeneo: una condivisione documentale aziendale accosta contratti, corrispondenza, verbali, documentazione tecnica e fogli di calcolo, spesso nella stessa cartella.

Se il rilevamento fosse tarato implicitamente su una sola di queste famiglie, la copertura risulterebbe buona in fase di verifica e insoddisfacente sul campo, con un divario tanto più pericoloso quanto meno visibile: il sistema continuerebbe a produrre report apparentemente normali, semplicemente omettendo intere porzioni del patrimonio documentale.

La contromisura è duplice. Sul piano della scelta tecnologica, la preferenza per il riconoscimento *zero-shot* — in cui le categorie sono fornite come descrizioni testuali anziché apprese da un corpus di dominio — riduce alla radice la specializzazione implicita ed è del resto imposta dal requisito di operatività senza addestramento (cfr. 3.2.3, 2.3.7). Sul piano della verifica, il corpus di valutazione va costruito **deliberatamente eterogeneo** per genere documentale, lunghezza e presenza di dati personali. Il rischio residuo resta comunque significativo e va dichiarato. Su domini fortemente specialistici i modelli *zero-shot* subiscono cali marcati: applicati a testo clinico scendono a un F1 di circa 0.41, perdendo oltre metà delle entità rispetto ai modelli specializzati sul dominio [32]. Che la specializzazione resti necessaria lo conferma, indirettamente, l'esistenza di suite di modelli sviluppate appositamente per singoli ambiti [46]. Ciò rende la valutazione empirica sul corpus effettivo dell'organizzazione un passaggio non eludibile prima dell'adozione.

4.2.5 Supporto multilingue e multilocalizzazione

Il contesto normativo di riferimento — svizzero ed europeo — implica la coesistenza di più lingue nello stesso patrimonio documentale, e con esse di identificatori nazionali differenti: il numero AVS svizzero, il codice fiscale italiano e i rispettivi omologhi obbediscono a formati fra loro incompatibili. La dipendenza dalla lingua, inoltre, non riguarda solo il rilevamento: attraversa i modelli di riconoscimento, le espressioni regolari ancorate a parole chiave e persino l'interpretazione dei registri dei trattamenti.

La manifestazione tipica di questo rischio non è un errore visibile, ma un **silenzio**. Un componente che interpreta testo libero riconosce le formulazioni delle lingue per cui è stato predisposto e ignora le altre, senza poter distinguere l'espressione che non comprende da quella che non è presente: una durata di conservazione dichiarata nel registro in una lingua non prevista non verrebbe interpretata, il controllo corrispondente si disattiverebbe senza alcuna segnalazione, e un documento fuori termine risulterebbe conforme. È il modo di fallire più pericoloso proprio perché non produce alcun sintomo.

Va inoltre osservato che il trasferimento di un modello di de-identificazione da una lingua all'altra non è un'operazione neutra e costituisce un problema di ricerca a sé, affrontato in letteratura con tecniche di adattamento a pochi esempi [47]: assumere che un modello multilingue si comporti in modo uniforme su tutte le lingue che dichiara di coprire sarebbe imprudente.

Il rischio va mitigato mantenendo in **configurazione**, e non nel codice, sia il catalogo delle categorie sia le regole che le riconoscono, così che l'estensione a una lingua o a un ordinamento ulteriore sia un intervento dichiarativo (cfr. 2.3.4); e privilegiando, per il riconoscimento delle entità, modelli addestrati su più lingue. Resta il fatto che ogni componente che interpreta testo libero va verificato lingua per lingua: è una superficie di rischio che si riduce, ma non si elimina.

4.2.6 Scalabilità su filesystem di grandi dimensioni

L'analisi opera per attraversamento ricorsivo di interi alberi di cartelle, il cui contenuto non è noto in anticipo né per volume né per composizione. Il sistema deve quindi reggere corpus dell'ordine delle migliaia di documenti (cfr. 2.4.1), con dimensioni dei singoli file estremamente variabili e formati eterogenei, alcuni dei quali onerosi da interpretare.

I modi in cui questo rischio si manifesta sono tre, di gravità crescente: tempi di esecuzione incompatibili con una verifica periodica; consumo di memoria proporzionale alla dimensione del documento più grande incontrato; e, soprattutto, l'interruzione dell'intera analisi a causa di un singolo file illeggibile o malformato — l'esito peggiore, perché fa dipendere il risultato complessivo dall'elemento meno affidabile del corpus.

Le contromisure riguardano il modo in cui l'elaborazione va organizzata. Ogni documento deve essere elaborato in **isolamento**: un file che non può essere letto va registrato come errore lasciando proseguire l'analisi, cosicché nessun singolo elemento possa comprometterla. I componenti costosi da inizializzare vanno predisposti una sola volta e riutilizzati per l'intero lotto, e il testo estratto va mantenuto in memoria per la sola durata dell'elaborazione del documento, come già impone il vincolo di minimizzazione (cfr. 2.3.5). Le esecuzioni successive alla prima devono infine rielaborare i soli documenti modificati (cfr. 2.3.6), il che rende il costo a regime proporzionale al cambiamento anziché al volume. Resta un rischio

residuo sul fronte della parallelizzazione richiesta dal requisito di prestazioni (cfr. 2.4.1): su corpus molto ampi l'elaborazione incrementale riduce il lavoro ricorrente ma non quello della prima passata, per la quale la riduzione dei tempi passa dalla distribuzione del carico su più processi.

4.2.7 Qualità e completezza dei ROPA in input

Il registro delle attività di trattamento è il termine di paragone rispetto al quale ogni verdetto viene formulato, ma è un documento redatto dall'organizzazione, spesso in prosa sbrigativa: categorie indicate con formule generiche («dati anagrafici»), durate di conservazione espresse come criterio anziché come termine, attività dichiarate in modo incompleto. Il sistema, in altre parole, dipende da un input che non controlla e che è tipicamente meno preciso di quanto il confronto richiederebbe.

Un registro povero non produce un errore evidente, ma un verdetto *silenziosamente* inaffidabile: le categorie non riconducibili ad alcun tipo rilevabile non parteciperebbero al confronto e le durate non interpretabili disattiverrebbero il controllo di conservazione, con il risultato paradossale che un registro più vago genererebbe un esito apparentemente migliore — l'incentivo esattamente opposto a quello desiderabile.

Il principio adottato è che il sistema **non deve mascherare la povertà del registro, ma esporla**. Le categorie dichiarate che non si risolvono in alcun tipo rilevabile devono restare esplicite e venire segnalate al DPO anziché essere scambiate per conformi (cfr. 2.2.2); allo stesso modo, una conservazione espressa come criterio deve produrre una segnalazione di *non verificabilità* distinta dall'esito conforme (cfr. 2.2.1). La risoluzione delle categorie espresse in linguaggio naturale, infine, va sottoposta a conferma umana (cfr. 2.1.11), così che l'arricchimento del registro resti una decisione dell'organizzazione e non un'inferenza del sistema.

4.2.8 Conformità normativa e aggiornamento dei framework legali

Il sistema incorpora assunzioni normative: quali dati siano da considerare personali, quali godano di protezione rafforzata, quali obblighi il registro debba documentare. Tali assunzioni derivano da fonti in evoluzione — il Regolamento (UE) 2016/679 [2] e la legge federale svizzera sulla protezione dei dati [3] — e possono differire in ordinamenti ulteriori a cui un'organizzazione sia soggetta.

Il rischio è che il sistema invecchi senza darne segno: un catalogo di categorie fissato al momento della realizzazione diventerebbe progressivamente incompleto, continuando a produrre verdetti formalmente corretti ma fondati su una nozione di dato personale superata.

La contromisura è di progettazione prima che normativa. Il catalogo delle categorie di PII e le regole che le riconoscono devono essere **configurazione e non codice**, il che

rende l'adeguamento un'operazione di manutenzione dichiarativa (cfr. 2.3.4). Soprattutto, il sistema non giudica rispetto a una lista di obblighi cablata al proprio interno, ma rispetto al *registro dell'organizzazione*: l'onere dell'aggiornamento normativo resta dove la norma lo colloca, sul titolare del trattamento, mentre al sistema compete verificarne la coerenza con i dati effettivamente presenti (cfr. 2.2.5). Resta un rischio residuo dichiarato: l'allineamento puntuale fra le due normative di riferimento, che divergono nella struttura oltre che nel dettaglio, non è oggetto di una mappatura formale.

4.2.9 Sicurezza e protezione dei dati durante l'analisi

Il sistema è esposto a un paradosso costitutivo: per verificare il trattamento dei dati personali deve accedervi, e nel farlo diventa a sua volta un trattamento. Ogni artefatto che produce — il testo estratto, il registro interno delle rilevazioni, il report per il DPO — è un candidato a contenere dati personali, e uno strumento di conformità che diventasse la nuova concentrazione di dati sensibili dell'organizzazione avrebbe peggiorato la situazione che intendeva migliorare.

Il rischio si articola su due fronti: la *trasmissione* dei dati verso l'esterno, qualora l'elaborazione si appoggiasse a servizi remoti, e la loro *persistenza* all'interno del sistema, che moltiplicherebbe le copie da proteggere.

Sul primo fronte la contromisura è il vincolo di elaborazione interamente locale, che si estende ai modelli impiegati (cfr. 2.3.1, 2.3.2). Sul secondo, la minimizzazione va realizzata come proprietà **strutturale** e non come cautela procedurale: il registro deve conservare unicamente riferimenti — categoria, posizione, provenienza del rilevamento — e nessuna delle sue strutture dati deve prevedere una sede in cui il valore di una PII possa essere memorizzato, cosicché la violazione del principio non sia semplicemente sconsigliata ma impossibile per costruzione (cfr. 2.3.5, 2.1.8); il testo estratto deve vivere per la sola durata dell'elaborazione. Un rischio residuo permane e va riconosciuto: pur non contenendo valori, il registro costituisce una *mappa* di dove i dati personali si trovino, informazione di per sé sensibile che va protetta con le stesse cautele riservate all'infrastruttura che sorveglia.

4.3 Valutazione dei rischi

4.3.1 Criteri di classificazione

I rischi individuati nelle sezioni precedenti sono classificati secondo due dimensioni — la **probabilità** che si manifestino nel corso del progetto e l'**impatto** che avrebbero sul risultato — combinate in un livello di **priorità** che ordina gli interventi di mitigazione.

Le scale adottate sono *qualitative* a tre livelli. È una prassi riconosciuta nella valutazione del rischio quando manchi una base statistica su cui fondare stime di frequenza: la guida del

NIST alla conduzione delle valutazioni di rischio prevede scale qualitative per la probabilità e per l'impatto, e la loro combinazione in un livello di rischio complessivo [48]. È il caso di questo lavoro: non esistono serie storiche di progetti comparabili da cui derivare stime di frequenza, e una scala numerica trasmetterebbe una precisione che i dati sottostanti non giustificano. La probabilità è quindi espressa come *bassa*, *media* o *alta*, e l'impatto come *basso*, *moderato* o *critico*, dove per critico si intende un impatto che comprometterebbe la validità del risultato e non soltanto la sua qualità.

La combinazione delle due dimensioni non è simmetrica, ma riflette l'asimmetria già discussa a proposito dei falsi negativi (cfr. 4.2.3): l'impatto pesa più della probabilità. La priorità è pertanto *alta* quando a un impatto critico si accompagna una probabilità almeno media; *bassa* quando una probabilità bassa si accompagna a un impatto non critico; *media* in tutti i casi intermedi — inclusi quelli, come i rischi di sicurezza, in cui un impatto critico è associato a una probabilità contenuta.

Le valutazioni riportate nel paragrafo seguente sono **residue**: esprimono cioè il rischio così come risulta *dopo* l'applicazione delle contromisure descritte, non la sua gravità potenziale in loro assenza.

4.3.2 Matrice probabilità–impatto

La Tabella 4.1 posiziona i quindici rischi identificati secondo i criteri appena definiti.

Tabella 4.1: Valutazione dei rischi identificati: probabilità, impatto e priorità residua.

§	Rischio	Probabilità	Impatto	Priorità
<i>Rischi di progetto</i>				
4.1.1	Dataset di valutazione	Alta	Critico	Alta
4.1.2	Integrazione multi-approccio	Media	Moderato	Media
4.1.3	Costo computazionale	Alta	Critico	Alta
4.1.4	Dipendenze di terze parti	Alta	Moderato	Media
4.1.5	Dilatazione del perimetro	Alta	Moderato	Media
4.1.6	Curva di apprendimento	Media	Moderato	Media
<i>Rischi di prodotto</i>				
4.2.1	Accuratezza del rilevamento	Media	Critico	Alta
4.2.2	Falsi positivi	Alta	Moderato	Media
4.2.3	Falsi negativi	Media	Critico	Alta
4.2.4	Generalizzazione cross-domain	Alta	Critico	Alta
4.2.5	Supporto multilingue	Media	Moderato	Media
4.2.6	Scalabilità	Media	Moderato	Media
4.2.7	Qualità dei ROPA in input	Alta	Critico	Alta
4.2.8	Aggiornamento normativo	Bassa	Moderato	Bassa
4.2.9	Sicurezza dei dati in analisi	Bassa	Critico	Media

Il quadro che ne emerge presenta due caratteristiche degne di nota. La prima è che i rischi a priorità alta si concentrano sugli **ingressi** e sui **criteri di misura** del sistema — la disponibilità di un corpus di valutazione, la qualità del registro fornito, la capacità di generalizzare su documenti eterogenei — più che sulla sua realizzazione interna: è un sistema la cui affidabilità dipende in misura sostanziale da ciò che riceve dall'esterno. La seconda è che la priorità alta coincide esattamente con l'impatto critico: tutti e sei i rischi classificati come prioritari sono quelli che, manifestandosi, comprometterebbero la validità del risultato e non soltanto la sua qualità, mentre la probabilità stimata si limita a distinguerli fra loro senza mai promuovere alla fascia alta un rischio di impatto moderato. L'unica eccezione in senso opposto è la sicurezza dei dati durante l'analisi, che pur avendo impatto critico resta a priorità media in ragione della probabilità contenuta: la contromisura ne riduce la frequenza attesa, non la gravità potenziale.

4.4 Strategie di mitigazione

Le contromisure sono descritte insieme a ciascun rischio, poiché è in quella sede che se ne comprende la ragione. Questa sezione le rilegge trasversalmente, per mostrare come un numero ristretto di scelte di progettazione risponda a più rischi contemporaneamente: è la proprietà che rende la mitigazione sostenibile, perché una contromisura dedicata a ciascun rischio moltiplicherebbe la complessità del sistema fino a generarne di nuovi.

4.4.0.0.1 Disaccoppiamento tramite contratti e configurazione. Imporre un'interfaccia unica ai rilevatori e mantenere fuori dal codice il catalogo delle categorie e le regole di riconoscimento risponde a quattro rischi distinti: consente di comporre approcci eterogenei senza incoerenze (cfr. 4.1.2), di sostituire un componente divenuto indisponibile senza propagare la modifica (cfr. 4.1.4), di estendere il sistema a nuove lingue e ordinamenti (cfr. 4.2.5) e di adeguarlo all'evoluzione normativa (cfr. 4.2.8).

4.4.0.0.2 Asimmetria dichiarata fra i due tipi di errore. Il principio *recall-first*, unito all'esposizione del grado di conferma di ogni rilevamento, governa congiuntamente i due rischi speculari dei falsi negativi e dei falsi positivi (cfr. 4.2.3, 4.2.2). La scelta non elimina né l'uno né l'altro: stabilisce quale dei due il sistema preferisce commettere e fornisce all'operatore lo strumento per assorbire il costo del secondo.

4.4.0.0.3 Minimizzazione come proprietà strutturale. Non conservare i valori delle PII in alcuna struttura dati, e non trasmettere nulla all'esterno, riduce l'esposizione del sistema alla violazione che esso stesso intende prevenire (cfr. 4.2.9). Il punto qualificante è che la minimizzazione non va affidata a una cautela procedurale, ma resa impossibile da violare

per costruzione: se nel modello dei dati non esiste una sede in cui un valore potrebbe essere scritto, non esiste nemmeno il modo di violare il principio per distrazione.

4.4.0.0.4 Misura riproducibile su corpus costruiti a questo scopo. La generazione di corpus sintetici a partire da un seme, articolati su due livelli e comprendenti una quota deliberata di documenti privi di dati personali, è la contromisura di quattro rischi: rende possibile la valutazione in assenza di benchmark pubblici (cfr. 4.1.1), consente di misurare l'accuratezza per singola categoria (cfr. 4.2.1), rende osservabili i falsi positivi (cfr. 4.2.2) e mette alla prova la generalizzazione su documenti eterogenei (cfr. 4.2.4).

4.4.0.0.5 Contenimento del costo di esecuzione. L'impiego di approcci leggeri per l'analisi sistematica, l'isolamento dell'elaborazione di ciascun documento e la rielaborazione dei soli documenti modificati agiscono insieme sui rischi di sostenibilità computazionale e di scalabilità (cfr. 4.1.3, 4.2.6), mentre l'esecuzione in un ambiente isolato e riproducibile neutralizza la dipendenza dalla piattaforma su cui il sistema viene installato (cfr. 4.1.4).

4.4.0.0.6 Esposizione delle lacune anziché loro occultamento. Dove il sistema non è in grado di pronunciarsi — una categoria dichiarata che non si risolve in alcun tipo rilevabile, una durata di conservazione espressa come criterio — l'esito non è il silenzio ma una segnalazione dedicata, distinta dall'esito conforme (cfr. 4.2.7). È la contromisura al più insidioso dei modi di fallire, quello in cui l'assenza di verifica è indistinguibile dalla verifica superata.

4.4.0.0.7 Governo esplicito del perimetro. La formulazione dei requisiti, che enuncia le esclusioni con la stessa forza delle inclusioni, e l'obbligo di motivare ogni decisione tecnica su evidenze anziché su preferenze costituiscono nel loro insieme la contromisura ai rischi di natura organizzativa (cfr. 4.1.5, 4.1.6): tengono traccia di ciò che è deliberatamente fuori ambito e impediscono che le scelte più qualificanti del lavoro siano compiute per inerzia.

4.4.0.0.8 Rischi accettati. Alcuni rischi residui non sono mitigati e vengono assunti consapevolmente, poiché la loro riduzione richiederebbe uno sforzo sproporzionato rispetto agli obiettivi del lavoro: la rinuncia, in questa fase, alla parallelizzazione dell'analisi, il carattere ottimistico di una valutazione condotta su corpus sintetici, la natura intrinsecamente sensibile del registro delle rilevazioni in quanto mappa della collocazione dei dati personali, e la necessità di una validazione empirica sul corpus specifico dell'organizzazione prima di ogni adozione operativa. Sono limiti dichiarati, non trascurati, e come tali sono ripresi fra gli sviluppi futuri.

Capitolo 5

Studi effettuati

Le scelte tecnologiche che reggono il sistema non sono state compiute per preferenza: ciascuna è stata preceduta da uno studio, condotto su evidenze di letteratura o su misurazioni dirette, il cui esito è riportato qui insieme al ragionamento che vi conduce. Il capitolo precede deliberatamente la descrizione dell'architettura, perché è la sede in cui le decisioni che l'architettura poi incarna vengono *prese*: leggerle a valle del progetto le farebbe apparire come giustificazioni posteriori, quando invece ne sono il presupposto.

Gli studi sono quattro, e procedono dal generale al particolare. Il primo stabilisce **dove l'AI generativa convenga e dove no**, questione che da sola determina l'impianto del rilevamento; il secondo ne isola un corollario spesso frainteso, la differenza fra costo di addestramento e costo di inferenza. I due successivi scelgono i componenti: se costruire il rilevamento sopra un motore esistente, e quale modello generativo adottare per gli impieghi selettivi che il primo studio lascia aperti. La verifica sperimentale sul sistema realizzato è invece nel Capitolo 8.

5.1 Uso dell'AI in rapporto al suo impatto ambientale ed economico

Nella scelta della metodologia per il rilevamento delle PII all'interno dei documenti si è posto un problema che le sole metriche di accuratezza non permettono di risolvere. I candidati (cfr. 3.3) sono tre — rule-based, NER e AI generativa — e la letteratura è concorde nell'attribuire ai modelli generativi il *recall* più elevato, in particolare sulle PII che dipendono dal contesto, dai riferimenti impliciti e dai collegamenti fra più documenti: proprio le categorie che sfuggono agli altri approcci. Se il criterio fosse la sola capacità di rilevamento, la scelta sarebbe già compiuta.

Il criterio non può però essere quello, perché il sistema qui progettato non analizza un documento: ne analizza *tutti*, ripetutamente. È questa condizione d'impiego — inferenza sistematica e **ricorrente** su un intero patrimonio documentale — a rendere il costo per

interrogazione un parametro architetturale e non un dettaglio di esercizio. Questa sezione raccoglie le evidenze quantitative su cui la scelta si fonda; i capitoli che vi rimandano (cfr. 3.3.6, 4.1.3, 2.3.2) ne assumono la conclusione senza ripeterne i dati.

5.1.1 Il divario energetico fra modelli discriminativi e generativi

La misurazione di riferimento è quella di Luccioni, Jernite e Strubell, che confrontano su hardware omogeneo (NVIDIA A100) il consumo di famiglie di modelli diverse a parità di compito [49]. Il risultato rilevante per questo lavoro è la separazione netta fra due regimi: i modelli *discriminativi* specializzati su un compito — la famiglia a cui appartengono gli encoder NER — consumano circa **0.002 kWh ogni 1000 inferenze**, mentre i modelli *generativi* decoder-only richiedono circa **0.047 kWh per 1000 interrogazioni** nella generazione di testo. Il rapporto è di oltre un ordine di grandezza, e non dipende dalla qualità dell'implementazione ma dal modo in cui le due architetture producono il risultato: l'una emette una classificazione in un solo passaggio, l'altra genera token in modo autoregressivo.

Il divario si riproduce sul piano economico. A parità di F1 su compiti di classificazione testuale, l'inferenza di un encoder come RoBERTa costa circa \$0.044 per 1000 esempi contro i \$13.50 di GPT-4: un rapporto prossimo a **300:1** [50]. La profilazione sistematica del costo dell'inferenza generativa — in energia, latenza e denaro — è del resto divenuta essa stessa oggetto di studio, segno che il problema è riconosciuto come strutturale e non contingente [51].

Un terzo dato completa il quadro e va richiamato per correttezza, perché lavora *contro* l'argomento appena esposto: l'addestramento di un modello generativo consuma ordini di grandezza più energia di una singola inferenza, tanto che il costo cumulato delle inferenze eguaglia quello del pre-addestramento solo dopo un numero di interrogazioni compreso fra 200 e 590 milioni [49]. Su scala globale, dunque, l'inferenza pesa meno dell'addestramento. Ciò non attenua però il problema qui in esame, per la ragione che segue: l'organizzazione che adotta il sistema non sostiene il costo di addestramento — che è già stato sostenuto da altri, una volta sola — ma sostiene per intero e ripetutamente quello di inferenza.

5.1.2 Il quadro comparativo

La Tabella 5.1 raccoglie i dati discussi, in corrispondenza degli approcci confrontati nella Tabella 3.1 sul piano dell'accuratezza.

5.1.3 Conseguenza per l'architettura del sistema

I dati riportati non conducono a un rifiuto dell'AI generativa, ma a una sua collocazione precisa, che si può enunciare in una regola: **l'AI generativa non va impiegata dove il costo**

Tabella 5.1: Costo energetico ed economico degli approcci di PII detection, secondo le evidenze di letteratura.

Approccio	Energia di addestramento	Energia / 1k inferenze	Costo / 1k inferenze
Regex / rule-based	Nessuna	Trascurabile	Trascurabile
NER supervised	Una tantum ^a	~0.002 kWh ^b	~\$0.044 ^c
NER zero-shot (GLiNER)	Nessuna ^d	~0.002 kWh ^b	~\$0.044 ^c
LLM generativo (GPT-4o)	Altissima ^e	~0.047 kWh ^b	~\$13.50 ^c
Pipeline ibrida	Bassa (sola componente NER)	Basso-medio ^f	Basso-medio ^f

^a Il fine-tuning di un encoder richiede ~44 h GPU complessive su A100: costo sostenuto una volta sola e non ricorrente [27].

^b Misure su A100: classificazione testuale ~0.002 kWh/1k inferenze, generazione di testo ~0.047 kWh/1k inferenze [49].

^c Ordine di grandezza per compiti di classificazione a parità di F1: RoBERTa contro GPT-4 [50]. ^d L'approccio zero-shot impiega un encoder pre-addestrato senza fine-tuning specifico per il compito [22]. ^e Il pre-addestramento supera l'inferenza cumulata fino a 200–590 M interrogazioni [49]. ^f Regex e NER su tutto il corpus, modello generativo sui soli casi ambigui: il costo complessivo è dominato dalla componente NER [7].

si moltiplica per il numero di documenti. Ne discendono due impieghi opposti dello stesso strumento.

Il rilevamento sistematico attraversa l'intero patrimonio documentale a ogni esecuzione: su un corpus di alcune migliaia di documenti, ciascuno dei quali richiede più interrogazioni per essere analizzato per intero, il numero di inferenze per singola passata si colloca nell'ordine delle decine di migliaia, e va moltiplicato per la frequenza di riesecuzione. In questo regime la differenza di un ordine di grandezza nel costo unitario non è un'inefficienza tollerabile: è ciò che separa un sistema eseguibile periodicamente su hardware aziendale da uno che non lo è. Il rilevamento sistematico è pertanto affidato ad approcci discriminativi.

Gli impieghi **selettivi** (cfr. 2.1.11) si trovano nella condizione opposta: la normalizzazione delle categorie dichiarate in un registro dei trattamenti si esegue una volta per registro, l'analisi approfondita si applica a un campione dell'ordine dell'uno per cento, l'interpretazione della struttura di cartelle riguarda l'albero e non i documenti. Qui il numero di inferenze non scala con il corpus, il costo complessivo resta di ordini di grandezza inferiore, e la comprensione semantica del modello generativo apporta un valore che gli altri approcci non possono fornire. La conclusione, quindi, non è che l'AI generativa costi troppo: è che *costa troppo per interrogazione*, e va perciò riservata ai compiti in cui le interrogazioni sono poche.

5.2 Costo di addestramento e costo di inferenza

L'argomento dell'impatto ambientale viene talvolta impiegato per escludere in blocco l'uso di modelli di apprendimento automatico. Applicato a questo lavoro, sarebbe un uso improprio: l'argomento della sezione precedente esclude l'**inferenza generativa ricorrente**, non l'impiego di modelli in quanto tale, e in particolare non esclude l'eventuale *fine-tuning* di un encoder locale. La distinzione merita di essere resa esplicita, perché le due voci di costo hanno natura e ordine di grandezza incomparabili.

Il **costo di inferenza** di un modello generativo è *ricorrente* e proporzionale al lavoro svolto: si sostiene per ogni documento, e nuovamente a ogni riesecuzione dell'analisi. Un sistema pensato per la verifica periodica lo paga indefinitamente, e il totale cresce con il tempo di esercizio e con la dimensione del patrimonio documentale. È un costo che l'organizzazione adottante sostiene per intero.

Il **costo di fine-tuning** di un piccolo encoder è invece *una tantum*: nell'ordine delle decine di ore GPU complessive [27], sostenute una volta sola, dopo le quali il modello risultante ha lo stesso profilo di consumo in inferenza di quello da cui deriva — circa 0.002 kWh ogni 1000 inferenze [49], ossia un ventesimo di un modello generativo. Ammortizzato sull'intera vita operativa del sistema, tale costo diventa marginale.

Ne segue che le due decisioni sono **indipendenti** e vanno motivate separatamente. L'esclusione dell'AI generativa dal rilevamento sistematico si fonda sul costo ricorrente, ed è definitiva. La rinuncia al fine-tuning di un encoder, invece, non si fonda sull'impatto ambientale — che sarebbe un argomento debole — bensì sul requisito di operatività immediata senza addestramento su dati dell'organizzazione (cfr. 2.3.7) e sul costo, questo sì proibitivo, dell'annotazione di un corpus di dati personali reali (cfr. 4.1.1). Sono due vincoli di natura diversa, e confonderli indebolirebbe entrambi.

5.3 Riutilizzo di un motore di detection esistente: valutazione di Microsoft Presidio

Come emerso dallo stato dell'arte (§3.5), Microsoft Presidio non risolve il problema di governance affrontato dal progetto; il suo *motore di rilevamento*, però, copre gran parte del livello regex e NER che il sistema deve comunque realizzare. Si valuta quindi se **costruire tale livello sopra Presidio** anziché scriverlo da zero. La decisione è affidata a una **valutazione empirica**: questa sezione ne fissa i termini, mentre l'esito — l'adozione di Presidio, con GLiNER per il livello NER — è riportato nel §8.1.

Due modelli di integrazione sono possibili, con implicazioni opposte sull'impianto del sistema:

- usare l'*AnalyzerEngine* di Presidio come **orchestratore** dell'intera detection: massimizza il riutilizzo, ma affiderebbe a un componente esterno la *riconciliazione* fra le sorgenti di rilevamento — cioè il punto su cui poggia la contromisura al rischio di integrazione (cfr. 4.1.2) e in cui risiede il contributo proprio del lavoro;
- incapsulare Presidio **dietro il contratto unico** imposto a ogni sorgente, come un detector fra gli altri che ne traduce i risultati nel formato interno: la riconciliazione e l'eventuale sorgente generativa restano di competenza del progetto, e Presidio diventa il fornitore di espressioni regolari, checksum e NER.

Il secondo modello è preferito, perché preserva il contributo proprio del lavoro e rende la scelta **reversibile**: un contratto unico consente di sostituire il fornitore senza toccare il resto del sistema. L'architettura che ne discende è descritta nel Capitolo 6.

I criteri di valutazione sono:

- **copertura** delle categorie del catalogo: Presidio fornisce riconoscitori pre-costruiti per gran parte di esse (email, IBAN, carta di credito, indirizzo IP, codice fiscale italiano), diversi con **validazione a checksum** già inclusa — proprio ciò che il progetto aveva rimandato;
- **estensibilità** sugli identificatori mancanti: il numero AVS svizzero richiederebbe comunque un riconoscitore su misura, come nell'implementazione attuale;
- **integrazione del livello NER**: Presidio supporta ufficialmente modelli esterni, incluso GLiNER, coerente con la scelta zero-shot (§3.2.3);
- **vincoli di progetto**: esecuzione interamente on-premise e configurabilità da file, coerente con l'estensibilità config-driven richiesta.

5.3.0.0.1 Profilo hardware. Un ulteriore elemento a favore riguarda i requisiti di calcolo. Il motore di Presidio e il livello di rilevamento basato su spaCy operano interamente su **CPU**, senza richiedere una GPU dedicata; l'accelerazione a GPU è soltanto opzionale ed è riservata ai modelli NER più onerosi (transformers, GLiNER), verso i quali il sistema ripiega comunque in automatico sulla CPU qualora la GPU non sia presente. Questo profilo è coerente con il vincolo di esecuzione on-premise su infrastruttura di PMI, dove hardware accelerato è raramente disponibile, e costituisce quindi un rafforzamento del posizionamento del progetto più che una limitazione.

Restano in ogni caso a carico del progetto, indipendentemente dall'esito: il riconoscitore per il numero AVS, il motore di riconciliazione delle sorgenti con i relativi livelli di conferma, la sorgente generativa selettiva e l'intero blocco di ingestione ROPA e verifica di conformità.

Il giudizio è affidato a una **misurazione diretta**: Presidio viene incapsulato come detector e valutato sullo stesso corpus annotato e con la stessa procedura di scoring (precision, recall, F1) impiegati per la baseline regex, così da confrontare i due approcci sullo stesso metro; l'esito del confronto è nel §8.1.

5.4 Scelta dello Small Language Model per gli impieghi selettivi dell'AI

Il progetto fa dell'AI generativa un uso **selettivo**: l'uso di LLM è escluso dal rilevamento sistematico e ricorrente per le ragioni di costo esposte in 5.1, e riservato a compiti mirati e non ricorrenti. Sono tre i compiti candidati: (1) la **normalizzazione del ROPA** (§6.1.4) —

ricondere descrizioni sbrigative («dati anagrafici») a un vocabolario chiuso di categorie, cioè estrazione di informazione strutturata a output vincolato; (2) l'**analisi profonda di documenti campione** (tipicamente 1 su 100/1000, dove la latenza non è vincolante); (3) l'**analisi della struttura del file system** per individuare collocazioni anomale (ad esempio un curriculum in una cartella `turni/`). I vincoli discendono dal progetto: il modello deve essere **piccolo, eseguibile su CPU, containerizzabile e scalabile** su hardware commodity. Si adotta la definizione operativa di *Small Language Model* (SLM) del survey di riferimento — modelli *decoder-only* da 100M a 5B parametri [52]. Come per Presidio (§5.3), la scelta è **empirica**: seguono i criteri, le misurazioni raccolte in letteratura e la conclusione su quale modello sia il migliore.

5.4.1 Criteri di selezione

I criteri, in ordine di priorità per il caso d'uso, e le fonti metodologiche che li sostengono sono riassunti nella Tabella 5.2.

Tabella 5.2: Criteri di selezione dello SLM e relative fonti.

Criterio	Motivazione nel progetto	Fonte
Eseguibilità su CPU (throughput, memoria)	Ambiente target senza GPU	[53]
Quantizzabilità (GGUF / llama.cpp)	Stare in RAM commodity e in Docker	[54, 55]
<i>Instruction following</i> / output strutturato	Task 1 e 3 (aderenza a schema/istruzioni)	[56, 57]
Licenza permissiva + on-premise	Sovranità del dato (dati sensibili, nLPD)	[58]
Contesto lungo	Task 2 e 3 (documenti/alberi ampi)	[59, 60]

5.4.2 Fattibilità dell'inferenza su CPU

La fattibilità dell'inferenza su sola CPU non è un'assunzione ma un risultato empirico. La **quantizzazione a 4 bit** riduce l'occupazione di memoria di circa $4\times$ rispetto a `float16` con perdita di accuratezza mediamente inferiore all'1% [54]; nel formato GGUF (K-quant Q4_K_M), uno studio su hardware consumer riporta il passaggio da 2.6 a 47.9 token/s (circa $18\times$) con la RAM che scende da 14.39 GB a $\sim 1.15\text{--}1.51$ GB (oltre il 90% in meno) [61], e un secondo studio conferma su CPU uno *speedup* di $1.5\text{--}2.5\times$ da F16 a Q4 [62]. Il motore di inferenza di riferimento è `llama.cpp` (grafo compilato in C++, formato GGUF a file singolo), che il progetto serve concretamente tramite **Ollama** — un server che incapsula `llama.cpp`, espone i modelli GGUF su porta ed è configurato da variabile d'ambiente (§7.5.2); le alternative orientate alla GPU non sono adatte, poiché il backend CPU di `vLLM` è stato misurato a 3.07 token/s su un modello da 0.5B e va in *out-of-memory* oltre 1.5B [53]. Il costo va dichiarato: su Llama-3.2-3B l'MMLU passa da 64.2 a 61.8 con GGUF Q4_K_M [63].

Lo studio più direttamente trasferibile al progetto misura sette modelli *instruction-tuned* quantizzati a 4 bit su tre ambienti CPU-only, in un runtime containerizzato in Docker [54]. I dati di throughput e di memoria sono riportati nelle Tabelle 5.3 e 5.4 (E5-2695 v2 = Xeon

12-core del 2013 senza AVX2/AVX-512; 8480+ = Xeon Sapphire Rapids 56-core; i7-13700H = laptop 14-core).

Tabella 5.3: Throughput (token/s), 4-bit GGUF su CPU [54].

Modello	Param.	E5-2695 v2	Xeon 8480+	Core i7-13700H
Gemma-3-1B-it	1B	30	100	70
Llama-3.2-1B-Instruct	1.2B	25	120	80
Phi-4-mini-instruct	3.8B	15	80	40
Qwen2.5-7B-Instruct	7.6B	8	45	15
DeepSeek-R1-Distill-Llama-8B	8B	7	40	10
Gemma-3-12B-it	12B	5	30	9
Mistral-Small-3.1-24B	24B	2	15	4

Tabella 5.4: Occupazione di memoria: FP16 vs 4-bit (Q4) e RAM minima [54].

Modello	Param.	FP16 (GB)	Q4 (GB)	RAM min.
Llama-3.2-1B-Instruct	1.2B	~2.4	0.5	4 GB+
Gemma-3-1B-it	1B	~2.0	0.6	4 GB+
Phi-4-mini-instruct	3.8B	~7.6	1.9	8 GB+
Qwen2.5-7B-Instruct	7.6B	~15	3.7	8 GB+
Gemma-3-12B-it	12B	~24	6.7	12 GB+

I modelli 1–4B offrono interazione quasi in tempo reale su CPU; i 6–8B sono lo *sweet spot* qualità/latenza su hardware datato. Per la scalabilità, oltre 8–16 thread i ritorni sono marginali: conviene eseguire **più istanze in parallelo** su gruppi di core disgiunti (un thread per core fisico, con *pinning* NUMA) piuttosto che una singola istanza su tutti i core [54]. Le versioni *instruction-tuned* sono essenziali. Il caso di studio dell'autore fa co-risiedere due modelli piccoli in Docker su una VM da 16 vCPU / 32 GB con latenza mediana inferiore a 2 s su Xeon moderno — l'analogo diretto del deployment previsto qui [54].

5.4.3 Modelli candidati

5.4.3.0.1 Qwen2.5 (Alibaba). Taglie 0.5B–72B con versioni quantizzate ufficiali; pre-addestramento su 18T token e supporto multilingue (29+ lingue) [64]. La generazione precedente offre numeri puntuali (Qwen2-1.5B: MMLU 56.5, GSM8K 58.5, HumanEval 31.1) [65]. La granularità di taglie (0.5B/1.5B/3B) lo rende tra i più flessibili per CPU.

5.4.3.0.2 Llama 3.2 1B/3B (Meta). Costruiti con *pruning* e distillazione per l'*edge*, contesto fino a 128K token; sul 3B, valori auto-riportati: IFEval 77.4, MMLU 63.4, *tool use* BFCL V2 67.0 (contro 25.7 del 1B) [60]. È il più efficiente in energia/latenza della sua fascia (§5.4.6).

5.4.3.0.3 Phi-3-mini / Phi-4-mini (Microsoft). Phi-3-mini (3.8B) raggiunge MMLU 69% e MT-bench 8.38 pur girando su telefono [66]; Phi-4-mini (3.8B) aggiunge vocabolario a 200K token, *grouped-query attention* e **instruction following e function calling nettamente migliorati**, con matematica/codice pari a modelli di taglia doppia [67, 66]. È il candidato più forte nella fascia ~4B per output vincolato.

5.4.3.0.4 Gemma 3 (Google). Famiglia 1B–27B con distillazione, contesto $\geq 128K$ e KV-cache alleggerita; Gemma3-4B-IT è dichiarato competitivo con Gemma2-27B-IT dopo il nuovo post-training [59].

5.4.3.0.5 SmolLM2 (Hugging Face). Famiglia 135M/360M/1.7B completamente aperta; SmolLM2-1.7B supera Qwen2.5-1.5B di ~6 punti su MMLU-Pro [68]. Opzione minimale/riproducibile, meno adatta a ragionamento complesso.

5.4.3.0.6 IBM Granite 3.x. Famiglia dense 2B/8B e MoE 1B/3B sotto licenza Apache 2.0, progettata per l'impresa e l'on-premise (RAG, classificazione, *entity extraction*, *tool use*); i modelli MoE sono indicati per CPU/edge [58]. Rilevante soprattutto sul piano della sovranità del dato: la licenza e l'orientamento dichiarato all'installazione presso il titolare del trattamento non pongono vincoli all'esecuzione interamente locale.

5.4.4 Analisi per task

5.4.4.0.1 Task 1 — ROPA verso categorie. È estrazione a output vincolato. Il benchmark *LLMStructBench* valuta 22 modelli aperti (da 0.6B a 70B) su 995 campioni verificati manualmente e conclude che **la strategia di *prompting* conta più della taglia**, e che fornire lo **schema JSON nel prompt** è l'opzione più sicura per output parsabili, specialmente sui modelli piccoli [56]. Ciò richiama la distinzione tra output *corretto* e *utilizzabile*: una risposta che viola lo schema è inservibile quanto una sbagliata [69]. Ne discende la raccomandazione: non il modello più grande, ma *constrained decoding*/schema nel prompt più un modello con forte *instruction following* — nella fascia CPU i migliori sono Phi-4-mini e Llama-3.2-3B; per formati verificabili è pertinente IFEval-FC [57].

5.4.4.0.2 Task 2 — Analisi di documenti campione. Cadendo il vincolo di throughput, dominano ragionamento e contesto lungo; lavorando sul contenuto fornito (setting *RAG*) la conoscenza parametrica pesa meno e anche modelli piccoli rendono bene [54]. È quindi lecito salire di taglia: un 7B (Qwen2.5-7B) o Phi-4-mini restano eseguibili su CPU (~45 e ~80 token/s su Xeon moderno).

5.4.4.0.3 Task 3 — Struttura del file system. **Lacuna dichiarata:** non è stato individuato un benchmark che misuri il rilevamento di anomalie su gerarchie di file system. In assenza

di metrica diretta, l'ancoraggio è alle capacità più prossime — *instruction following* (Llama-3.2-3B, Phi-4-mini) e contesto lungo (modelli a 128K) — ma la validazione su un dataset costruito ad hoc resta necessaria.

5.4.5 Confronto e conclusione

Nessun singolo modello *domina* su tutti e tre i task; tuttavia, per contenere la complessità del deployment — una sola immagine, un solo peso da distribuire e mantenere, nessuna gestione di co-residenza di modelli diversi — il progetto adotta **un unico modello per l'intera pipeline**. La Tabella 5.5 mostra i candidati migliori per ciascun task; il criterio di scelta è quindi l'**intersezione**: quale modello sia adeguato a tutti e tre insieme.

Tabella 5.5: Raccomandazione per task (fascia CPU).

Task	Requisito dominante	Candidati
1 — ROPA→categorie	Output strutturato + instruction following	Phi-4-mini 3.8B, Llama-3.2-3B, Qwen2.5-3B
2 — Analisi documenti	Ragionamento + contesto lungo (latenza libera)	Qwen2.5-7B, Phi-4-mini 3.8B
3 — Struttura FS	Instruction following + contesto lungo	Llama-3.2-3B, Phi-4-mini, Gemma-3-4B

Il modello scelto è Phi-4-mini (3.8B). È l'**unico candidato presente in tutte e tre le righe** della Tabella 5.5: offre il miglior equilibrio *instruction-following/function-calling* nella fascia ~4B [67], ciò che il Task 1 e il Task 3 richiedono come requisito dominante, e resta al contempo adeguato al Task 2, dove la caduta del vincolo di throughput rende sufficiente un modello da ~4B lavorando in *RAG* sul contenuto fornito [54]. Sul piano operativo è la scelta più leggera compatibile con tutti i compiti: **1.9 GB in Q4** e ~80 token/s su Xeon moderno, un solo peso GGUF servito da un solo `llama.cpp` in Docker [54]. Rinunciare a un secondo modello dedicato (ad esempio Qwen2.5-7B per il ragionamento del Task 2) è un compromesso consapevole: si accetta un margine di qualità potenzialmente inferiore sul Task 2 in cambio di un deployment radicalmente più semplice e portabile, coerente con il vincolo di containerizzazione del progetto. Come **alternativa orientata a sovranità e governance**, a parità di modello unico, resta IBM Granite 3.x (dense 2B/8B, Apache 2.0), la cui licenza e il cui orientamento all'esecuzione presso il titolare rispondono al medesimo vincolo di località dell'inferenza [58].

La scelta resta soggetta a **validazione empirica** sul dominio, come per Presidio: un *gold set* etichettato (per il Task 1, frasi di «categorie di dati» con l'insieme atteso di `pii_type`) su cui misurare precision/recall/F1 a prompt fissato, riuscendo l'impianto di scoring del progetto. Il componente è isolato dietro il contratto `CategoryMapper`, per cui cambiare modello — o passare a un dizionario deterministico — non tocca il resto della pipeline.

5.4.6 Energia, efficienza e limiti

Sul versante *edge*, uno studio sull'impronta energetica riporta che su Raspberry Pi 5 Llama 3.2 è il più efficiente in potenza (fino a 2.05 Wh) e a più bassa latenza (4.06 s su MMLU),

mentre Phi-3 Mini raggiunge l'accuratezza più alta (MMLU 62.3%) ma con la latenza peggiore (34.36 s) [70]. Esiste dunque un compromesso esplicito accuratezza–energia–latenza: poiché i tre task sono a **bassa frequenza** (non ricorrenti sull'intero corpus), è accettabile un modello più accurato e più lento — l'inverso del ragionamento applicato al rilevamento sistematico, dove è la ricorrenza dell'inferenza a escludere l'AI generativa (cfr. 5.1).

5.4.6.0.1 Limiti delle evidenze raccolte. (i) i numeri dei *technical report* sono **auto-riportati** e vanno trattati come dichiarazioni di parte, con rischio di *benchmark contamination* [66]; (ii) la quantizzazione a 4 bit introduce un decadimento reale (es. MMLU 64.2→61.8), per cui sul Task 1 va valutato Q8 come compromesso [63]; (iii) per il Task 3 nessun benchmark è diretto: l'ancoraggio a *instruction following* e contesto lungo è un *proxy*; (iv) per l'output strutturato la variabile dominante è la strategia di prompt, non solo il modello [56]; (v) i throughput dipendono dalla microarchitettura (AVX-512/VNNI/AMX) e vanno riprodotti sull'hardware target [53].

Capitolo 6

Architettura del sistema

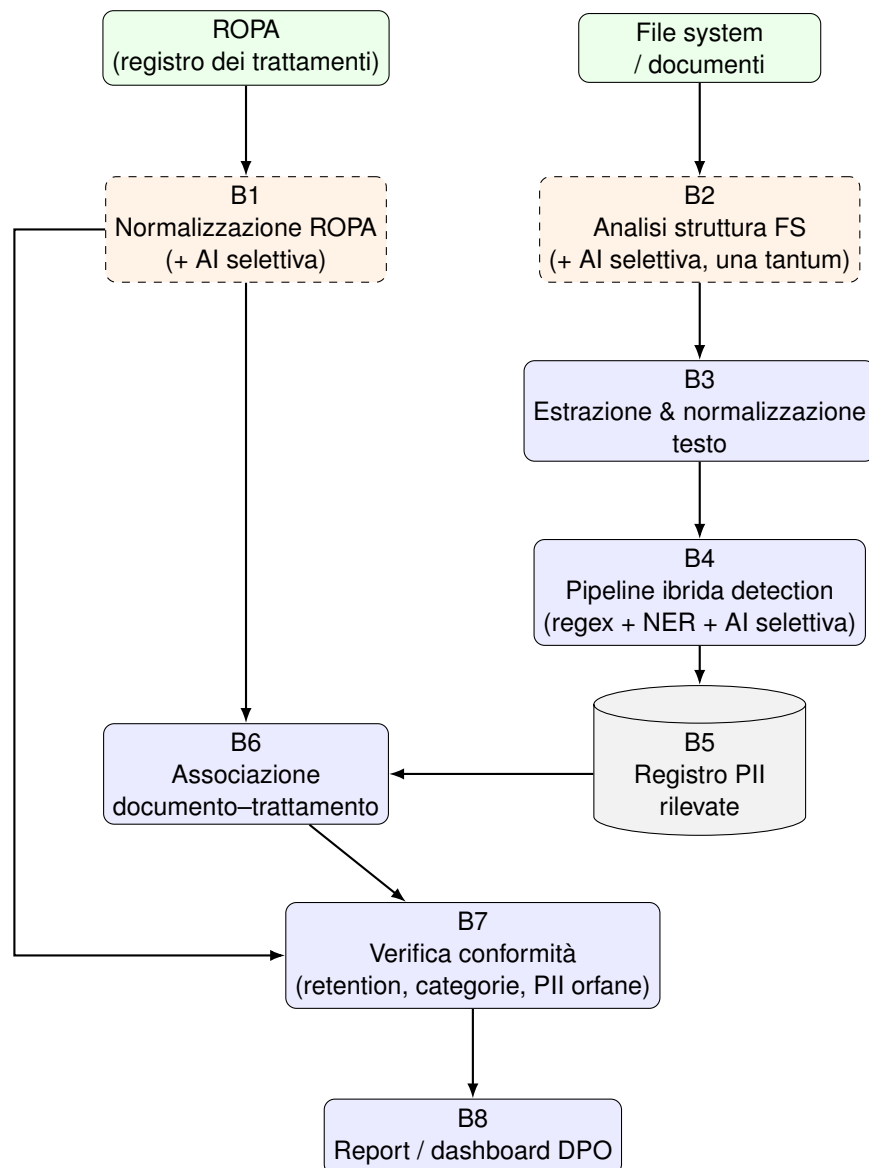


Figura 6.1: Architettura a macro-blocchi del sistema

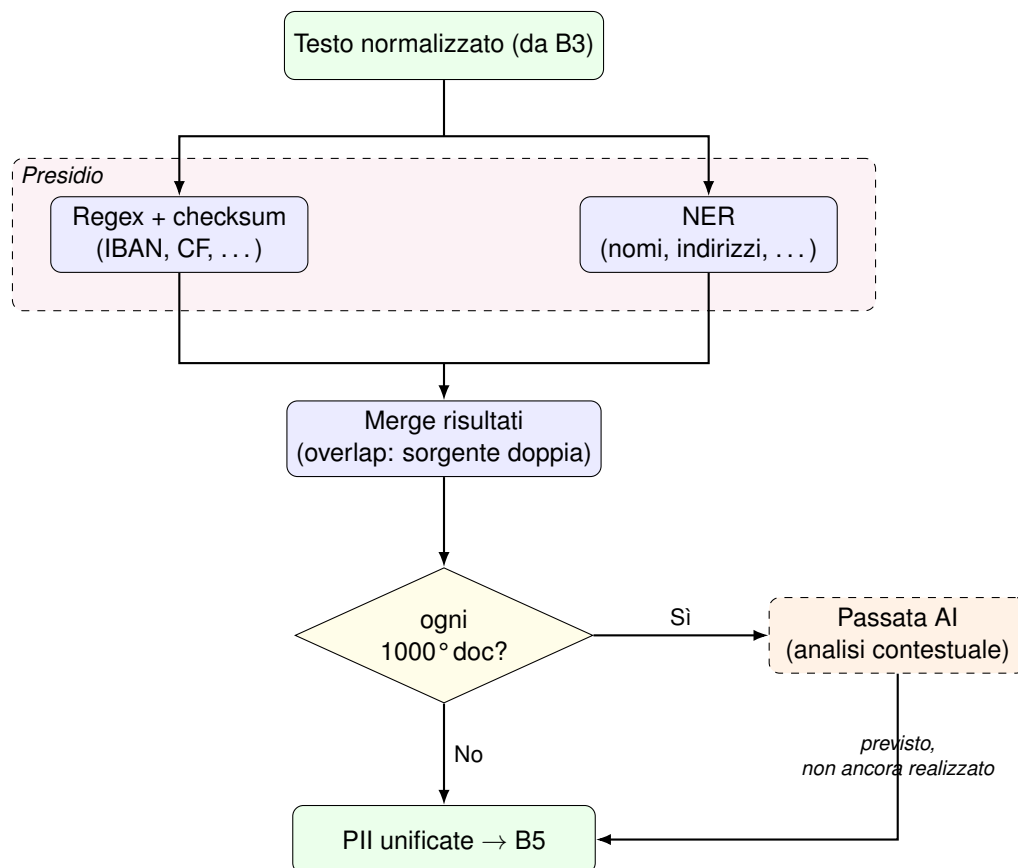


Figura 6.2: Dettaglio del blocco B4: pipeline ibrida di rilevamento PII. I detector regex e NER sono forniti da Presidio (AnalyzerEngine) dietro il contratto PIIDetector; il merge resta proprietario. Il ramo tratteggiato in basso a destra — il campionamento e la passata AI contestuale — è *previsto ma non ancora realizzato*: il punto di innesto esiste già nel merge, che accetta candidati di provenienza generativa e li classifica come tali, mentre il rilevatore che li produrrebbe non è al momento cablato nella pipeline (§6.4.3).

6.1 Ingestione e modello dati del ROPA

Il blocco B1 della Figura 6.1 acquisisce il ROPA (*Record of Processing Activities*, il registro delle attività di trattamento) e lo trasforma in una rappresentazione interna. Non è una fase accessoria: il ROPA è il termine di paragone *atteso* rispetto al quale la verifica di conformità (B7) confronta le PII effettivamente rilevate nei documenti. La sua struttura dati condiziona quindi ciò che il sistema è in grado di verificare. Questa sezione motiva l'approccio scelto per modellarlo.

6.1.1 Strutturazione a monte o delega all'AI

L'ingestione del ROPA ammette due strategie agli estremi opposti. Nella prima la **struttura dati è definita a monte**: il sistema impone uno schema noto e l'ingestione lo popola in modo deterministico a partire da un ROPA già compilato. Nella seconda, **totalmente agentica**, i documenti del ROPA vengono forniti così come sono a un modello generativo, delegando a esso l'individuazione della struttura — l'approccio «si immettono i documenti e si osserva cosa emerge».

Si adotta la prima. Un registro dei trattamenti è un *artefatto di conformità*: la sua affidabilità e la sua auditabilità non possono dipendere da una strutturazione inferita, potenzialmente allucinata e non riproducibile. Uno schema esplicito garantisce inoltre che il modello sia interrogabile in modo prevedibile dal blocco di confronto (B7) e mantenibile nel tempo. L'AI non viene esclusa, ma **circoscritta** a un ruolo di arricchimento su un singolo campo (§6.1.4); ciò è coerente con l'uso selettivo dell'AI già previsto per il blocco B1 nella Figura 6.1.

6.1.2 Il registro-tipo CNIL come schema di riferimento

Il contenuto obbligatorio del registro è fissato dall'art. 30 del Regolamento (UE) 2016/679 [**gdprArt30**] e, per il contesto svizzero, dall'art. 12 della LPD [**nlpdArt12**]; i due elenchi sono ampiamente sovrapponibili. In assenza di una pluralità di modelli operativi consolidati, il riferimento pratico più diffuso è il registro-tipo pubblicato dalla *Commission Nationale de l'Informatique et des Libertés* (CNIL), l'autorità di controllo francese, derivato direttamente dall'art. 30 [**cnilRegistreTraitement**, **cnilRegistreModeleBasique**]. Adottarlo come base evita di progettare uno schema dal principio e lega il modello ad una fonte ampiamente autorevole e condivisa.

Il modello CNIL è organizzato per *attività di trattamento*: una scheda per ciascun trattamento, con i campi seguenti.

- identificazione del trattamento (nome, date di creazione e aggiornamento, base giuridica);
- attori: titolare del trattamento, eventuali co-titolari, responsabile della protezione dei dati (DPO), responsabili del trattamento esterni (*processors*);

- finalità del trattamento;
- categorie di persone interessate;
- categorie di dati personali, con evidenziazione delle categorie particolari (dati sensibili);
- categorie di destinatari (interni, esterni, responsabili esterni);
- trasferimenti verso Paesi terzi (destinazione e garanzie);
- durate di conservazione;
- misure di sicurezza tecniche e organizzative.

6.1.3 Normalizzazione in un modello relazionale

La scheda CNIL è *denormalizzata* — un foglio piatto per trattamento — e ricca di campi che il sistema non ha motivo di modellare. Poiché la verifica di conformità (B7) confronta le categorie di dati *dichiarate* con quelle *rilevate*, il modello interno conserva soltanto ciò che alimenta quel confronto — l'identità del trattamento e le sue categorie di dati personali — e scarta deliberatamente gli altri campi della scheda (attori, categorie di interessati, destinatari, trasferimenti verso Paesi terzi, misure di sicurezza). È una scelta **sottrattiva**: riduce la superficie da mantenere e da tenere allineata alla fonte, senza togliere nulla a ciò che il sistema deve poter verificare.

Ciò che resta viene normalizzato in una gerarchia a **tre livelli**, che ricalca l'organizzazione del registro CNIL:

- `ProcessingActivity` (livello 1) — l'identità del trattamento: un identificatore stabile, il nome e la finalità.
- `DeclaredMacroCategory` (livello 2) — un blocco di categorie di dato del registro (ad esempio «stato civile, identità»), con la relativa **durata di conservazione**. La conservazione vive a questo livello, non sul singolo dato: le categorie raccolte nel blocco la ereditano. Accanto al testo originale si conserva una durata in mesi *calcolabile a macchina* (`retention_months`), così la verifica «conservazione superata?» diventa un confronto e non una lettura di prosa.
- `DeclaredCategory` (livello 3) — la singola categoria dichiarata, il punto di aggancio al motore di rilevamento.

Un solo collegamento fa del ROPA il riferimento operativo del sistema, e non un semplice archivio. `DeclaredCategory` è messa in corrispondenza (relazione 1–N, tramite l'attributo `pii_types`) con il **catalogo delle categorie di PII** usato dal motore di rilevamento (blocco B4). Il ROPA dichiara così quali categorie un trattamento *dovrebbe* contenere; il blocco B4 rileva quelle *effettivamente* presenti nei documenti. Il confronto tra i due insiemi è ciò che

alimenta la verifica di conformità (B7): da un lato le PII rilevate ma non dichiarate (*orfane*, cfr. §6.5.4), dall'altro le categorie dichiarate ma mai riscontrate. Una categoria che non si risolve in alcun `pii_type` (insieme vuoto) resta **esplicita** nel modello: è dichiarata ma non rilevabile dal motore, e come tale va segnalata al DPO invece di essere scambiata per conforme.

6.1.4 Ruolo circoscritto dell'AI: normalizzazione, non strutturazione

I livelli 1 e 2 vengono popolati in modo **deterministico**: poiché un ROPA compilato secondo il modello CNIL ha una struttura nota, leggere l'identità del trattamento, i blocchi di categorie e le durate di conservazione è un'operazione di parsing, senza AI. L'unico punto che richiede interpretazione è il passaggio al **livello 3**, cioè il contenuto dei blocchi di categorie, che nei registri reali è spesso redatto in linguaggio naturale e in modo sbrigativo (ad esempio «dati anagrafici, coordinate bancarie, contatti»). Qui l'AI interviene **una tantum e a livello di singolo campo**, per *scindere* ed espandere quel testo libero nelle singole categorie e risolverle sulle voci canoniche del catalogo di PII (ad esempio «coordinate bancarie» → `iban`; «dati anagrafici» → `person_name`, `date_of_birth`, `italian_id`). L'AI opera **dentro un vocabolario chiuso e verificato**: ogni `pii_type` proposto è validato contro il catalogo e gli identificatori sconosciuti vengono scartati, così il modello non può essere inquinato da un'allucinazione. La mappatura è marcata come *proposta* (`MappingState.PROPOSED`) e sottoposta a **conferma del DPO** (*human-in-the-loop*) prima di essere fissata (`CONFIRMED`). Questo confina l'AI a un compito circoscritto, verificabile e reversibile, coerente con la strategia di uso selettivo dell'AI adottata altrove nel sistema, senza cedere alla strutturazione agentica dell'intero documento.

Va precisato che l'AI, in questo passaggio, è un'**alternativa** e non un obbligo. La risoluzione delle categorie è realizzata come strategia intercambiabile dietro un contratto unico, con almeno due realizzazioni: una **deterministica**, basata su dizionario e riconoscimento di parole chiave, e una **generativa**, affidata al modello linguistico locale. La configurazione predefinita adotta la prima: il confronto sperimentale (Cap. 5) ha mostrato che il dizionario è una *baseline* forte — precisa e a costo nullo — mentre il modello generalizza meglio sulle diciture impreviste ma introduce falsi positivi e latenza. Che il livello 3 venga popolato con o senza AI è quindi una scelta di configurazione, non un vincolo architetturale; in entrambi i casi l'esito resta una *proposta* soggetta alla conferma del DPO.

6.1.5 Punti aperti di progettazione

Sul modello del ROPA, a differenza di quello della persistenza, alcune decisioni restano tuttora aperte e si riportano qui come lavoro in corso.

6.1.5.0.1 Allineamento nLPD–GDPR. Il modello parte dai campi CNIL (impianto GDPR); vanno annotate le differenze rilevanti dell'art. 12 LPD, in particolare le soglie e le eccezioni

previste per le piccole e medie imprese, che incidono su quali trattamenti debbano comparire nel registro.

6.1.5.0.2 ROPA non strutturato. Il modello assume un registro già compilato in forma tabellare. Il caso di un ROPA fornito come testo libero (non nel formato CNIL) richiederebbe un passo di strutturazione più invasivo: va deciso se supportarlo e in che misura, restando entro il vincolo di non delegare all'AI la strutturazione complessiva.

6.2 Un blocco previsto e non realizzato: l'analisi della struttura (B2)

Prima di descrivere i blocchi realizzati conviene dichiarare quello che non lo è. Il blocco B2 della Figura 6.1 prevedeva un'analisi *una tantum* della struttura del file system — riconoscere l'organizzazione delle cartelle, individuare le aree di interesse, eventualmente inferire il ruolo di ciascun ramo dell'albero — da eseguire prima dell'estrazione e del rilevamento.

Quel blocco non è stato realizzato, e il percorso effettivo dei documenti va dal file system direttamente all'estrazione (B3). La sua funzione minima — sapere *quali* file esistono e quali siano analizzabili — è oggi assolta dall'enumerazione ricorsiva che precede ogni scansione, la quale produce anche l'anteprima mostrata all'operatore prima dell'avvio. Ciò che manca è la parte *interpretativa*: nessuna inferenza sul significato della struttura di cartelle. Va osservato che una funzione analoga è ricomparsa altrove, ma per via esplicita anziché inferita: le regole che associano un prefisso di percorso a un trattamento (§6.6.1) attribuiscono un significato alla collocazione di un documento, con la differenza sostanziale che a stabilirlo è il DPO e non un'euristica. Se B2 venisse in futuro realizzato, il suo esito naturale sarebbe proprio *proporre* quelle regole anziché sostituirsi al giudizio umano.

6.3 Livello di estrazione del testo

Il blocco B3 trasforma un documento nel suo formato originale (PDF, Word, fogli di calcolo, testo semplice) nella sequenza di testo normalizzato su cui opera il rilevamento. È il confine che disaccoppia i due mondi: a monte i formati eterogenei dei documenti aziendali, a valle un unico contratto testuale — il *documento normalizzato*, costituito dall'identificatore del documento e dal suo testo — che B4 riceve senza dover conoscere il formato di provenienza (cfr. l'ingresso «Testo normalizzato» nella Figura 6.2).

6.3.0.0.1 Uno scope a stadi. L'estrazione affronta per prima la classe di documenti *born-digital*, quelli in cui il testo è già rappresentato come tale: qui la lettura è diretta e priva di rumore. Rientrano in questa classe i PDF nativi, i file Word — nel formato corrente e in quello *legacy*, quest'ultimo tramite un convertitore di sistema — il testo semplice e, con

pari dignità, i **fogli di calcolo** (.xlsx, .xslm, .ods), le cui celle vengono appiattite in testo. Questi ultimi non sono un'aggiunta opportunistica: elenchi di clienti, dipendenti e fornitori sono fra i contenitori di dati personali più densi di una condivisione aziendale, e ignorarli renderebbe la copertura del sistema irrealistica. L'insieme dei formati leggibili è dichiarato in un solo punto del codice, così che ogni componente a valle — l'anteprima di scansione, il conteggio dei file ignorati, la scansione batch — lo erediti senza duplicarne l'elenco. Restano fuori, come estensione pianificata e non come limite dell'impianto, due casi più onerosi: i documenti *scansionati*, che richiedono un passo di riconoscimento ottico dei caratteri (OCR), e i *layout* complessi (tabelle, testo multicolonna), che richiedono una ricostruzione della struttura di pagina. Entrambi si innestano dietro lo stesso confine, sostituendo o arricchendo l'estrattore senza toccare il livello di rilevamento.

6.3.0.0.2 Estrazione e rilevamento come *concern separati*. Tenere l'estrazione dietro un confine netto ha una conseguenza metodologica: la qualità dell'estrazione e quella del rilevamento si possono misurare *separatamente*, attribuendo un eventuale errore all'uno o all'altro stadio. È una proprietà di cui la validazione del rilevamento (Cap. 8) si avvale per isolare il costo introdotto dal passaggio attraverso l'estrazione.

6.4 Livello di rilevamento ibrido delle PII

Il blocco B4 della Figura 6.1 è il motore di rilevamento vero e proprio: dato il testo già estratto e normalizzato dal blocco a monte, individua le PII presenti nel documento e le consegna, unificate, al registro (B5). È un livello *ibrido*, che combina due tecniche indipendenti e complementari; la sua architettura interna è dettagliata nella Figura 6.2.

6.4.1 Due sorgenti complementari dietro un contratto unico

Le due tecniche coprono classi di PII diverse. Le **espressioni regolari**, affiancate dalla verifica di *checksum* dove il formato la prevede, riconoscono gli *identificatori strutturati* — IBAN, codice fiscale, numero di carta, indirizzo IP — dove la forma è rigida e la precisione può essere massima. Il **riconoscimento di entità nominate** (NER) copre invece i dati meno strutturati e dipendenti dal contesto — nomi di persona, indirizzi — dove nessuna regola fissa è sufficiente. La validazione a *checksum* non è un affinamento previsto ma una proprietà già operante: un IBAN o un codice fiscale formalmente ben scritto ma incoerente nella cifra di controllo viene scartato, ed è la ragione per cui il corpus di valutazione (Cap. 8) deve generare valori *aritmeticamente validi* — altrimenti sarebbe il *gold* stesso a essere respinto, gonfiando artificiosamente i falsi negativi.

Entrambe le tecniche sono esposte al resto del sistema attraverso un unico contratto, `PIIDetector` (nella forma `detect(testo) → lista di candidati`): l'orchestrazione dipen-

de solo da questa forma, non dalla tecnologia sottostante, così una sorgente può essere sostituita senza toccare il resto del livello (cfr. §5.3).

6.4.1.0.1 Estendere il rilevamento senza scrivere codice. Lo stesso contratto rende possibile *comporre* più rilevatori in un unico posto della pipeline. Su questa possibilità poggia il punto di estensione richiesto dai requisiti (cfr. §2.3.4): accanto ai riconoscitori predefiniti opera un rilevatore **guidato da configurazione**, che esegue le regole dichiarate in un file esterno. Aggiungere un identificatore proprio di un settore o di un ordinamento — e la categoria corrispondente nel catalogo — è quindi una modifica *dichiarativa*, non un intervento sul codice sorgente. La coerenza di questa scelta è stata verificata applicandola a sé stessa: l'unico identificatore che era stato cablato nel motore come caso speciale (il numero AVS svizzero) è stato ricondotto al medesimo meccanismo, così che esista una sola strada per tutti i pattern anziché due.

6.4.2 Il merge: unificazione e grado di certezza

Le due sorgenti producono candidati in modo autonomo; il *merge* li unifica in un unico insieme di PII, decidendone il grado di certezza in base a *quante* e *quali* sorgenti concordano su una stessa porzione di testo. Confrontando gli span per sovrapposizione, ogni PII risultante ricade in uno di quattro casi — un vocabolario chiuso:

- **Fonte singola** — rilevata da una sola tecnica, senza sovrapposizioni. Non viene mai scartata (cfr. il principio *recall-first* più sotto), ma etichettata come meno certa.
- **Doppia conferma** — pattern e NER concordano sullo stesso tipo di PII su span sovrapposti. È il caso più solido: la confidenza viene rinforzata.
- **Conflitto** — gli span si sovrappongono ma le due sorgenti assegnano tipi diversi. Il merge **non arbitra**: conserva entrambe le ipotesi e demanda la risoluzione al blocco a valle, che dispone di più contesto.
- **Scoperta dall'AI** — individuata dalla passata generativa selettiva (campionata), assente dalle altre sorgenti. È l'unico dei quattro casi che il sistema, allo stato attuale, non può ancora produrre (§6.4.3).

6.4.2.0.1 Principio recall-first. Nessuna sorgente scarta di propria iniziativa un candidato debole: al più lo segnala con confidenza bassa. La ragione è la finalità di conformità del sistema: un falso positivo costa al DPO una verifica in più, mentre una PII *mancata* è un rischio che resta invisibile. Il merge eredita questo principio — non elimina, ma unifica e classifica — lasciando ogni eventuale scarto a una decisione umana o a un blocco a valle.

6.4.2.0.2 Un solo modello dato. La PII unificata prodotta dal merge non richiede un modello dati intermedio: la sua *certezza* è già descritta da tre attributi che essa porta con sé — la confidenza (quanto è forte il rilevamento), il livello di conferma appena descritto, e l'elenco delle *provenienze* (quali e quanti detector concordano, ciascuno con la propria confidenza). Mantenere questi campi sul singolo oggetto, invece di duplicarli in strutture parallele, conserva un'unica fonte di verità per la PII rilevata e ne preserva la tracciabilità verso il DPO.

6.4.3 Il ramo generativo: un innesto predisposto

La passata AI rappresentata nella Figura 6.2 merita una precisazione di stato, perché è l'unica parte del blocco che descrive un'intenzione anziché un comportamento. L'idea è impiegare un modello generativo come *secondo parere* su un campione dei documenti — non su tutti, per la ragione di costo discussa in 5.1 — così da misurare quanto l'AI aggiunga rispetto alle due sorgenti deterministiche e su quali categorie.

Ciò che è realizzato è l'**innesto**: il merge accetta una terza sorgente di candidati, li tratta come le altre nel confronto per sovrapposizione e conserva la loro provenienza generativa quando coprono una porzione di testo che nessun'altra sorgente ha toccato. Ciò che non è realizzato è il rilevatore che quei candidati dovrebbe produrre, né la politica di campionamento che decide quando invocarlo. La distinzione è rilevante nel leggere il capitolo: il vocabolario del merge è definito per quattro casi perché il quarto è *strutturalmente* previsto, ma un'esecuzione odierna del sistema ne produce solo tre.

6.5 Persistenza del registro delle PII

Il blocco B5 della Figura 6.1 — il registro delle PII rilevate — non è un semplice contenitore dell'ultimo stato osservato, ma l'elemento su cui poggia la capacità del sistema di ragionare sull'evoluzione dei dati personali nel tempo. La scelta del modello di persistenza che lo sostiene non è quindi un dettaglio realizzativo, ma una decisione architetturale che condiziona quali domande il sistema è in grado di rispondere. Questa sezione formalizza il requisito che la persistenza deve soddisfare, confronta gli approcci valutati e motiva quello adottato.

Un vincolo di progetto attraversa tutta la discussione: **il registro non conserva i valori effettivi delle PII**, ma solo riferimenti a esse (tipo, posizione, documento di appartenenza). È una scelta di minimizzazione del dato — un registro di conformità non deve a sua volta trasformarsi in un archivio di dati personali — e ha una conseguenza diretta sul significato che possono assumere gli eventi di «modifica» (cfr. §??).

6.5.1 Il requisito di tracciamento temporale

Le linee guida di progetto richiedono che il sistema mantenga una traccia tale per cui, se un utente in seguito modifica un file, sia possibile accorgersi che una determinata PII è stata cancellata o modificata rispetto a una rilevazione precedente. Il registro deve cioè poter rispondere non solo alla domanda «quali PII sono presenti ora?», ma anche «*che cosa è cambiato* rispetto alla scansione precedente?».

Questo è un requisito di **tracciamento temporale** e ha una conseguenza diretta sul modello dati: non basta uno stato corrente scollegato dal passato, serve un meccanismo che leghi esplicitamente le rilevazioni successive dello stesso documento. È il criterio rispetto al quale vengono valutate le opzioni che seguono.

6.5.2 Approcci valutati

Il problema non si riduce a una scelta binaria tra snapshot ed event logger: esistono tre approcci distinti, con profili di complessità e garanzie molto diversi, riassunti nella Tabella 6.1.

Tabella 6.1: Approcci valutati per la persistenza del registro delle PII.

#	Approccio	Idea di fondo	Fonte di verità
A	Snapshot completi	Ogni scansione salva l'intero stato rilevato, indipendente dalle altre	Ultimo snapshot
B	Event sourcing puro	Si salvano solo gli eventi; lo stato corrente è <i>derivato</i> col replay	Il log degli eventi
C	Changelog ibrido (delta-based)	Stato corrente mutabile e log append-only secondario delle variazioni	Lo stato corrente (log complementare)

6.5.2.1 Opzione A — Snapshot completi

A ogni esecuzione della pipeline si salva un'istantanea completa di tutte le PII rilevate (`pii_type`, `document_path`, `position`, `associated_processing`, `scan_date`) autoconsistente e indipendente dalle altre. Il modello dati è minimale (una sola entità, nessuna logica di correlazione), non esiste rischio di derivare uno stato errato perché ogni snapshot è lo stato, e la realizzazione è immediata.

Il limite è però esattamente sul requisito del §6.5.1: non esiste **alcuna relazione esplicita tra scansioni consecutive**. Con due snapshot presi a distanza di ore, ricostruire «che cosa è cambiato» richiede un confronto (diff) calcolato a posteriori, riga per riga, senza alcuna struttura che leghi le due fotografie. A ciò si aggiunge la ridondanza dei dati (le PII invariate vengono riscritte a ogni scansione) e il fatto che il tracciamento temporale non è una proprietà del modello ma un'operazione applicativa da implementare comunque. L'opzione non è scartata perché errata, ma perché sposta interamente sul livello applicativo il lavoro di correlazione che il requisito richiede, senza offrire una struttura dati che lo supporti.

6.5.2.2 Opzione B — Event sourcing puro

Lo stato dell'applicazione non viene mai salvato direttamente: si registra ogni cambiamento come evento immutabile in un log append-only, e lo stato corrente si ottiene *rigiocando*

(replay) l'intera sequenza di eventi. Nella formulazione originale di Fowler, il log diventa la fonte di verità principale e lo stato è puramente derivato [fowlerEventSourcing2005]. Il pattern offre per costruzione una storia completa e immutabile — si può ricostruire lo stato a qualsiasi istante passato — con auditabilità nativa, proprietà interessante per un sistema di conformità; qui il tracciamento temporale non è un'aggiunta, ma il modello stesso.

Su questo approccio la letteratura è però netta. Lo studio empirico di Overeem et al., basato su interviste con 25 ingegneri che hanno adottato il pattern in sistemi reali, identifica cinque sfide ricorrenti, tra cui una curva di apprendimento ripida e l'evoluzione dello schema (*schema evolution*): non difficoltà percepite, ma *liabilities* documentate del pattern [71]. Inoltre, poiché lo stato corrente non è memorizzato ma va ricostruito, anche l'interrogazione più semplice («quali PII ci sono ora in questo documento?») richiede di rigiocare gli eventi; per rendere efficienti le letture, l'event sourcing viene quasi sempre affiancato al pattern CQRS (*Command Query Responsibility Segregation*), che introduce un *secondo* modello di lettura mantenuto in parallelo, cioè ulteriore complessità architetturale [microsoftAzureEventSourcing, awsEventSourcingPattern, 72]. A questo si sommano la crescita illimitata del log e la gestione di proiezioni, versioning degli eventi e consistenza eventuale. La stessa documentazione di riferimento raccomanda di adottare il pattern **solo** quando i benefici di auditabilità e ricostruzione storica giustificano la complessità introdotta, notando che per la maggior parte dei sistemi la gestione dati tradizionale è sufficiente [microsoftAzureEventSourcing].

6.5.2.3 Opzione C — Changelog ibrido delta-based

L'approccio adottato mantiene **due** strutture complementari:

1. uno **stato corrente mutabile** — la lista delle PII attualmente presenti, interrogabile direttamente senza alcuna ricostruzione;
2. un **log append-only delle variazioni** (*changelog*), *secondario*, che registra ogni delta tra una scansione e la successiva e lega esplicitamente ogni variazione alla scansione precedente dello stesso documento.

La differenza cruciale rispetto all'opzione B è che il log **non è l'unica fonte di verità**: lo stato corrente esiste già come dato di prima classe, quindi nessun replay è necessario per le operazioni quotidiane. Il changelog serve a rispondere alla domanda «che cosa è cambiato», non a ricostruire lo stato. L'approccio combina due concetti presenti in letteratura: il **dataset delta-driven** proprio del *Change Data Capture* (CDC), che cattura le variazioni tra uno stato e il successivo *senza* fare del log la fonte di verità esclusiva, mantenendo lo stato corrente come dato normale e interrogabile [kleppmannDesigningDataIntensive2017]; e l'idea di **log immutabile append-only** come traccia di audit, mutuata dall'event sourcing ma applicata in forma non esclusiva.

Nota terminologica. Il CDC descritto da Kleppmann cattura i cambiamenti leggendo il *transaction log* di un database sorgente. Nel sistema qui proposto, invece, i cambiamenti vengono *rilevati* confrontando i risultati di scansioni successive di documenti esterni: non esiste un database sorgente di cui intercettare le transazioni. Il modello è dunque *ispirato* al paradigma delta-based del CDC, non un'implementazione di CDC in senso stretto.

6.5.3 Motivazione della scelta

La scelta dell'opzione C è il risultato di un confronto esplicito di *trade-off*, sintetizzato nella Tabella 6.2, e si articola in quattro punti:

1. **Risolve il difetto di A.** Il changelog lega ogni variazione alla scansione precedente dello stesso documento: le scansioni non sono più fotografie isolate e la domanda «che cosa è cambiato» ha una risposta strutturale, non un diff calcolato a posteriori.
2. **Evita il costo di B.** Lo stato corrente è memorizzato e interrogabile direttamente: niente replay, niente proiezioni, niente CQRS, niente consistenza eventuale da gestire. Le sfide di adozione documentate da Overeem et al. [71] non si applicano, perché non si sta adottando event sourcing puro.
3. **Conserva l'auditabilità dove serve.** Se in futuro servisse la ricostruzione storica completa, resta possibile rigiocando il changelog — ma è una *possibilità*, non un requisito per il funzionamento quotidiano: si ottiene gran parte del beneficio di B senza pagarne il prezzo in anticipo.
4. **È coerente con la minimizzazione del dato.** Registrando solo riferimenti e non valori, sia lo stato corrente sia il changelog restano leggeri e privi di PII effettive.

Tabella 6.2: Sintesi comparativa dei tre approcci di persistenza.

Criterio	A — Snapshot	B — Event sourcing	C — Changelog ibrido
Relazione esplicita tra scansioni	✗	✓	✓
Interrogazione stato corrente	immediata	replay / CQRS	immediata
Complessità realizzativa	bassa	alta	medio-bassa
Auditabilità / storia completa	✗	✓	parziale (estendibile)
Ridondanza dei dati	alta	nessuna	bassa
Coerenza con minimizzazione del dato	✓	✓	✓
Rischio di stato derivato errato	nullo	presente	nullo
Sfide di adozione documentate	—	5 [71]	non applicabili

6.5.4 Modello dati concettuale

Il modello che segue è *technology-agnostic*: la scelta della tecnologia di persistenza concreta è discussa nel Capitolo 7. Oltre alle entità già definite nel modello (Document, Scan, ProcessingActivity), la persistenza introduce due entità principali.

PIIInstance rappresenta lo stato corrente, mutabile: «questa PII, in questo documento, così com'è ora». Comprende un identificatore, il pii_type, il riferimento al Document, la posizione nel testo normalizzato come intervallo di caratteri (start, end), un riferimento opzionale (0..1) al ProcessingActivity — assente se la PII è *orfana*, cioè non riconducibile ad alcun trattamento dichiarato — e il riferimento all'ultima Scan che l'ha osservata. Porta inoltre con sé gli attributi che ne descrivono la *certezza* e la provenienza (confidence, confirmation_level, sources, §6.4.2), cosicché la tracciabilità verso il DPO sopravviva alla persistenza, e un indicatore removed che segna le istanze non più presenti senza cancellarne la riga (§??). È la tabella che il report per il DPO legge **direttamente**.

PIIChange è il log append-only, secondario, e rappresenta «che cosa è successo a un'istanza tra una scansione e l'altra». Comprende un identificatore, il riferimento all'PIIInstance, il change_type (vocabolario chiuso, cfr. §??), il riferimento alla Scan che ha prodotto la variazione, il riferimento alla previous_scan per lo stesso documento — ciò che lega le rilevazioni nel tempo — e un timestamp.

Le relazioni tra le entità sono: Document (1)–(N) PIIInstance (*contiene*); PIIInstance (1)–(N) PIIChange (*ha storia*); Scan (1)–(N) PIIChange (*genera*); PIIInstance (N)–(0..1) ProcessingActivity (*corrisponde a*, con 0 = orfana). La Figura 6.3 ne dà una sintesi grafica.

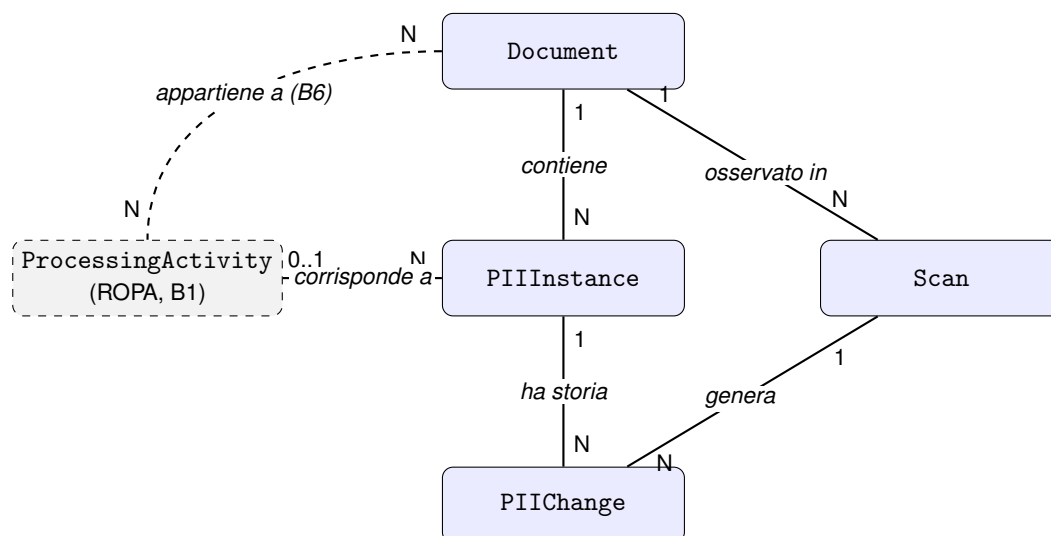


Figura 6.3: Diagramma entità-relazione del registro delle PII rilevate (blocco B5): lo stato corrente (PIIInstance) e la sua storia (PIIChange), legati alle scansioni (Scan) e ai documenti (Document). I legami con ProcessingActivity del registro ROPA (tratteggiati, blocco B1) sono due: l'associazione *documento–trattamento* (N:N, blocco B6), registrata su Document.activity_ids con la sua provenienza (association_source); e il riferimento *per singola PII* (0..1) a un trattamento giustificante, la cui assenza segnala una PII *orfana*, popolato dalla verifica di conformità (B7) a partire dall'associazione stessa. Non mostrata la tabella delle regole cartella→trattamento (registry_folder_rule, B6), che alimenta l'associazione per prefisso.

6.6 Associazione e verifica di conformità

I blocchi B6 e B7 della Figura 6.1 sono il punto in cui le due metà del sistema — il *dichiarato* (il ROPA, §6.1) e il *rilevato* (il registro delle PII, §6.5) — si incontrano e producono il **verdetto di conformità**. Fino a questo punto il sistema sa, separatamente, quali categorie un trattamento *dichiara* e quali PII un documento *contiene*; è il confronto fra i due insiemi a trasformare un rilevatore di PII in uno strumento di *governance*, ed è ciò che distingue il progetto dagli strumenti di sola rilevazione (§3.5).

6.6.1 Associazione documento–trattamento (B6)

Il confronto ha senso solo rispetto a un termine di paragone: prima di chiedersi «questo documento è conforme?», bisogna sapere *a quale trattamento* il documento appartiene. È il compito del blocco B6: associare un documento a una o più `ProcessingActivity` dichiarate.

L'associazione è **esplicita** — la stabilisce il DPO, non un'inferenza automatica — e la scelta è metodologica, non di comodo. Un'associazione *per contenuto*, che attribuisse il documento al trattamento i cui dati dichiarati somigliano di più a quelli rilevati, sarebbe **circolare**: sceglierebbe per costruzione il trattamento che rende il documento «il più conforme possibile», svuotando di senso la verifica successiva. L'associazione esplicita mantiene invece il confronto **onesto**: il trattamento è deciso da un umano, e B7 misura poi, senza sconti, ciò che quel trattamento dichiara rispetto a ciò che il documento contiene. L'associazione resta comunque dietro un contratto (`ActivityAssigner`), così un proponente automatico o assistito dall'AI potrà in futuro *proporre* l'attribuzione lasciando al DPO la conferma — lo stesso schema *human-in-the-loop* già adottato per la mappatura del ROPA (§6.1.4).

La relazione è **molti-a-molti**: un documento può rientrare in più trattamenti (un modulo usato tanto nella selezione del personale quanto nelle paghe) e un trattamento raccoglie molti documenti. L'insieme dei trattamenti di un documento è registrato sul documento stesso (`Document.activity_ids`); poiché ROPA e registro delle PII vivono su basi dati separate (§6.5), gli identificatori di trattamento fanno da collegamento *logico* fra i due mondi, senza chiave esterna fisica.

Perché l'associazione esplicita sia **praticabile su cartelle reali** — dove attribuire un documento alla volta è improponibile — il DPO dispone di due modalità, entrambe *decise da lui*: l'associazione **diretta** di un singolo documento e le **regole cartella→trattamento**, con cui mappa una volta un *prefisso di percorso* a uno o più trattamenti; ogni documento che ricade sotto quel prefisso — i file nuovi inclusi, alla scansione successiva — *eredita* l'associazione. Una regola non è un'inferenza per contenuto (che si è appena esclusa perché circolare): resta una scelta esplicita del DPO, solo espressa per *collocazione* anziché documento per documento. La **provenienza** di ogni associazione è tracciata (`association_source`, con valori `MANUAL` e `RULE`) e il manuale **prevale**: riapplicare le regole non sovrascrive mai un'asso-

ciazione fatta a mano. Le regole si applicano come passo separato *dopo* la scansione, così una scansione schedulata associa da sé i documenti nuovi — un passo verso l'autonomia «dai una cartella e fa da solo».

6.6.2 Il confronto dichiarato–rilevato e il suo vocabolario (B7)

Fissata l'associazione, la verifica di conformità è un'operazione **insiemistica**. Da un lato l'insieme dei *pii_type dichiarati*: l'unione delle categorie dichiarate da *tutti* i trattamenti a cui il documento è associato. Quali mappature concorrano a formare quell'insieme è un parametro esplicito del confronto: il verdetto *autorevole* considera le sole mappature confermate dal DPO (§6.1.4), mentre la consultazione interattiva conta anche quelle ancora proposte, così da restare informativa prima che la revisione del registro sia stata completata. La distinzione va tenuta presente nel leggere un verdetto, perché la stessa categoria può risultare orfana nell'una lettura e coperta nell'altra; ciò che invece non è ammissibile è che i due criteri convivano *dentro* un medesimo verdetto, e la regola su quali categorie continuo è perciò definita in un unico punto e usata da tutti i controlli, conservazione inclusa. Dall'altro l'insieme dei *pii_type rilevati*: i tipi delle PII effettivamente presenti nel documento secondo il registro. Confrontando i due insiemi, ogni tipo ricade in una voce di un **vocabolario chiuso**, che è il linguaggio con cui il sistema riporta il verdetto al DPO:

Coperta un tipo *rilevato* che è anche *dichiarato* da almeno uno dei trattamenti associati. È il caso conforme: il dato presente è previsto dal registro.

Orfana un tipo *rilevato* che **nessuno** dei trattamenti associati dichiara. È il segnale di conformità cardine: un dato personale presente nel documento che il ROPA non prevede — un trattamento di fatto non dichiarato (art. 30 GDPR, art. 12 nLPD). Si dice «orfana» perché priva di un trattamento-genitore che la giustifichi.

Mancante un tipo *dichiarato* da un trattamento associato ma **non** riscontrato nel documento. Non è di per sé una violazione: può indicare una sovra-dichiarazione, oppure semplicemente che quel documento non contiene quel dato.

Non risolubile una categoria *dichiarata* nel ROPA che non si risolve in alcun *pii_type* (insieme vuoto, §6.1.3): è dichiarata ma non rilevabile dal motore. Resta esplicita e va segnalata al DPO, invece di essere scambiata per conforme.

Sotto la cardinalità molti-a-molti la definizione di orfana è deliberatamente **permissiva**: un tipo è orfano solo se *nessuno* dei trattamenti associati lo dichiara (unione, non intersezione). Un dato coperto anche da un solo trattamento pertinente è giustificato. Specularmente, a ogni singola PII rilevata la verifica assegna il primo trattamento associato che ne dichiara il tipo come «trattamento giustificante» (*processing_activity_id*), oppure nessuno se la PII

è orfana: è il momento in cui il riferimento opzionale (0..1) lasciato vuoto dal registro (§6.5.4) viene finalmente popolato.

Due principi ereditati dal resto del sistema governano la lettura del verdetto. Il primo è **recall-first** (§6.4.2): il sistema preferisce segnalare un'orfana di troppo che mancarne una, perché un falso positivo costa al DPO una verifica, mentre una PII non dichiarata e non vista è un rischio invisibile. Il secondo è che il verdetto è un **segnale, non una sentenza**: un'orfana afferma «non risulta dichiarata», non «è certamente una violazione». Può infatti nascere da una violazione reale oppure da un buco di mappatura a monte — una categoria del ROPA non ancora risolta sul tipo corrispondente. È il DPO, *human-in-the-loop*, a chiudere il giudizio; il sistema gli mette davanti l'evidenza strutturata.

6.6.3 Verifica della conservazione

Alla verifica delle categorie si affianca un controllo di **conservazione** (*retention*): un dato può essere legittimamente dichiarato eppure trattenuto oltre il periodo consentito. Il ROPA porta, a livello di macro-categoria, una durata massima *calcolabile a macchina* (*retention_months*, §6.1.3); confrontandola con l'età del documento si segnala una categoria conservata oltre il limite quando un suo dato è ancora presente. La misura dell'età resta una **stima**, ma dichiarata: la data di riferimento del documento è ricavata dai metadati interni del file quando li possiede (*modDate* di un PDF, *core properties* dei formati Office) e, in loro assenza, dalla sua ultima modifica sul file system (il *mtime*). Poiché le due fonti non valgono lo stesso — il *mtime* è azzerato da qualunque copia massiva o ripristino da backup — il sistema conserva accanto alla data la sua **provenienza** (§11.1) e la espone nel verdetto, così che una segnalazione fondata su un metadato interno non venga confusa con un indizio da verificare. Sono inoltre distinti due esiti che il primo prototipo confondeva: la *violazione*, quando il documento eccede un termine dichiarato, e la *conservazione non verificabile*, quando il registro esprime un criterio anziché una durata — caso in cui il silenzio non deve essere scambiato per conformità. Il verdetto è infine consultabile sull'intero corpus, ordinato per mesi di sfioramento, e non soltanto documento per documento.

6.7 Interfaccia e restituzione al DPO

Il blocco B8 chiude la catena: rende consultabile ciò che i blocchi precedenti hanno prodotto. La scelta architettureale che lo caratterizza è che il sistema si presenta come una **applicazione web unica**, servita su una sola porta, e non come una raccolta di comandi da terminale. La ragione è di destinatario: il DPO non è necessariamente una figura tecnica, e uno strumento di conformità utilizzabile solo da chi sa invocare un comando resta, di fatto, inutilizzato. Le interfacce a riga di comando continuano a esistere e sono pienamente funzionali, ma il loro ruolo è quello di strumenti di servizio — automazione, esecuzioni schedate, diagnostica — non di via d'accesso ordinaria.

6.7.0.0.1 Una sola applicazione, due interfacce. La revisione del registro dei trattamenti (B1) e il cruscotto di conformità nascono come applicazioni distinte, con cicli di vita e destinatari in parte diversi; sono però *montate* nella stessa applicazione, che le espone sotto percorsi differenti. Il risultato è un unico servizio da avviare e configurare — coerente con il requisito di semplicità di installazione (§2.3.3) — senza che le due parti debbano fondersi in un solo componente: ciascuna conserva il proprio insieme di pagine e i propri test.

6.7.0.0.2 Consultare non deve modificare. Il cruscotto calcola il verdetto di conformità nel momento in cui una pagina viene aperta, e questo pone un vincolo preciso: la sola consultazione non deve produrre effetti sullo stato del sistema. Il verdetto mostrato in lettura è quindi calcolato in modalità *non persistente*, mentre la scrittura degli esiti per singola PII avviene soltanto in risposta a un'azione esplicita del DPO — tipicamente l'associazione di un documento a un trattamento. È una distinzione che vale la pena rendere esplicita, perché il medesimo calcolo serve due scopi diversi: informare e registrare.

6.7.0.0.3 Riferimenti, mai valori. La minimizzazione del dato (§2.3.5) non è un vincolo della sola persistenza ma attraversa anche la restituzione: le PII sono presentate al DPO come *riferimenti* — categoria, posizione nel documento, provenienza del rilevamento, stato di conformità — e in nessuna schermata compare il valore del dato personale rilevato. Un cruscotto che mostrasse i valori trasformerebbe lo strumento di verifica in una nuova copia dei dati da proteggere, vanificando la scelta compiuta a monte nel registro.

6.7.0.0.4 Due letture, per documento e per corpus. La restituzione è organizzata su due livelli complementari. La vista *per documento* risponde alla domanda «questo file è in regola?» e riunisce in un solo luogo le PII rilevate, l'associazione ai trattamenti e il verdetto. Le viste *per corpus* rispondono invece alle domande che un responsabile pone su un intero archivio — che cosa è oltre il termine di conservazione, quali documenti nessuna regola ha ancora associato — e sono ciò che rende lo strumento praticabile su volumi reali, dove l'esame documento per documento non è una procedura eseguibile.

Capitolo 7

Implementazione

7.1 Ingestione del ROPA (blocco B1)

Il blocco B1 realizza il percorso che porta un registro dei trattamenti, così come lo compila un'organizzazione, alla rappresentazione interna descritta nel modello di dominio (§6.1). La realizzazione segue il principio *sottrattivo* già motivato in architettura — del documento sorgente si conserva soltanto ciò che alimenta la verifica di conformità — e lo traduce in una catena di tre passi disaccoppiati (lettura, interpretazione, persistenza), seguiti da una passata separata di mappatura assistita. Ogni passo è un modulo a sé, così che un cambiamento di formato del file o di strategia di mappatura non tocchi gli altri.

7.1.1 Il formato sorgente: il template CNIL reale

La fonte non è un formato inventato per comodità, ma il **modello di registro ufficiale della CNIL**, l'autorità francese per la protezione dei dati: un foglio di calcolo (ODS/XLSX) con una scheda operativa (*fiche*) per ciascun trattamento, oltre a schede di servizio (tutorial, elenco dei trattamenti, un modello vuoto da replicare, liste di supporto). Ancorare l'ingestione a un artefatto reale, anziché a un file cucito sul parser, rende la funzionalità dimostrabile su un input che un'organizzazione userebbe davvero. Nella *fiche*, l'ingestione modella **una sola sezione** — «Categories of personal data», con le sue colonne *categoria*, *descrizione* e *periodo di conservazione* — e scarta deliberatamente tutte le altre (attori, finalità secondarie, categorie di interessati, destinatari, trasferimenti, misure di sicurezza), coerentemente con la scelta sottrattiva. È questa sezione, e solo questa, a fornire il dichiarato che B7 confronterà con il rilevato.

7.1.2 Lettura: un'unica forma per ODS ed Excel

Il primo passo (*sheet_reader*) legge una scheda del foglio in una griglia di stringhe, dietro un'interfaccia **indipendente dal formato** (*Adapter*): l'estensione del file decide il *back-end* — *.ods* tramite *odfpy*, *.xlsx/.xlsm* tramite *openpyxl* — ma entrambi restituiscono la stessa

struttura, così che il passo interpretativo a valle ignori del tutto la provenienza. Un dettaglio pratico del formato ODS ha richiesto attenzione: nel modello CNIL diverse celle portano un **commento** d'aiuto (il «triangolino rosso»), che nel documento risiede *dentro* la cella come annotazione. Letto ingenuamente, quel testo verrebbe concatenato davanti al valore reale (per esempio «Cf. Article 87...Social Security Number»); il lettore esclude perciò i paragrafi contenuti nelle annotazioni, restituendo il solo valore della cella. È una piccola scelta di robustezza, ma tocca proprio la sezione che si vuole conservare, e come tale è coperta da test sul file CNIL reale.

7.1.3 Interpretazione: guidata dalle etichette, non dalle posizioni

Il secondo passo (*normalizer*) trasforma la griglia in un'attività di trattamento. La regola cardine è che le informazioni si cercano per **etichetta**, non per indice di riga: il nome del trattamento è il valore accanto alla cella «Name of the processing operation», la finalità quello accanto a «Main purpose», e le categorie sono le righe che seguono l'intestazione «Categories of personal data» fino alla prima riga vuota. Così la mappatura **sopravvive** a spostamenti di righe e alla presenza delle numerose sezioni che non si modellano: il parser vi passa sopra senza inciampare. La stessa regola fa da filtro naturale sulle schede di servizio: una scheda priva del nome di trattamento — il tutorial, le liste, il modello *vuoto* — non è un'attività e viene saltata sollevando un errore che il chiamante intercetta, sicché un intero *workbook* CNIL produce esattamente un'attività per ogni *fiche* compilata. Il periodo di conservazione, infine, viene reso **calcolabile**: la dicitura originale («5 years from the payment of the salary») è conservata per audit accanto a una durata in mesi estratta da essa (60), quando il testo esprime una durata fissa; resta *None* quando esprime un criterio, coerentemente con il campo del modello (§7.3.1).

7.1.4 Persistenza e mappatura assistita

Il terzo passo affida l'attività così ottenuta al ROPARepository, che la scrive nella base dati ROPA riusando lo stesso modello di dominio come schema (*Active Record*, §7.3). La **risoluzione delle categorie sui pii_type** del catalogo condiviso — l'aggancio verso il rilevamento — è invece una passata **separata e successiva**, non parte del parsing deterministico: per ogni categoria ancora non risolta, una strategia dietro il contratto CategoryMapper propone i pii_type corrispondenti al suo testo libero. La strategia è intercambiabile — un **dizionario** deterministico (parole chiave), un **modello generativo** (§5.4) o una combinazione dei due — e la passata è **idempotente**: ciò che il mapper non sa risolvere resta invariato, così una riesecuzione mappa solo ciò che è nuovo. Il risultato è comunque una *proposta*: lo stato del mapping resta PROPOSED finché il DPO non lo conferma (*human-in-the-loop*, §6.1.4), a ribadire che l'AI arricchisce ma non decide.

L'interazione del DPO avviene tramite un'interfaccia web dedicata (blocco B8, la review montata sotto /ropa), da cui può confermare le mappature, **importare** un nuovo registro caricandone il file dal browser — il file viaggia verso il servizio come *byte* via *multipart*, coerente con il deployment containerizzato in cui l'applicazione non accede al file system del client — e **rimuovere** un trattamento. L'import riusa senza modifiche la catena di ingestione appena descritta: la stessa lettura, la stessa interpretazione, la stessa mappatura.

7.2 Tecnologia di persistenza

Il modello concettuale della persistenza — il *changelog ibrido delta-based* per il registro delle PII (§6.5) e la gerarchia del registro ROPA (§6.1) — è *technology-agnostic*: restava da fissare la tecnologia concreta con cui realizzarlo. I candidati erano due:

- una base dati **relazionale** (SQLite), che offre *join* e interrogazioni relazionali «gratuite» ed è coerente col deployment locale in container senza servizi esterni;
- una persistenza **su file semi-strutturati** (JSON/JSONL), più leggera ma priva di garanzie relazionali native.

La scelta è ricaduta su **SQLite**, per ragioni che discendono dalla natura del dato e dai vincoli di progetto:

- **Il dato è intrinsecamente relazionale.** Entrambi i registri sono grafi di entità legate da chiavi esterne (documento–scansione–istanza–variazione; attività–macro-categoria–categoria) e le interrogazioni che servono ai blocchi a valle — stato corrente di un documento, storia di un'istanza, PII orfane, confronto dichiarato-vs-rilevato — sono *join* e operazioni insiemistiche che un motore relazionale risolve nativamente. Su file JSON andrebbero reimplementate a mano, riscrivendo ciò che un DBMS offre già.
- **SQLite è *embedded*, senza server.** È un singolo file, nessun processo o servizio esterno da gestire: coerente col deployment containerizzato e, soprattutto, con la **minimizzazione del dato** — i dati non lasciano l'infrastruttura on-premise e nessun servizio di terzi è coinvolto.
- **Una sola definizione (*Active Record*).** Con SQLAlchemy l'entità di dominio è la tabella (§7.3): niente coppia dominio/riga né traduzione a mano, coerente col principio DRY.
- **La porta resta aperta.** SQLAlchemy poggia su SQLAlchemy: migrare a un RDBMS in rete (ad esempio PostgreSQL), qualora si superi il singolo nodo, cambia l'URL di connessione — letto da variabile d'ambiente (§7.5) — e non il modello dei dati. SQLite è dunque la scelta adeguata *ora*, non un vicolo cieco.

La stessa tecnologia serve entrambi i registri, su due basi dati separate (variabili d'ambiente ROPA_DB_URL e PII_DB_URL); *che cosa* vi si conserva, campo per campo, è dettagliato nel §7.3.

7.3 Dati persistiti nei database

Il sistema mantiene due basi dati distinte e separate (§7.5): il **registro ROPA** (blocco B1) e il **registro delle PII rilevate** (blocco B5). Ogni entità è definita una sola volta come tabella SQLModel — il modello di dominio è lo schema di persistenza (*Active Record*) — così da non duplicare la struttura. I modelli concettuali e le relazioni sono discussi in §6.1.3 e §6.5.4; questa sezione documenta, **campo per campo, ciò che viene effettivamente scritto su disco e perché**. La realizzazione corrente è su SQLite (§7.2), ma il modello dei dati qui descritto è indipendente dalla tecnologia sottostante.

Un principio attraversa entrambe le basi ed è la chiave di lettura di ogni campo che segue: la **minimizzazione del dato** (§2.3.11). In particolare il registro delle PII conserva soltanto *riferimenti* — tipo, posizione, provenienza, documento — e **mai il valore** della PII, come si vedrà nell'assenza deliberata di una colonna che lo contenga.

7.3.1 Registro ROPA (blocco B1)

Tre tabelle rispecchiano la gerarchia a tre livelli del registro CNIL (§6.1.3).

7.3.1.0.1 processing_activity — l'attività di trattamento (livello 1).

id identificatore stabile dell'attività (chiave primaria), assegnato dal chiamante; ancora dell'intera sotto-struttura.

name nome del trattamento, per leggibilità e report al DPO.

purpose finalità dichiarata del trattamento, elemento cardine dell'art. 30 GDPR.

7.3.1.0.2 macro_category — il blocco di categorie con la sua conservazione (livello 2).

id chiave surrogata (auto-generata).

activity_id chiave esterna all'attività: colloca la macro-categoria nella gerarchia.

raw_text dicitura originale della macro-categoria, conservata *tale e quale* per auditabilità (nulla del documento sorgente va perduto).

retention_text dicitura originale del periodo di conservazione, conservata per audit.

`retention_months` la stessa conservazione resa **calcolabile dalla macchina** (durata in mesi), affinché il controllo di retention (B7) sia un confronto e non una lettura di prosa; vale None quando il registro esprime un criterio anziché una durata fissa.

7.3.1.0.3 `declared_category` — la categoria dichiarata, ponte verso il rilevamento (livello 3).

`id` chiave surrogata.

`macro_category_id` chiave esterna alla macro-categoria di appartenenza.

`raw_text` testo libero originale della categoria (es. «nome»), conservato per audit.

`pii_types` l'insieme di `pii_type` del catalogo condiviso a cui la categoria è ricondotta: è l'**aggancio** che rende il confronto fra ciò che il ROPA *dichiara* e ciò che il motore *rileva* (B7) una semplice operazione insiemistica.

`mapping_state` stato del mapping (PROPOSED dall'AI / CONFIRMED dal DPO): materializza il *human-in-the-loop*: l'AI propone la corrispondenza, ma non la fissa mai da sola.

7.3.2 Registro delle PII rilevate (blocco B5)

Quattro tabelle realizzano il *changelog* delta-based (§6.5.4): stato corrente più storia. Una quinta tabella, `registry_folder_rule`, risiede nella stessa base dati ma appartiene al blocco di associazione (B6, §6.6.1) ed è descritta in coda a questa sezione. È nel *changelog* che la minimizzazione si vede nel dettaglio.

7.3.2.0.1 `registry_document` — il documento analizzato.

`document_id` identificatore stabile del documento (chiave primaria), coerente con il `document_id` prodotto da B3; nella scansione ricorsiva di una cartella è il **percorso relativo** (PO-SIX) alla radice, ciò che consente alle regole cartella→trattamento di combaciare per prefisso (§6.6.1).

`path` percorso originale del file, come riferimento; opzionale.

`first_seen` istante di prima registrazione del documento.

`activity_ids` gli identificatori dei trattamenti dichiarati a cui il documento è associato (B6): è una **lista** (cardinalità N:N, §6.6.1) e i suoi valori riferiscono `ProcessingActivity` nell'altra base dati, come collegamento *logico* senza chiave esterna fisica.

`association_source` la **provenienza** dell'associazione — MANUAL (impostata dal DPO) oppure RULE (derivata da una regola cartella→trattamento) — oppure None se il documento non è ancora associato. È il campo che fa **vincere il manuale sulle regole**: riapplicare le regole salta i documenti a provenienza MANUAL.

`reference_date` la data cui il documento si assume risalga, letta dai metadati interni del file quando li possiede e dal file system altrimenti, usata come età del contenuto per il controllo *approssimato* di conservazione (B7, §6.6.3); è un *riferimento*, non un valore di PII. Vale None se ignota.

`reference_date_source` la provenienza della data precedente (`content_metadata` o `file_mtime`), perché il verdetto possa dichiarare quanto vale la stima che espone.

`source_mtime`, `source_size` l'impronta *tecnica* del file osservata all'ultima analisi — da non confondere con la data semantica: serve unicamente a stabilire se il file sia cambiato e vada quindi rianalizzato (§11.2).

`last_scanned_at` il momento dell'ultima analisi effettiva del documento; una scansione in cui esso venga omesso perché invariato non lo aggiorna.

`detector_signature` l'impronta della configurazione di rilevamento con cui le istanze correnti sono state prodotte: se cambia, il documento va rianalizzato anche a file immutato.

Il vocabolario di `association_source` è un'enumerazione chiusa (`AssociationSource: MANUAL/RULE`), come per gli altri campi a dominio fissato del modello.

7.3.2.0.2 `registry_scan` — una scansione, cioè un'osservazione nel tempo.

`id` chiave surrogata.

`document_id` chiave esterna al documento osservato.

`created_at` istante della scansione: è l'**asse temporale** su cui poggia l'intero changelog.

7.3.2.0.3 `registry_pii_instance` — la singola PII, stato corrente e mutabile. Nessuna colonna di questa tabella contiene il *valore* della PII: è il punto in cui la minimizzazione diventa concreta.

`id` chiave surrogata.

`document_id` chiave esterna al documento che la contiene.

`pii_type` la categoria del catalogo condiviso (es. `iban`) — *non* il valore.

`start`, `end` la **posizione** come intervallo di caratteri nel testo normalizzato: un riferimento a *dove* si trova la PII, non al suo contenuto.

`confidence` confidenza aggregata dal merge, per consentire al DPO ordinamento e soglie.

`confirmation_level` esito del merge (`single_source` / `double_confirmed` / `conflicting` / `ai_discovered`): traccia *quanto* è certa la rilevazione.

`sources` gli identificatori dei detector che l'hanno trovata: traccia *chi* l'ha rilevata (tracciabilità, §2.7.3).

`processing_activity_id` l'attività dichiarata a cui la PII corrisponde, oppure `None` se la PII è **orfana** (non riconducibile ad alcun trattamento); popolata dal blocco di associazione (B6), senza chiave esterna fisica finché i due database restano distinti.

`last_scan_id` chiave esterna all'ultima scansione che l'ha osservata.

`removed` flag di stato: `True` se l'istanza non è più presente, marcata dalla logica di delta; la riga resta, così la storia non si perde.

L'**assenza** di un campo `value/text` è una scelta di progetto, non una dimenticanza: garantisce che un registro di conformità non si trasformi a sua volta in un archivio di dati personali.

7.3.2.0.4 `registry_pii_change` — il log **append-only** delle variazioni.

`id` chiave surrogata.

`pii_instance_id` chiave esterna all'istanza cui la variazione si riferisce.

`change_type` il tipo di variazione, vocabolario chiuso: `NEW` / `CONFIRMED` / `MOVED` / `REMOVED` (§??).

`scan_id` chiave esterna alla scansione che ha prodotto la variazione.

`previous_scan_id` chiave esterna alla scansione *precedente* dello stesso documento — ciò che lega le rilevazioni nel tempo; vale `None` alla prima scansione (*bootstrap*).

`timestamp` istante in cui la variazione è stata registrata.

7.3.2.0.5 `registry_folder_rule` — la regola cartella→trattamento (blocco B6). Fisicamente nella stessa base dati del registro delle PII, ma logicamente parte dell'associazione (§6.6.1): mappa un prefisso di percorso ai trattamenti che i documenti sotto di esso ereditano.

`prefix` il prefisso di percorso (POSIX, normalizzato senza slash finale; `""` è la radice e combacia con ogni documento). Chiave primaria.

`activity_ids` i trattamenti con cui associare i documenti che cadono sotto `prefix`; riferiscono `ProcessingActivity` nell'altra base dati (nessuna chiave esterna fisica), come `Document.activity_ids`.

`created_at` istante di creazione della regola.

In sintesi: il registro ROPA conserva il dichiarato (con il testo originale accanto alla forma strutturata, per audit), mentre il registro delle PII conserva soltanto *riferimenti* a ciò che è stato rilevato e la loro storia — mai i valori. Le due basi restano separate; i campi `Document.activity_ids` (associazione N:N, B6) e `PIIInstance.processing_activity_id` (trattamento giustificante per singola PII, popolato da B7) sono i punti in cui i due mondi si collegano *logicamente*, sull'identificatore-stringa del trattamento.

7.4 Realizzazione del livello di rilevamento

Le due scelte tecnologiche che governano questo livello — costruire il rilevamento a pattern e NER sopra Microsoft Presidio, e quale modello generativo adottare per gli impieghi selettivi — non sono state compiute qui: sono l'esito degli studi condotti prima della progettazione (§5.3, §5.4), la cui verifica sperimentale è nel Capitolo 8. Questa sezione ne descrive la realizzazione, e in particolare la proprietà che ne governa la manutenzione nel tempo.

7.4.1 Estensibilità config-driven: aggiungere categorie e pattern

Una proprietà di progetto del livello di rilevamento è che il suo vocabolario non è cablato nel codice ma **dichiarato in configurazione**. Il catalogo delle categorie di PII (`categories.yaml`) e i pattern a espressioni regolari personalizzati (`custom_patterns.yaml`) sono file validati al caricamento; aggiungere una nuova categoria o un nuovo identificatore strutturato — un codice aziendale, un riferimento interno, un numero di settore — è quindi una **modifica di configurazione, non di codice**. Le regole così dichiarate vengono eseguite a runtime dal riconoscitore generico (`RegexDetector`) e affiancate ai riconoscitori pre-costruiti di Presidio tramite un `CompositeDetector`, così da presentarsi al merge come un'unica sorgente di pattern.

La scelta separa *ciò che* si rileva — competenza del DPO o del manutentore, in configurazione — da *come* lo si rileva, cioè il motore, che resta stabile. Il file dei pattern personalizzati è riservato agli identificatori che Presidio non copre nativamente (il numero AVS svizzero ne è il primo esempio), per non duplicare ciò che il motore già fornisce. La validazione al caricamento — il pattern deve compilare, ogni `pII_type` deve esistere nel catalogo — fa fallire subito una configurazione errata, invece di degradare il rilevamento in silenzio. Resta fuori, come sviluppo dichiarato, la validazione a *checksum* degli identificatori: un pattern è riconosciuto per forma, non verificato nel contenuto.

7.5 Deployment containerizzato e portabilità

La containerizzazione non è un dettaglio di confezionamento, ma un **requisito di progetto** — l'installazione deve avvenire tramite un meccanismo standardizzato e riproducibile — e un

corollario diretto dell'architettura: i componenti sono concepiti come **servizi raggiungibili su porta e configurati da variabili d'ambiente**, con l'applicazione nel ruolo di interfaccia leggera. È inoltre funzionale alla **minimizzazione del dato**: tutti i servizi risiedono nella rete privata dell'orchestrazione e i dati trattati non lasciano l'infrastruttura on-premise. Questa sezione fissa l'approccio di deployment adottato e ne descrive la realizzazione.

7.5.1 Portabilità e isolamento delle dipendenze

L'obiettivo operativo è una build **riproducibile su qualsiasi piattaforma**: la stessa immagine deve girare identica in sviluppo e in produzione, indipendentemente dall'architettura dell'host. L'argomento più forte a favore emerge da un problema concreto incontrato nello sviluppo. Il livello di rilevamento NER poggia su uno *stack* di apprendimento automatico pesante e fragile (PyTorch e il modello NER GLiNER); sulla macchina di sviluppo — un interprete Python x86_64 in emulazione su hardware ARM — la versione di PyTorch disponibile risultava troppo datata per la restante toolchain (NumPy, transformers), rendendo il livello NER **non costruibile in locale**. Un container Linux, con dipendenze moderne, risolve il problema alla radice e — punto generale — **disaccoppia la build dall'architettura dell'host**: ciò che in locale è un vicolo cieco diventa, nel container, l'ambiente di riferimento in cui la stessa immagine è riproducibile ovunque. La containerizzazione non è quindi soltanto un requisito formale, ma la risposta pragmatica alla fragilità delle dipendenze di apprendimento automatico.

7.5.2 Architettura di deployment

Il sistema è orchestrato come un insieme di servizi. Il **servizio applicativo** racchiude il nucleo — ingestione ROPA, motore di detection con i livelli regex/checksum e NER, verifica di conformità, interfaccia per il DPO — insieme alle relative dipendenze, comprese quelle di apprendimento automatico più onerose (Presidio, spaCy, GLiNER), che nell'immagine Linux si installano senza attriti. Il **runtime del modello generativo** — un servizio **Ollama** (basato su `llama.cpp`) che serve lo SLM scelto (§5.4) su porta, con host e nome del modello letti da variabili d'ambiente (`OLLAMA_HOST`, `ROPA_LLM_MODEL`) — è un container separato, coerente con l'uso selettivo e non ricorrente dell'AI. La **base dati** è, allo stato attuale, un file SQLite su volume persistente (§7.2). Tutta la configurazione — URL della base dati, host e nome del modello — è letta da **variabili d'ambiente**, così la stessa immagine gira in locale e in container cambiando solo la configurazione; i pesi dei modelli e lo stato della base dati vivono su **volumi** dedicati, per non essere riscaricati o persi tra un'esecuzione e l'altra. Coerentemente col profilo hardware del progetto (§5.3), tutti i servizi operano su **CPU**.

7.5.3 Contratti e direzione

Il disaccoppiamento è garantito dai contratti già introdotti all'interno dell'architettura: `PIIDetector` per i rilevatori, `CategoryMapper` per la normalizzazione del ROPA, un client astratto per il modello generativo. Poiché ogni funzionalità dipende solo dalla *forma* del contratto e non dalla sua collocazione, **spostare una funzionalità in un sotto-container dedicato ha un costo quasi nullo**. La direzione è quindi l'articolazione progressiva in **sotto-container per funzionalità**: il runtime generativo è già un servizio a sé; il livello NER, oggi in-process, è scorporabile in un servizio dedicato qualora se ne debba scalare il carico; la persistenza migrerà da SQLite a una base dati relazionale in rete (ad esempio PostgreSQL) quando si superi il singolo nodo. Rimane come sviluppo la pubblicazione di immagini **multi-architettura** (amd64/arm64). Il container costituisce la via canonica per eseguire il sistema completo, dipendenze di apprendimento automatico incluse, mentre il nucleo leggero resta eseguibile nativamente per l'iterazione rapida di sviluppo.

Capitolo 8

Test e validazione

8.1 Validazione del livello di rilevamento

DA AGGIORNARE

Le scelte lasciate aperte in fase di implementazione — se costruire il livello regex/NER sopra Microsoft Presidio (§5.3) e quale motore di NER adottare — sono qui chiuse su base empirica.

8.1.0.0.1 Metodo. La valutazione riusa l'impianto di scoring del progetto: un corpus annotato di documenti sintetici in cui ogni PII è marcata con il proprio `pii_type` e la sua posizione. Un rilevamento conta come vero positivo quando condivide il `pii_type` con uno span di *ground truth* e vi si sovrappone con $Intersection\ over\ Union \geq 0.5$; ogni rilevamento e ogni span sono appaiati al più una volta (*matching greedy*). Da qui si derivano precision, recall e F1 micro-mediati. Ogni configurazione è incapsulata dietro il contratto `PIIDetector` e misurata sullo stesso corpus, così da confrontarle sullo stesso metro.

8.1.0.0.2 Risultati. Il confronto è nella Tabella 8.1. Il livello a pattern di Presidio (espressioni regolari e checksum) supera la baseline regex proprietaria (F1 0.85 contro 0.79) con precisione perfetta e senza falsi positivi, e copre nativamente categorie che la baseline mancava; viene quindi adottato, coerentemente con la scelta di incapsularlo dietro `PIIDetector` (§5.3). Per il livello NER, il modello generico spaCy si rivela inadeguato: F1 0.08, con recall quasi nullo e falsi positivi (etichetta come *location* identificatori strutturati, ad esempio un IBAN). Sostituendolo con GLiNER, NER *zero-shot* specifico per le PII, il livello NER sale a F1 0.44 e — dato decisivo — la detection completa (pattern + NER) passa da F1 0.77 a **0.89**: GLiNER chiude i buchi del NER, portando *address* da 0 a F1 1.00 e il recall di *person_name* a 1.00.

8.1.0.0.3 Limiti. Il corpus è ridotto (poche decine di span) e sintetico: i valori vanno letti come **direzionali**, non come stime definitive, e la loro conferma richiede un corpus più ampio

Tabella 8.1: Validazione del livello di rilevamento sul corpus annotato (P, R, F1).

Configurazione	P	R	F1
Baseline regex (proprietaria)	0.93	0.68	0.79
Presidio — solo pattern/checksum	1.00	0.74	0.85
NER spaCy (it_core_news_lg)	0.17	0.05	0.08
NER GLiNER (gliner_multi_pii)	0.75	0.32	0.44
Presidio (pattern + NER spaCy)	0.75	0.79	0.77
Presidio (pattern + NER GLiNER)	0.89	0.89	0.89

e realistico. Restano inoltre due punti aperti: la precisione di GLiNER su `person_name` è ancora migliorabile (over-tagging occasionale), e l'inferenza GLiNER su CPU è più lenta di spaCy — accettabile, trattandosi di un modello *encoder* e non generativo, ma da tenere presente per il livello di rilevamento ricorrente (blocco B4).

Capitolo 9

Risultati e discussione

Capitolo 10

Sviluppi futuri

Nel corso del lavoro sono state concepite alcune funzionalità che non sono state realizzate. Non si tratta di omissioni né di lavoro rimasto incompiuto per mancanza di tempo: sono **decisioni**, prese confrontando ciò che ciascuna avrebbe aggiunto con ciò che sarebbe costata, e va reso conto tanto delle une quanto delle altre. Un progetto che documenta solo ciò che ha costruito racconta metà del proprio percorso, e non consente a chi lo riprende di capire se una strada sia stata esclusa perché senza sbocco o semplicemente non tentata.

Il criterio applicato non è stato «questa funzionalità è utile?», domanda a cui tutte e tre rispondono affermativamente, bensì il concorso di tre condizioni: il **valore** che la funzionalità porta all'utilizzatore, il **costo** della sua realizzazione, e il **contributo metodologico**, ossia se essa produca un risultato misurabile e discutibile in sede di tesi oppure sia lavoro di realizzazione che nulla aggiunge alla domanda di ricerca. È soprattutto la terza condizione a spiegare gli esiti che seguono: a parità di utilità pratica, una funzionalità che non genera alcun dato da valutare compete con il tempo destinato a ciò che invece ne genera.

Le sezioni che seguono trattano una funzionalità ciascuna secondo lo stesso schema — che cosa avrebbe portato, che cosa sarebbe costata, perché è stata accantonata e a quali condizioni potrebbe essere ripresa — e la Tabella 10.1 ne riassume la valutazione. L'ultima sezione raccoglie invece i limiti già dichiarati altrove nel documento, che restano aperti come naturale prosecuzione del lavoro svolto.

10.1 Analisi contestuale della struttura del file system

L'idea, formulata fra i requisiti funzionali (cfr. 2.1.10) e prevista in architettura come blocco autonomo (cfr. 6.2), era di trattare l'**organizzazione delle cartelle** come un segnale: i nomi dei rami, la profondità e il raggruppamento dei documenti dicono qualcosa sulla finalità per cui quei documenti sono tenuti insieme. Nella forma più economica immaginata, l'ingresso sarebbe stato la resa testuale dell'albero prodotta da un comune comando di sistema —

poche decine di righe di testo per un'intera condivisione — da sottoporre a un modello linguistico.

Il valore atteso era duplice: **proporre** automaticamente l'associazione fra rami dell'albero e attività di trattamento, che oggi il DPO stabilisce a mano, e segnalare le *anomalie posizionali*, ossia i documenti il cui contenuto non è coerente con la cartella che li ospita. È un valore reale, e resta l'unico modo per far emergere una violazione del principio di limitazione della finalità che non sia già visibile nel confronto fra dati rilevati e dati dichiarati.

Il costo, però, non sta dove sembra. Procurarsi l'ingresso è banale; interpretarlo non lo è. Un'inferenza sul significato di una struttura di cartelle è un'euristica, e un'euristica va **misurata**: servirebbe un corpus di alberi documentali annotati con l'attribuzione «corretta» di ciascun ramo, che non esiste e andrebbe costruito sinteticamente, raddoppiando di fatto l'apparato di valutazione (cfr. 4.1.1) per una funzionalità accessoria. A ciò si aggiunge un problema di merito: un'attribuzione inferita dalla macchina diventa il presupposto del verdetto di conformità, che verrebbe così a poggiare su una supposizione anziché su una dichiarazione.

La ragione decisiva dell'accantonamento è tuttavia un'altra: la parte *utile* della funzionalità è stata ottenuta per altra via. Le regole che associano un prefisso di percorso a una o più attività (cfr. 6.6.1) producono lo stesso effetto operativo — associare un intero ramo in un solo gesto, inclusi i documenti che vi compariranno in seguito — con la differenza sostanziale che a stabilire il significato della collocazione è il DPO. Ciò che resterebbe da realizzare non è quindi l'associazione, ma la sola *proposta* di quelle regole: un incremento reale ma marginale, che presuppone comunque l'apparato di valutazione appena descritto. La funzionalità è perciò rinviata, e la sua forma naturale di ripresa è quella di un proponente che suggerisca regole al DPO senza mai applicarle da sé.

10.2 Canale di segnalazione dai dipendenti

Il secondo requisito rimasto sulla carta (cfr. 2.1.9) prevedeva un canale attraverso cui il personale dell'organizzazione potesse segnalare al DPO i dati personali incontrati nel lavoro quotidiano, sfuggiti al rilevamento automatico. Il DPO avrebbe valutato le segnalazioni e le avrebbe usate per arricchire il registro dei trattamenti o i parametri di rilevamento, secondo il ciclo illustrato nella Figura 10.1. Il valore atteso era il più alto dei tre, e mirava al punto più debole del sistema. I falsi negativi sono per definizione invisibili (cfr. 4.2.3): nessuna misura interna può rivelare un dato personale che il rilevatore non vede. I dipendenti sono l'unico sensore che osserva i documenti nel loro contesto d'uso, e sono quindi l'unica sorgente capace di chiudere quella cecità dall'esterno.

Il costo è però sproporzionato, e per una ragione strutturale più che di quantità di lavoro. Il sistema è progettato per un **unico profilo di utilizzatore**, il DPO, e su questa premessa

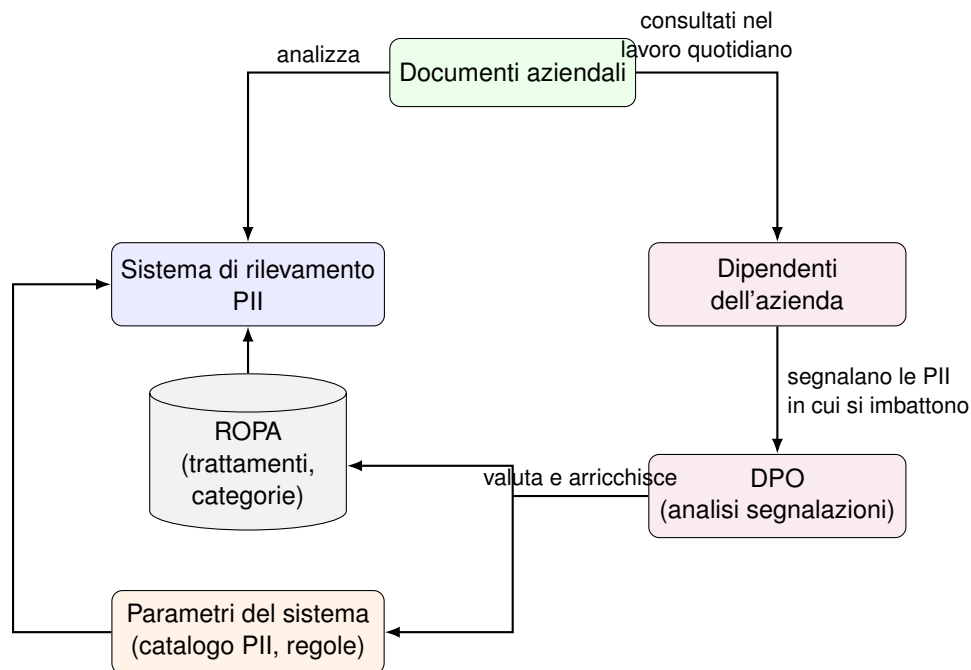


Figura 10.1: Meccanismo di *human-in-the-loop*: nel normale lavoro con i documenti aziendali i dipendenti segnalano al DPO le PII in cui si imbattono; il DPO analizza le segnalazioni e arricchisce il ROPA o i parametri del sistema, migliorando il rilevamento. Il processo di analisi resta trasparente ai dipendenti, per i quali i file sono semplici documenti.

poggia l'intera restituzione: una sola interfaccia, che espone senza distinzioni tutto ciò che il sistema sa. Occorre qui evitare un equivoco: proteggere quella console con un'autenticazione è cosa dovuta e di costo trascurabile, ed è annoverata fra gli interventi da compiere (cfr. 10.5); ciò che è oneroso non è *autenticare*, ma **autorizzare**. Aprire un canale al personale significherebbe passare da un'identità a un insieme di identità da censire e mantenere allineato all'organico, e con esso introdurre ruoli, permessi differenziati, un'interfaccia distinta per chi non deve vedere la console, la notifica e la moderazione delle segnalazioni. È questa la parte preponderante del lavoro, mentre la funzionalità in sé si riduce a un modulo di inserimento.

Vi è inoltre un argomento che da solo sconsiglia la realizzazione nella forma prevista. Il registro delle rilevazioni non contiene i valori dei dati personali, ma ne costituisce la **mappa**: dice dove si trovano, ed è per questo un artefatto sensibile (cfr. 4.2.9). Esporre a tutto il personale un'interfaccia affacciata su quel registro significherebbe ampliarne la superficie di accesso dalla singola figura del DPO all'intera organizzazione: un peggioramento del livello di sicurezza in uno strumento che esiste per ridurre l'esposizione dei dati.

A questo si aggiunge la considerazione che ha determinato l'esito: la funzionalità è un **processo organizzativo**, non una tecnica di rilevamento. Non produce alcun risultato misurabile — non migliora una metrica, non consente un confronto, non sostiene né confuta

un'ipotesi — e il suo effetto dipende interamente dalla partecipazione delle persone, che un prototipo non può mettere alla prova. È lavoro di realizzazione che non alimenta la domanda di ricerca. La funzionalità è pertanto esclusa dal perimetro, e non semplicemente rinviata; qualora venisse ripresa, la forma corretta sarebbe un canale di sola immissione, separato dalla console del DPO e privo di qualsiasi accesso in lettura al registro.

10.3 Estensione dei formati documentali

Il requisito di estrazione (cfr. 2.1.2) è soddisfatto sui formati *born-digital* dell'ufficio — documenti impaginati, videoscrittura, fogli di calcolo e testo semplice. Restano fuori i documenti che sono immagini, ossia le scansioni, e con essi il riconoscimento ottico dei caratteri, i layout complessi a più colonne e le tabelle, oltre ai formati meno diffusi. Allo stato attuale se ne fa a meno: i lettori impiegati sono sufficienti ai formati coperti, e nessun motore di analisi documentale più elaborato è stato introdotto. Qualora l'insieme dei formati da trattare venisse ampliato, sarebbe questo il punto in cui valutare l'adozione di uno strumento dedicato — Docling o analoghi — in luogo dei lettori specifici per singolo formato.

Il valore è reale ma **lineare**: ogni formato aggiunto rende leggibile un'altra fetta del patrimonio documentale, e nulla più. Non cambia ciò che il sistema sa fare, ma solo su quanti documenti riesce a farlo — il che, va detto, in un archivio reale con anni di scansioni può non essere poco.

Anche il costo è in larga parte lineare, ed è per questo che la funzionalità è stata giudicata senza contributo di ricerca: lavoro meccanico, ripetitivo e senza decisioni interessanti da prendere. Il riconoscimento ottico fa però eccezione, perché introduce un salto qualitativo: non è un lettore in più, è una **nuova sorgente di errore** collocata a monte del rilevamento. Il testo che ne esce contiene errori di trascrizione, e ogni PII non rilevata diventerebbe attribuibile indifferentemente al rilevatore o all'estrattore.

È quest'ultimo il motivo per cui l'estensione non è stata affrontata ora: la valutazione misura deliberatamente la *tassa di estrazione* come grandezza distinta dalle prestazioni del rilevamento (cfr. Capitolo 8), e introdurre un estrattore rumoroso avrebbe confuso le due sorgenti di errore proprio mentre le si stava separando. La funzionalità resta rinviata ed è, delle tre, quella ripresa con il minor rischio: il perimetro è contenuto, il contratto di estrazione già in essere non cambia, e l'ordine ragionevole è affrontare prima i formati testuali residui e solo dopo, isolandone la misura, il riconoscimento ottico.

10.4 Quadro riassuntivo

Il quadro mette in evidenza un fatto che le singole sezioni lasciano implicito: nessuna delle tre è stata scartata perché di poco valore. La segnalazione dai dipendenti è anzi quella

Tabella 10.1: Valutazione delle funzionalità concepite e non realizzate.

Funzionalità	Valore	Costo	Contributo metodologico	Esito
Analisi della struttura del file system (B2)	Medio	Alto	Basso: la parte utile è già ottenuta	Rinviata
Segnalazione dai dipendenti	Alto	Alto	Nulla: processo organizzativo	Esclusa
Formati ulteriori e riconoscimento ottico	Medio	Medio	Negativo: confonde la misura	Rinviata

dal valore più alto, ed è l'unica esclusa in via definitiva. A determinare gli esiti è stata la terza colonna: dove la funzionalità non produce un risultato valutabile, il suo costo compete direttamente con il tempo destinato a ciò che invece lo produce.

10.5 Limiti dichiarati e direzioni residue

Oltre alle funzionalità sopra discusse, restano aperti alcuni limiti già riconosciuti nel corso del documento, che costituiscono la naturale prosecuzione del lavoro:

- **Autenticazione della console.** L'interfaccia del DPO è oggi raggiungibile da chiunque abbia accesso di rete alla porta su cui è servita. Anche per un applicativo a utilizzatore singolo, e per quanto collocato in una rete interna, l'accesso va protetto da una schermata di autenticazione: è prassi di base, e a maggior ragione qui, dove ciò che si protegge è la mappa della collocazione dei dati personali (cfr. 4.2.9). L'intervento è circoscritto e di costo trascurabile, e va tenuto distinto dalla gestione di più profili discussa in 10.2: qui si tratta di verificare *un'unica* identità, non di amministrarne molte.
- **Parallelizzazione dell'analisi.** L'elaborazione incrementale riduce il costo delle esecuzioni successive alla prima, ma non quello della passata iniziale, che resta sequenziale (cfr. 4.2.6).
- **Validazione su corpus reale.** Le misure disponibili provengono da corpus sintetici, più ordinati del reale, e vanno intese come limite superiore; la verifica sul patrimonio documentale effettivo di un'organizzazione resta un passaggio non eludibile prima dell'adozione (cfr. 4.1.1, 4.2.4).
- **Passata generativa selettiva.** Il punto di innesto per un rilevatore basato su modello generativo, da impiegare su un campione di documenti, è previsto e predisposto ma non ancora attivo (cfr. 6.4.3).
- **Datazione dal contenuto.** La data di riferimento di un documento è oggi stimata dai metadati e dal file system; ricavarla dal corpo del testo — dove spesso compare in chiaro — renderebbe la verifica della conservazione assai più solida, ma richiede un riconoscimento delle date e una disambiguazione fra le molte presenti in un documento.

Capitolo 11

Diario degli sviluppi (*work in progress*)

Nota. Capitolo temporaneo di servizio. Raccoglie, area per area, ciò che è stato realizzato nel sistema con le relative decisioni progettuali, in una forma già utilizzabile come prosa di tesi: ciascuna voce indica il paragrafo canonico in cui confluirà e va rimossa da qui una volta assorbita. Va eliminato a lavoro concluso.

11.1 Data di riferimento del documento e provenienza della stima

Destinazione: §6.6.3 (architettura) e il paragrafo sull'estrazione del Capitolo 7.

La verifica della conservazione confronta l'età di un documento con la durata dichiarata nel registro dei trattamenti, e presuppone quindi di sapere *quando* quel documento risale. Il file system, da solo, è un testimone inaffidabile: la copia di una condivisione di rete, una migrazione o un ripristino da backup riscrivono la data di ultima modifica di ogni file, e un documento del 2019 appare improvvisamente come scritto ieri. Assumere il *mtime* come data del documento significa quindi accettare un falso negativo sistematico proprio nello scenario — l'archivio storico mai più toccato — che il controllo dovrebbe intercettare.

I documenti nativi digitali, però, portano spesso dentro di sé una risposta migliore: il dizionario informativo di un PDF conserva `modDate` e `creationDate`, i formati Office espongono le *core properties* con data di creazione e di ultima modifica. Il modulo `extraction/dates.py` interroga per primo questo livello e ricade sul *mtime* solo quando non trova nulla di utilizzabile, secondo un ordine di preferenza dichiarato: data di modifica interna, data di creazione interna, data del file system.

La scelta progettuale che qualifica il meccanismo non è però la catena in sé, bensì il fatto che il sistema **dichiara che cosa ha usato**: il valore restituito (`ReferenceDate`) porta con sé la provenienza (`DateSource`, con i due casi `content_metadata` e `file_mtime`) e il campo

esatto da cui la data è stata letta. Il verdetto di conformità può così qualificare quanto vale il segnale che mostra al DPO, invece di presentare una stima come se fosse un accertamento; una segnalazione fondata sul solo *mtime* resta un indizio da verificare, non una violazione conclamata.

I metadati reali sono peraltro spesso sporchi — campi inizializzati all'epoca Unix, orologi impostati male, date collocate nel futuro — e credervi ciecamente reintrodurrebbe l'errore da un'altra porta. Una sola guardia di plausibilità, concentrata in un unico punto del modulo perché non possa divergere da un formato all'altro, scarta i valori anteriori al 1990 o successivi al momento presente e passa al candidato seguente: una data di modifica corrotta accanto a una data di creazione sensata produce quest'ultima, non un ritorno immediato al file system. Allo stesso modo, l'impossibilità di leggere i metadati di un file — un PDF troncato, proprietà illeggibili — non è trattata come un errore: datare un documento non deve poter interrompere la sua analisi, e il ritorno al *mtime* è già la risposta corretta.

Resta un'approssimazione consapevole, ed è opportuno dirlo: i metadati interni possono mancare del tutto (testo semplice, Word legacy, ODF), possono essere stati riscritti da una conversione di formato e possono a loro volta mentire. Il sistema non tenta, allo stato attuale, di ricavare la data dal corpo del documento — una lettera che si apre con «Milano, 12 marzo 2019» resta illeggibile sotto questo profilo — perché l'euristica sarebbe rumorosa e andrebbe validata con lo stesso rigore del rilevamento delle PII.

11.2 Due orologi distinti nel registro dei documenti

Destinazione: paragrafo sul modello dati del registro, Capitolo 7.

Il registro delle PII conserva ora, per ciascun documento, **due** informazioni temporali che è essenziale non confondere, perché rispondono a domande diverse e si comportano in modo diverso. La prima è la *data di riferimento* (*reference_date*, con la sua provenienza in *reference_date_source*): è una grandezza **semantica**, dice quanto è vecchio il *contenuto*, e può provenire dall'interno del file (§11.1); è ciò su cui lavora la verifica della conservazione. La seconda è l'*impronta* del file (*source_mtime* e *source_size*): è una grandezza **tecnica**, descrive lo stato osservabile del file su disco al momento dell'ultima analisi, e serve a rispondere a una domanda del tutto diversa — questo documento è cambiato da quando l'abbiamo guardato, e vale quindi la pena rianalizzarlo?

La distinzione non è pedanteria terminologica: usare la data semantica anche per la seconda domanda produrrebbe un errore silenzioso. La data di modifica interna di un PDF resta spesso invariata quando il file viene riscritto, e un documento effettivamente modificato apparirebbe intatto, venendo saltato indefinitamente da ogni analisi successiva. Simmetricamente, usare la data del file system come età del contenuto è l'approssimazione che il paragrafo precedente ha messo in discussione. Due domande, due misure, due campi.

L'impronta è deliberatamente il segnale più economico che funziona — data di modifica e dimensione, ottenute con una sola interrogazione al file system — e non un'impronta crittografica del contenuto: calcolare un digest di ogni file di una condivisione aziendale costerebbe esattamente ciò che l'analisi incrementale intende risparmiare. A queste si affianca `last_scanned_at`, il momento dell'ultima analisi effettiva del documento, che una scansione in cui il documento venga saltato non aggiorna.

Ne discende una regola di aggiornamento precisa: date e impronta si rinfrescano *solo* quando la scansione ha effettivamente osservato il file. La riconciliazione dei documenti spariti dal disco, che registra una scansione vuota per marcarne le PII come rimosse, non ha alcun file da guardare e non deve perciò azzerare ciò che l'ultima osservazione reale aveva registrato.

11.3 Il silenzio non è conformità: conservazione non verificabile

Destinazione: §6.6.3 (architettura) e il paragrafo sul verdetto di conformità, Capitolo 7.

Il controllo di conservazione confronta l'età del documento con la durata dichiarata nel registro, ma quella durata non è sempre un numero. I registri reali esprimono spesso un *criterio* anziché un termine — «per la durata del rapporto di lavoro», «fino alla revoca del consenso» — e l'ingestione, non trovando alcuna quantità da estrarre, lascia la durata indefinita. Nella prima versione del controllo quelle categorie venivano semplicemente saltate, con una conseguenza che è opportuno enunciare per quello che è: il documento risultava conforme sull'asse della conservazione senza che alcuna verifica fosse mai avvenuta. Il caso «nessuno ha potuto controllare» era indistinguibile dal caso «controllato e in regola».

Il verdetto distingue ora esplicitamente i due esiti. Ogni macro-categoria dichiarata i cui dati siano ancora presenti nel documento produce, in alternativa, una **violazione** — quando il documento è più vecchio del termine dichiarato — oppure una segnalazione di **conservazione non verificabile**, quando il registro esprime un criterio. La seconda non rende il documento non conforme, esattamente come la segnalazione delle categorie dichiarate ma non riconducibili ad alcun tipo rilevabile: è un'informazione destinata al DPO, che indica dove il registro andrebbe reso più preciso. Vale la pena osservare che questa segnalazione non dipende dalla data del documento: la lacuna è nel registro, non nel documento, e va comunicata anche quando la data non è ricostruibile.

Alla stessa logica appartiene un'altra asimmetria corretta in questa fase. Il confronto fra dichiarato e rilevato considera, di norma, le sole categorie *confermate* dal DPO, ignorando le associazioni ancora in stato di proposta; il controllo di conservazione, invece, le contava tutte. Lo stesso documento poteva così ricevere due giudizi incoerenti sulla medesima categoria — «non dichiarata», e quindi orfana, per un ramo del verdetto; «dichiarata e conservata oltre il

termine» per l'altro. La regola su quali categorie continuo è ora scritta in un solo punto del codice e utilizzata da entrambi i rami, così che i due non possano più divergere.

Infine, sul versante dell'ingestione, il riconoscimento delle durate è stato esteso alle unità francesi (*an, ans, mois*) accanto a quelle inglesi e italiane già supportate. Non è un dettaglio linguistico: il modello di registro adottato come riferimento è quello dell'autorità francese, e una dicitura come «5 ans» lasciata non interpretata avrebbe disattivato in silenzio il controllo di conservazione per quella categoria — ricadendo esattamente nel difetto descritto sopra, ma per una causa evitabile.

11.4 Dalla scheda del documento alla vista d'insieme

Destinazione: paragrafo sulla dashboard del DPO, Capitolo 7.

Il verdetto di conformità nasce come giudizio su un singolo documento, ed è la forma corretta per la domanda «questo file è in regola?». Non è però la domanda che un DPO rivolge a una condivisione aziendale: la sua è «mostrami tutto ciò che è oltre il termine». Con trecento documenti, ottenere quella risposta aprendo una scheda alla volta non è una procedura praticabile, ed è il modo più sicuro perché una violazione reale resti invisibile in mezzo al rumore.

È stata perciò aggiunta una vista d'insieme, disponibile sia come pagina web sia come comando da terminale, che applica il medesimo verdetto all'intero registro e ne conserva soltanto le righe che hanno qualcosa da dire, ordinate per **gravità**, cioè per il numero di mesi di sfornamento. La misura della gravità non è un dettaglio estetico: un mese oltre il termine è un promemoria, dieci anni oltre sono un incidente, e un elenco che non li distingue costringe a rileggerlo tutto.

Tre vincoli hanno guidato la realizzazione. Primo, **nessuna logica nuova**: la vista richiama la stessa funzione che produce il verdetto del singolo documento, così le due schermate non possono divergere. Secondo, **nessuna scrittura**: consultare una pagina non deve modificare lo stato del sistema, mentre il controllo per-documento, quando invocato per l'assegnazione, persiste la copertura di ciascuna PII. Terzo, il registro dei trattamenti viene letto **una sola volta** e tenuto in memoria per la durata della vista, anziché una volta per documento.

I documenti privi di associazione a un trattamento sono deliberatamente esclusi: senza un trattamento non esiste una durata dichiarata con cui confrontarsi, e includerli significherebbe sommergere le violazioni effettive sotto ogni file non ancora classificato della condivisione. È una scelta che vale la pena rendere esplicita nella pagina stessa, perché un elenco vuoto non significhi «tutto a posto» quando in realtà significa «non c'è ancora nulla da confrontare».

11.5 Rianalizzare solo ciò che è cambiato

Destinazione: §2.3.6 (requisiti) e il paragrafo sulla scansione batch, Capitolo 7.

La scansione di una cartella rileggeva per intero, a ogni esecuzione, ogni documento incontrato: estrazione del testo e rilevamento su file identici a quelli della volta precedente. Su un albero aziendale è il costo dominante dell'intera procedura, ed è un costo interamente sprecato; soprattutto, rende impraticabile l'esecuzione periodica che il modello d'impiego previsto — «si indica una cartella e il sistema se ne occupa» — presuppone. Il registro sapeva già *che cosa* fosse cambiato nel contenuto di un documento, grazie al confronto fra scansioni successive, ma non sapeva *se valesse la pena guardarlo*.

Il rimedio adottato è il più semplice che funzioni: si confrontano la data di modifica e la dimensione del file con quelle registrate all'ultima analisi e, se coincidono, il documento non viene riletto. Si è scelto deliberatamente di non calcolare un'impronta crittografica del contenuto, che sarebbe più precisa ma richiederebbe di leggere ogni byte di ogni file — esattamente il costo che la misura intende evitare. Il confronto è per *differenza* e non per posteriorità: un file ripristinato a una versione precedente è cambiato quanto uno modificato, e un criterio del tipo «più recente di» lo escluderebbe per sempre dall'analisi. Una tolleranza di un secondo assorbe le diverse granularità temporali dei file system, che altrimenti farebbero apparire modificato ogni documento.

Il punto delicato dell'intervento non è però il criterio, bensì un'invariante preesistente. La scansione riconcilia il registro con il disco marcando come rimosse le PII dei documenti spariti, e per farlo si basa sull'insieme dei documenti «visti». Se un documento saltato perché invariato non entrasse in quell'insieme, la seconda scansione dichiarerebbe rimosse le PII di ogni file mai toccato — un danno molto peggiore del costo che si voleva risparmiare. Il significato dell'insieme è stato quindi reso esplicito: vi appartiene ogni file **trovato sul disco**, indipendentemente da ciò che è accaduto dopo. La stessa precisazione ha corretto un difetto preesistente e indipendente: un file che fallisce l'estrazione non entrava in quell'insieme, e le sue PII venivano dichiarate rimosse pur essendo il file ancora al suo posto. Coerentemente, un file mai analizzato con successo non viene mai considerato invariato, ma ritentato a ogni esecuzione.

Un'ultima insidia riguarda il motore di rilevamento anziché i documenti. Abilitare un modello di riconoscimento più capace, o aggiungere una regola alla configurazione dei pattern, cambia ciò che verrebbe trovato in un file i cui byte non si sono mossi: il registro continuerebbe a mostrare i risultati del motore precedente presentandoli come aggiornati. Un registro fermo è un problema; un registro che mente sul proprio aggiornamento è peggio. Per questo, accanto all'impronta del file, viene registrata un'impronta della **configurazione di rilevamento** — identificatori dei rilevatori attivi e contenuto dei file di configurazione — e una

sua variazione invalida da sola il salto, senza che l'operatore debba ricordarsi di forzare una rianalisi completa.

Due comportamenti restano volutamente fuori dall'ottimizzazione. L'anteprima della scansione dichiara in anticipo quanti file verrebbero saltati, perché la pagina che promette di dire «che cosa verrà analizzato» deve continuare a dire il vero anche adesso che la risposta è più articolata. E i file caricati dal browser vengono sempre rianalizzati per intero: materializzati sul server al momento del caricamento, portano come data di modifica quella del caricamento stesso, sicché nessuna impronta calcolata su di essi significherebbe qualcosa.

Capitolo 12

Punti aperti (*work in progress*)

Nota. Capitolo temporaneo di servizio. Riunisce, per comodità di discussione, le decisioni ancora da prendere sparse nel documento; ciascuna rimanda alla sezione in cui vive e dove andrà riportata la soluzione una volta chiusa. Va rimosso a lavoro concluso.

12.1 Decisioni progettuali aperte

12.1.1 Requisiti da confermare

12.1.1.0.1 PA-1 — Aggregazione delle PII frammentate. Va deciso se e in che misura il sistema debba ricondurre frammenti dispersi (in uno o più documenti) a un'unica entità coerente, e con quali criteri verificabili (co-occorrenza, similarità semantica, identificatori condivisi). Il requisito non è al momento incluso nell'analisi dei requisiti (cfr. Capitolo 2): la decisione resta aperta.

12.1.1.0.2 PA-2 — Costruzione di una knowledge base delle entità. Va deciso se il sistema debba costruire, a partire dalle PII estratte, una rappresentazione strutturata delle entità (via NER) come base per il confronto col ROPA, e con quale livello di dettaglio. Il requisito non è al momento incluso nell'analisi dei requisiti (cfr. Capitolo 2): la decisione resta aperta. Impatta direttamente su PA-1 e sullo *scope* complessivo (cfr. rischio di *scope creep*, §4.1.5).

12.1.1.0.3 PA-12 — Affidabilità della data del documento. La verifica della conservazione (cfr. §6.6.3) poggia su una data stimata: i metadati interni del file quando esistono, altrimenti la data di ultima modifica sul file system. Entrambe le fonti hanno limiti noti e non eliminabili con l'approccio attuale — il `mtime` viene azzerato da copie massive, migrazioni e ripristini da backup; i metadati interni mancano del tutto in alcuni formati (testo semplice, Word legacy, ODF) e possono essere riscritti da una conversione. Resta da decidere se valga la pena ricavare la data dal corpo del documento (formule del tipo «Milano, 12 marzo 2019»),

il che richiederebbe un'euristica da validare con lo stesso rigore adottato per il rilevamento delle PII, oppure se accettare la stima attuale dichiarandone i limiti nel verdetto, come avviene ora.

12.1.2 Modello dati del ROPA

Voci già isolate come «punti aperti di progettazione» in §6.1.5, qui riunite.

12.1.2.0.1 PA-3 — Granularità della corrispondenza categorie–PII. Va fissato se una categoria di dati del ROPA mappi su un singolo `pii_type` o su un insieme. Una voce sbrigativa («dati anagrafici») corrisponde di norma a più categorie canoniche, il che suggerisce una relazione 1 – N da formalizzare.

12.1.2.0.2 PA-4 — Allineamento nLPD–GDPR. Il modello parte dai campi CNIL (impianto GDPR); vanno annotate le differenze rilevanti dell'art. 12 LPD, in particolare soglie ed eccezioni per le PMI, che incidono su quali trattamenti debbano comparire nel registro.

12.1.2.0.3 PA-5 — ROPA non strutturato. Il modello assume un registro già compilato in forma tabellare. Va deciso se (e quanto) supportare un ROPA fornito come testo libero, restando entro il vincolo di non delegare all'AI la strutturazione complessiva del documento.

12.1.3 Persistenza del registro PII

12.1.3.0.1 Snapshot o event-logger Da decidere se il metodo per salvare le differenze tra versioni del FS analizzato seguano il paradigma degli snapshot o dell'event-logger.

12.1.3.0.2 PA-6 — Registrazione della «scomparsa» di una PII: esplicita o implicita? Quando una scansione non ritrova un'istanza confermata in precedenza, va deciso se scrivere attivamente una `PIIChange` di tipo `REMOVED` (esplicita, log autoesplicativo ma con confronto attivo a ogni scansione) o dedurre lo stato «rimossa» a posteriori (implicita, più semplice ma non autosufficiente). Ipotesi preferita: esplicita.

12.1.3.0.3 PA-7 — Significato dell'evento «modifica». Poiché il registro non salva i valori delle PII, un evento di «modifica» può riferirsi solo al cambio di **posizione** (`MOVED`) o a una **riclassificazione** del tipo. Va confermata l'ipotesi di non prevedere un generico `MODIFIED`, ma di scomporlo in variazioni dal significato preciso.

12.1.3.0.4 PA-8 — Vocabolario chiuso dei tipi di variazione. Si propone di formalizzare `change_type` come enumerazione chiusa (`NEW`, `CONFIRMED`, `MOVED`, `REMOVED`). Resta da confermare se il set è completo per tutti i casi d'uso previsti (dipende anche da PA-6).

12.1.4 Scelte implementative

12.1.4.0.1 PA-9 — Tecnologia di persistenza. Il modello concettuale è *technology-agnostic*; va scelta la tecnologia concreta tra base dati relazionale (es. SQLite, con *join* nativi) e persistenza su file semi-strutturati (JSON/JSONL, più leggera ma senza garanzie relazionali). La decisione è deliberatamente rimandata a valle della chiusura del modello dati (in particolare di PA-6, PA-7, PA-8) (cfr. §7.2).

12.1.4.0.2 PA-10 — Riutilizzo di Microsoft Presidio come livello di detection. Va deciso, su base empirica, se costruire il livello regex/NER sopra Presidio (incapsulato dietro il contratto `PIIDetector`) anziché scriverlo da zero. Il giudizio è affidato a una misurazione diretta (precision/recall/F1) sullo stesso corpus della baseline (cfr. §5.3).

12.1.5 Metodologia di detection

12.1.5.0.1 PA-11 — Strategia per il modello NER. Va scelto e motivato l'approccio tra: (A) zero-shot puro (GLiNER base), (B) modello pre-fine-tunato da terzi (es. `gliner-pii`), (C) fine-tuning leggero in proprio. Dai vincoli di progetto (*zero-training*, assenza di dati annotati, esecuzione locale) l'opzione C è da scartare a favore di A/B; manca la discussione strutturata con tabella di confronto nel Capitolo 6.

12.2 Contenuti ancora da redigere

Non decisioni, ma segnaposto e sezioni incomplete sparse nel testo, raccolti qui per non perderli.

- **TB-1** — Architettura: confronto strutturato e scelta del modello NER (tabella dei tre approcci di PA-11) da inserire nel Capitolo 6.
- **TB-2** — Architettura: diagramma ER in TikZ del modello di persistenza (`PIIInstance`, `PIIChange`, `Document`, `Scan`, `ProcessingActivity`), da produrre dopo la chiusura di PA-6/7/8 (cfr. §6.5.4).
- **TB-3** — Implementazione: roadmap implementativa e di validazione (baseline zero-shot GLiNER, integrazione modello pre-fine-tunato, eventuale esplorazione di fine-tuning) nel Capitolo 7.

Elenco delle figure

6.1	Architettura a macro-blocchi del sistema	54
6.2	Dettaglio del blocco B4: pipeline ibrida di rilevamento PII	55
6.3	Diagramma ER del registro delle PII rilevate	66
10.1	Meccanismo di <i>human-in-the-loop</i> : nel normale lavoro con i documenti aziendali i dipendenti segnalano al DPO le PII in cui si imbattono; il DPO analizza le segnalazioni e arricchisce il ROPA o i parametri del sistema, migliorando il rilevamento. Il processo di analisi resta trasparente ai dipendenti, per i quali i file sono semplici documenti.	87

Elenco delle tabelle

3.1 Prestazioni di rilevamento degli approcci di PII detection, secondo le evidenze di letteratura.	27
4.1 Valutazione dei rischi identificati: probabilità, impatto e priorità residua.	40
5.1 Costo energetico ed economico degli approcci di PII detection, secondo le evidenze di letteratura.	45
5.2 Criteri di selezione dello SLM e relative fonti.	48
5.3 Throughput (token/s), 4-bit GGUF su CPU [54].	49
5.4 Occupazione di memoria: FP16 vs 4-bit (Q4) e RAM minima [54].	49
5.5 Raccomandazione per task (fascia CPU).	51
6.1 Approcci valutati per la persistenza del registro delle PII.	63
6.2 Sintesi comparativa dei tre approcci di persistenza.	65
8.1 Validazione del livello di rilevamento sul corpus annotato (P, R, F1).	82
10.1 Valutazione delle funzionalità concepite e non realizzate.	89

Bibliografia

- [1] Shirley Radack. «Protecting Personally Identifiable Information». In: *Guide to Protecting the Confidentiality of Personally Identifiable Information (PII)*. NIST Special Publication (apr. 2010). URL: <https://nvlpubs.nist.gov/nistpubs/legacy/sp/nistspecialpublication800-122.pdf>.
- [2] Parlamento europeo e Consiglio dell'Unione europea. *Regolamento (UE) 2016/679*. 27 Apr. 2016. URL: <http://data.europa.eu/eli/reg/2016/679/oj> (visitato il giorno 16/06/2026).
- [3] *RS 235.1 - Legge Federale Del 25 Settembre 2020 Sulla Protezione Dei Dati (LPD) / Fedlex*. URL: <https://www.fedlex.admin.ch/eli/cc/2022/491/it> (visitato il giorno 16/06/2026).
- [4] *IEEE-29148-SRS-LaTeX-Template/IEEE-29148-2018-SRS-Template.Tex at Main · Wxinx/IEEE-29148-SRS-LaTeX-Template*. GitHub. URL: <https://github.com/wxinx/IEEE-29148-SRS-LaTeX-Template/blob/main/IEEE-29148-2018-SRS-Template.tex> (visitato il giorno 13/06/2026).
- [5] Sona College of Technology, India et al. «A FRAMEWORK FOR ANALYSING UNSTRUCTURED DATA IN COMPUTING DEVICES». In: *ICTACT Journal on Soft Computing* 12.1 (1 ott. 2021), pp. 2464–2468. ISSN: 09766561, 22296956. DOI: 10.21917/ijsc.2021.0352. URL: <https://ictactjournals.in/IJSC/ArticleDetails?id=6604> (visitato il giorno 09/06/2026).
- [6] A. Makhambet e A. Moldagulova. «A Comparative Analysis of Machine Learning Methods for Personal Information Recognition (PII) in Unstructured Texts». In: *Computing & Engineering* 3.1 (31 mar. 2025), pp. 41–52. ISSN: 3080-3829. DOI: 10.51301/ce.2025.i1.07. URL: <https://ce.journal.satbayev.university/index.php/journal/article/view/1308> (visitato il giorno 09/06/2026).
- [7] Harshit Rajgarhia et al. *An Evaluation Study of Hybrid Methods for Multilingual PII Detection*. arXiv.org. 8 Ott. 2025. URL: <https://arxiv.org/abs/2510.07551v1> (visitato il giorno 13/06/2026).

- [8] Nagaraju Velishetty. «Personal Identifiable Information (PII) Detection and Identification for Fintech with AI and Text Analytics». Tesi di laurea mag. Dublin, National College of Ireland, 10 ago. 2024. 23 pp. URL: <https://norma.ncirl.ie/7037/> (visitato il giorno 09/06/2026).
- [9] Jianliang Yang et al. «Exploring the Application of Large Language Models in Detecting and Protecting Personally Identifiable Information in Archival Data: A Comprehensive Study*». In: *2023 IEEE International Conference on Big Data (BigData)*. 2023 IEEE International Conference on Big Data (BigData). Sorrento, Italy: IEEE, 15 dic. 2023, pp. 2116–2123. ISBN: 979-8-3503-2445-7. DOI: 10.1109/BigData59044.2023.10386949. URL: <https://ieeexplore.ieee.org/document/10386949/> (visitato il giorno 17/06/2026).
- [10] Orlando Amaral et al. *NLP-based Automated Compliance Checking of Data Processing Agreements against GDPR*. 18 Giu. 2023. DOI: 10.48550/arXiv.2209.09722. arXiv: 2209.09722 [cs.SE]. URL: <http://arxiv.org/abs/2209.09722> (visitato il giorno 09/06/2026). Pre-pubblicato.
- [11] Sinan Gultekin et al. «An Energy-Based Comparative Analysis of Common Approaches to Text Classification in the Legal Domain». In: *AI, Machine Learning and Applications*. 27 Gen. 2024, pp. 31–41. DOI: 10.5121/cs.it.2024.140203. arXiv: 2311.01256 [cs.CL]. URL: <http://arxiv.org/abs/2311.01256> (visitato il giorno 19/06/2026).
- [12] Johannes Zschache e Tilman Hartwig. «Comparing Energy Consumption and Accuracy in Text Classification Inference». In: *Scientific Reports* 16.1 (17 apr. 2026), p. 12717. ISSN: 2045-2322. DOI: 10.1038/s41598-026-45023-0. arXiv: 2508.14170 [cs.CL]. URL: <http://arxiv.org/abs/2508.14170> (visitato il giorno 19/06/2026).
- [13] Pritesh Jha. *PIIBench: A Unified Multi-Source Benchmark Corpus for Personally Identifiable Information Detection*. arXiv.org. 17 Apr. 2026. DOI: 10.5281/zenodo.19533263. URL: <https://arxiv.org/abs/2604.15776v1> (visitato il giorno 16/06/2026).
- [14] Ian Goodfellow, Yoshua Bengio e Aaron Courville. *Deep Learning*. 2016. URL: <https://www.deeplearningbook.org/contents/ml.html> (visitato il giorno 21/06/2026).
- [15] Noora Al Qahtani. *A Comparative Study of Supervised vs. Unsupervised Learning Techniques for Real-Time Intrusion Detection in Cybersecurity Networks*. ResearchGate. URL: https://www.researchgate.net/publication/396389868_A_Comparative_Study_of_Supervised_vs_Unsupervised_Learning_Techniques_for_Real-Time_Intrusion_Detection_in_Cybersecurity_Networks (visitato il giorno 20/06/2026).
- [16] Imed Keraghel, Stanislas Morbieu e Mohamed Nadif. *Recent Advances in Named Entity Recognition: A Comprehensive Survey and Comparative Study*. 20 Dic. 2024.

- DOI: 10.48550/arXiv.2401.10825. arXiv: 2401.10825 [cs.CL]. URL: <http://arxiv.org/abs/2401.10825> (visitato il giorno 25/06/2026). Pre-pubblicato.
- [17] Byamugisha Africano et al. «PII Detection in Low-Resource Languages Using Explainable Deep Learning Techniques». In: *Proceedings of the 2024 Sixteenth International Conference on Contemporary Computing*. IC3 2024: 2024 Sixteenth International Conference on Contemporary Computing. Noida India: ACM, 8 ago. 2024, pp. 94–103. ISBN: 979-8-4007-0972-2. DOI: 10.1145/3675888.3676036. URL: <https://dl.acm.org/doi/10.1145/3675888.3676036> (visitato il giorno 25/06/2026).
- [18] Jacob Devlin et al. «BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding». In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. NAACL-HLT 2019. A cura di Jill Burstein, Christy Doran e Thamar Solorio. Minneapolis, Minnesota: Association for Computational Linguistics, giu. 2019, pp. 4171–4186. DOI: 10.18653/v1/N19-1423. URL: <https://aclanthology.org/N19-1423/> (visitato il giorno 21/06/2026).
- [19] Xichen Zhang et al. «Personally Identifiable Information Detection in Smart Edge Computing: A Robust Model Evaluation». In: *Knowledge-Based Systems* 328 (25 ott. 2025), p. 114153. ISSN: 0950-7051. DOI: 10.1016/j.knosys.2025.114153. URL: <https://www.sciencedirect.com/science/article/pii/S0950705125011943> (visitato il giorno 20/06/2026).
- [20] Yongqin Xian et al. *Zero-Shot Learning – A Comprehensive Evaluation of the Good, the Bad and the Ugly*. 23 Set. 2020. DOI: 10.48550/arXiv.1707.00600. arXiv: 1707.00600 [cs.CV]. URL: <http://arxiv.org/abs/1707.00600> (visitato il giorno 21/06/2026). Pre-pubblicato.
- [21] Tom B. Brown et al. *Language Models Are Few-Shot Learners*. 22 Lug. 2020. DOI: 10.48550/arXiv.2005.14165. arXiv: 2005.14165 [cs.CL]. URL: <http://arxiv.org/abs/2005.14165> (visitato il giorno 21/06/2026). Pre-pubblicato.
- [22] Urchade Zaratiana, Ash Lewis e George Hurn-Maloney. *GLiNER2-PII: A Multilingual Model for Personally Identifiable Information Extraction*. arXiv.org. 11 Mag. 2026. URL: <https://arxiv.org/abs/2605.09973v1> (visitato il giorno 17/06/2026).
- [23] Luca Mainetti e Andrea Elia. «Detecting Personally Identifiable Information Through Natural Language Processing: A Step Forward». In: *ResearchGate* (20 mar. 2026). DOI: 10.3390/asi8020055. URL: https://www.researchgate.net/publication/390936945_Detecting_Personally_Identifiable_Information_Through_Natural_Language_Processing_A_Step_Forward (visitato il giorno 17/06/2026).

- [24] «Enterprise-Scale PII De-Identification with Microsoft Presidio Anonymizer: Architecture, Use Cases, and Best Practices». In: *ResearchGate* (3 mar. 2026). ISSN: 3050-9416. URL: https://www.researchgate.net/publication/399570056_Enterprise-Scale_PII_De-Identification_with_Microsoft_Presidio_Anonymizer_Architecture_Use_Cases_and_Best_Practices (visitato il giorno 17/06/2026).
- [25] Yanai Elazar et al. *What's In My Big Data?* 5 Mar. 2024. DOI: 10.48550/arXiv.2310.20707. arXiv: 2310.20707 [cs.CL]. URL: <http://arxiv.org/abs/2310.20707> (visitato il giorno 24/06/2026). Pre-pubblicato.
- [26] Minqi Jiang et al. *Generative Data Refinement: Just Ask for Better Data*. 11 Set. 2025. DOI: 10.48550/arXiv.2509.08653. arXiv: 2509.08653 [cs.LG]. URL: <http://arxiv.org/abs/2509.08653> (visitato il giorno 25/06/2026). Pre-pubblicato.
- [27] Noopur Zambare et al. *Towards Fair and Efficient De-identification: Quantifying the Efficiency and Generalizability of De-identification Approaches*. 27 Gen. 2026. DOI: 10.48550/arXiv.2602.15869. arXiv: 2602.15869 [cs.CL]. URL: <http://arxiv.org/abs/2602.15869> (visitato il giorno 25/06/2026). Pre-pubblicato.
- [28] Ben. *Best Open Source Models for PII Redaction*. Grepture. 25 Mar. 2026. URL: <https://grepture.com/blog/best-open-source-models-pii-redaction> (visitato il giorno 25/06/2026).
- [29] Kushagra Mishra, Harsh Pagare e Kanhaiya Sharma. «A Hybrid Rule-Based NLP and Machine Learning Approach for PII Detection and Anonymization in Financial Documents». In: *Scientific Reports* 15.1 (2 lug. 2025), p. 22729. ISSN: 2045-2322. DOI: 10.1038/s41598-025-04971-9. URL: <https://www.nature.com/articles/s41598-025-04971-9> (visitato il giorno 25/06/2026).
- [30] *Comparing Best NER Models For PII Identification*. 18 Nov. 2025. URL: <https://www.protecto.ai/blog/best-ner-models-for-pii-identification/> (visitato il giorno 25/06/2026).
- [31] *Knowledgator/Gliner-Pii-Base-v1.0 · Hugging Face*. 29 Gen. 2026. URL: <https://huggingface.co/knowledgator/gliner-pii-base-v1.0> (visitato il giorno 25/06/2026).
- [32] *Open Source PHI De-Identification: A Technical Review*. IntuitionLabs. URL: <https://intuitionlabs.ai/articles/open-source-phi-de-identification-tools> (visitato il giorno 25/06/2026).
- [33] Andres Felipe Zambrano et al. «De-Identifying Student Personally Identifying Information in Discussion Forum Posts with Large Language Models». In: *Information and Learning Sciences* 126.5–6 (7 apr. 2025), pp. 401–424. ISSN: 2398-5348. DOI: 10.1108/ILS-11-2024-0156. URL: <https://doi.org/10.1108/ILS-11-2024-0156> (visitato il giorno 25/06/2026).

- [34] Lokesh Koli, Shubham Kalra e Karanpreet Singh. *Decoding Complexity: Intelligent Pattern Exploration with CHPDA (Context Aware Hybrid Pattern Detection Algorithm)*. 9 Feb. 2025. DOI: 10.48550/arXiv.2502.07815. arXiv: 2502.07815 [cs.CR]. URL: <http://arxiv.org/abs/2502.07815> (visitato il giorno 25/06/2026). Pre-pubblicato.
- [35] *Microsoft Presidio*. URL: <https://microsoft.github.io/presidio/> (visitato il giorno 22/06/2026).
- [36] Urchade Zaratiana et al. *GLiNER: Generalist Model for Named Entity Recognition Using Bidirectional Transformer*. 14 Nov. 2023. DOI: 10.48550/arXiv.2311.08526. arXiv: 2311.08526 [cs.CL]. URL: <http://arxiv.org/abs/2311.08526> (visitato il giorno 22/06/2026). Pre-pubblicato.
- [37] *Scrubadub*. 22 Giu. 2026. URL: <https://github.com/LeapBeyond/scrubadub> (visitato il giorno 22/06/2026).
- [38] *Overview of Varonis AI-Powered Data Discovery and Classification*. URL: <https://www.varonis.com/blog/ai-data-classification> (visitato il giorno 22/06/2026).
- [39] *Data Classification Buyer's Guide: How to Choose a Data Classification Solution*. URL: <https://www.varonis.com/blog/data-classification-buyers-guide> (visitato il giorno 16/06/2026).
- [40] *Varonis Software Pricing & Plans 2025: See Your Cost*. URL: <https://www.vendr.com/marketplace/varonis> (visitato il giorno 22/06/2026).
- [41] *BigID Data Sheet: GDPR Compliance*. Slideshare. URL: <https://www.slideshare.net/slideshow/bigid-data-sheet-gdpr-compliance-120471407/120471407> (visitato il giorno 16/06/2026).
- [42] *Privacy Suite*. BigID. URL: <https://bigid.com/privacy-suite/> (visitato il giorno 16/06/2026).
- [43] *Detecting Personal Names in Texts: Technical Whitepaper*. Technical whitepaper. Limassol, Cipro: PII-Tools Ltd., 2024.
- [44] *PII Tools | Discover, Analyze & Remediate Sensitive Data Anywhere*. PII Tools. URL: <https://pii-tools.com/> (visitato il giorno 16/06/2026).
- [45] Yuzhao Heng et al. «ProgGen: Generating Named Entity Recognition Datasets Step-by-step with Self-Reflexive Large Language Models». In: *Findings of the Association for Computational Linguistics ACL 2024*. Findings of the Association for Computational Linguistics ACL 2024. Bangkok, Thailand e virtual meeting: Association for Computational Linguistics, 2024, pp. 15992–16030. DOI: 10.18653/v1/2024.findings-acl.947. URL: <https://aclanthology.org/2024.findings-acl.947> (visitato il giorno 13/06/2026).

- [46] Anthony Yazdani, Ihor Stepanov e Douglas Teodoro. *GLiNER-BioMed: A Suite of Efficient Models for Open Biomedical Named Entity Recognition*. 20 Mag. 2025. DOI: 10.48550/arXiv.2504.00676. arXiv: 2504.00676 [cs.CL]. URL: <http://arxiv.org/abs/2504.00676> (visitato il giorno 24/06/2026). Pre-pubblicato.
- [47] Saadullah Amin et al. «Few-Shot Cross-lingual Transfer for Coarse-grained De-identification of Code-Mixed Clinical Texts». In: *Proceedings of the 21st Workshop on Biomedical Language Processing*. Proceedings of the 21st Workshop on Biomedical Language Processing. Dublin, Ireland: Association for Computational Linguistics, 2022, pp. 200–211. DOI: 10.18653/v1/2022.bionlp-1.20. URL: <https://aclanthology.org/2022.bionlp-1.20> (visitato il giorno 13/06/2026).
- [48] Joint Task Force Transformation Initiative. *Guide for Conducting Risk Assessments*. NIST SP 800-30r1. Gaithersburg, MD: National Institute of Standards and Technology, 2012, NIST SP 800–30r1. DOI: 10.6028/NIST.SP.800-30r1. URL: <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-30r1.pdf> (visitato il giorno 16/08/2026).
- [49] Alexandra Sasha Luccioni, Yacine Jernite e Emma Strubell. «Power Hungry Processing: Watts Driving the Cost of AI Deployment?» In: *The 2024 ACM Conference on Fairness, Accountability and Transparency*. 3 Giu. 2024, pp. 85–99. DOI: 10.1145/3630106.3658542. arXiv: 2311.16863 [cs.LG]. URL: <http://arxiv.org/abs/2311.16863> (visitato il giorno 19/06/2026).
- [50] Hao Yu et al. *Open, Closed, or Small Language Models for Text Classification?* 19 Ago. 2023. DOI: 10.48550/arXiv.2308.10092. arXiv: 2308.10092 [cs.CL]. URL: <http://arxiv.org/abs/2308.10092> (visitato il giorno 25/06/2026). Pre-pubblicato.
- [51] Erik Johannes Husom et al. *The Price of Prompting: Profiling Energy Use in Large Language Models Inference*. 3 Mar. 2026. DOI: 10.48550/arXiv.2407.16893. arXiv: 2407.16893 [cs.CY]. URL: <http://arxiv.org/abs/2407.16893> (visitato il giorno 19/06/2026). Pre-pubblicato.
- [52] Zhenyan Lu et al. *Small Language Models: Survey, Measurements, and Insights*. 26 Feb. 2025. DOI: 10.48550/arXiv.2409.15790. arXiv: 2409.15790 [cs.CL]. URL: <http://arxiv.org/abs/2409.15790> (visitato il giorno 18/07/2026). Pre-pubblicato.
- [53] Qingyu Song et al. *A Systematic Evaluation of On-Device LLMs: Quantization, Performance, and Resources*. 16 Mar. 2026. DOI: 10.48550/arXiv.2505.15030. arXiv: 2505.15030 [cs.LG]. URL: <http://arxiv.org/abs/2505.15030> (visitato il giorno 18/07/2026). Pre-pubblicato.
- [54] Kyrylo S Malakhov. «Deploying LLMs on CPU-only Environments with Llama.Cpp Library Set: MedLocalGPT Project Case». In: ().

- [55] Uygar Kurt. *Which Quantization Should I Use? A Unified Evaluation of Llama.Cpp Quantization on Llama-3.1-8B-Instruct*. 11 Gen. 2026. DOI: 10.48550/arXiv.2601.14277. arXiv: 2601.14277 [cs.LG]. URL: <http://arxiv.org/abs/2601.14277> (visitato il giorno 18/07/2026). Pre-pubblicato.
- [56] Sönke Tenckhoff, Mario Koddenbrock e Erik Rodner. *LLMStructBench: Benchmarking Large Language Model Structured Data Extraction*. 16 Feb. 2026. DOI: 10.48550/arXiv.2602.14743. arXiv: 2602.14743 [cs.CL]. URL: <http://arxiv.org/abs/2602.14743> (visitato il giorno 18/07/2026). Pre-pubblicato.
- [57] Nikolai Skripko. *Instruction-Following Evaluation in Function Calling for Large Language Models*. 22 Set. 2025. DOI: 10.48550/arXiv.2509.18420. arXiv: 2509.18420 [cs.AI]. URL: <http://arxiv.org/abs/2509.18420> (visitato il giorno 18/07/2026). Pre-pubblicato.
- [58] *Ibm-Granite/Granite-3.0-Language-Models*. IBM Granite, 17 lug. 2026. URL: <https://github.com/ibm-granite/granite-3.0-language-models> (visitato il giorno 18/07/2026).
- [59] Gemma Team et al. *Gemma 3 Technical Report*. 25 Mar. 2025. DOI: 10.48550/arXiv.2503.19786. arXiv: 2503.19786 [cs.CL]. URL: <http://arxiv.org/abs/2503.19786> (visitato il giorno 18/07/2026). Pre-pubblicato.
- [60] *Meta-Llama/Llama-3.2-1B-Instruct* · Hugging Face. 6 Dic. 2024. URL: <https://huggingface.co/meta-llama/Llama-3.2-1B-Instruct> (visitato il giorno 18/07/2026).
- [61] Josip Tomo Licardo e Nikola Tankovic. *Performance Trade-offs of Optimizing Small Language Models for E-Commerce*. 24 Ott. 2025. DOI: 10.48550/arXiv.2510.21970. arXiv: 2510.21970 [cs.AI]. URL: <http://arxiv.org/abs/2510.21970> (visitato il giorno 18/07/2026). Pre-pubblicato.
- [62] Haolin Zhang e Jeff Huang. *Challenging GPU Dominance: When CPUs Outperform for On-Device LLM Inference*. 9 Mag. 2025. DOI: 10.48550/arXiv.2505.06461. arXiv: 2505.06461 [cs.DC]. URL: <http://arxiv.org/abs/2505.06461> (visitato il giorno 18/07/2026). Pre-pubblicato.
- [63] Agatsya Yadav e Renta Chintala Bhargavi. «Optimizing LLMs Using Quantization for Mobile Execution». In: vol. 1654. 2026, pp. 330–339. DOI: 10.1007/978-3-032-06697-8_33. arXiv: 2512.06490 [cs.LG]. URL: <http://arxiv.org/abs/2512.06490> (visitato il giorno 18/07/2026).
- [64] Qwen et al. *Qwen2.5 Technical Report*. 3 Gen. 2025. DOI: 10.48550/arXiv.2412.15115. arXiv: 2412.15115 [cs.CL]. URL: <http://arxiv.org/abs/2412.15115> (visitato il giorno 18/07/2026). Pre-pubblicato.

- [65] An Yang et al. *Qwen2 Technical Report*. 10 Set. 2024. DOI: 10.48550/arXiv.2407.10671. arXiv: 2407.10671 [cs.CL]. URL: <http://arxiv.org/abs/2407.10671> (visitato il giorno 18/07/2026). Pre-pubblicato.
- [66] Marah Abdin et al. *Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone*. 30 Ago. 2024. DOI: 10.48550/arXiv.2404.14219. arXiv: 2404.14219 [cs.CL]. URL: <http://arxiv.org/abs/2404.14219> (visitato il giorno 18/07/2026). Pre-pubblicato.
- [67] Microsoft et al. *Phi-4-Mini Technical Report: Compact yet Powerful Multimodal Language Models via Mixture-of-LoRAs*. 7 Mar. 2025. DOI: 10.48550/arXiv.2503.01743. arXiv: 2503.01743 [cs.CL]. URL: <http://arxiv.org/abs/2503.01743> (visitato il giorno 18/07/2026). Pre-pubblicato.
- [68] Loubna Ben Allal et al. *SmolLM2: When Smol Goes Big – Data-Centric Training of a Small Language Model*. 4 Feb. 2025. DOI: 10.48550/arXiv.2502.02737. arXiv: 2502.02737 [cs.CL]. URL: <http://arxiv.org/abs/2502.02737> (visitato il giorno 18/07/2026). Pre-pubblicato.
- [69] Cosimo Galeone et al. *When Correct Isn't Usable: Improving Structured Output Reliability in Small Language Models*. 4 Mag. 2026. DOI: 10.48550/arXiv.2605.02363. arXiv: 2605.02363 [cs.CL]. URL: <http://arxiv.org/abs/2605.02363> (visitato il giorno 18/07/2026). Pre-pubblicato.
- [70] Md Romyull Islam et al. *Characterizing and Understanding Energy Footprint and Efficiency of Small Language Model on Edges*. 7 Nov. 2025. DOI: 10.48550/arXiv.2511.11624. arXiv: 2511.11624 [cs.DC]. URL: <http://arxiv.org/abs/2511.11624> (visitato il giorno 18/07/2026). Pre-pubblicato.
- [71] Michiel Overeem et al. «An Empirical Characterization of Event Sourced Systems and Their Schema Evolution – Lessons from Industry». In: *Journal of Systems and Software* 178 (ago. 2021), p. 110970. ISSN: 01641212. DOI: 10.1016/j.jss.2021.110970. arXiv: 2104.01146 [cs.SE]. URL: <http://arxiv.org/abs/2104.01146> (visitato il giorno 09/07/2026).
- [72] Greg Young. «CQRS Documents by Greg Young». In: ().